Learn CFD

A Practical Introduction to Computational Fluid Dynamics

Almajd Alhinai

July 2, 2026

Contents

Part I

Mathematical Fundamentals

Chapter 1

introduction

Mathematics is the language of fluid mechanics. Every CFD simulation, regardless of its complexity, is ultimately based on mathematical equations that describe how physical quantities such as velocity, pressure, temperature, and density vary in space and time. Before studying the governing equations of fluid flow and the numerical methods used to solve them, it is essential to understand the mathematical foundations upon which they are built.

This chapter introduces differential equations as the primary tools used to model physical systems. Beginning with a simple first-principles derivation of pipe flow, we demonstrate how ordinary differential equations arise naturally from fundamental physical laws and how progressively relaxing simplifying assumptions leads to partial differential equations. The chapter then introduces the conservation laws of mass, momentum, and energy, which form the basis of all fluid-flow models. Finally, we discuss boundary and initial conditions, classify common types of partial differential equations, and compare analytical and numerical solution techniques. Together, these topics establish the mathematical framework that underpins Computational Fluid Dynamics and prepares the reader for the development of the governing fluid-flow equations in subsequent chapters.

After completing this chapter, the reader should be able to:

- distinguish between ordinary and partial differential equations;
- derive a governing equation from first physical principles;
- explain how conservation laws lead to the governing equations of fluid flow;
- identify and apply appropriate boundary and initial conditions;
- classify PDEs as elliptic, parabolic, or hyperbolic;
- distinguish between analytical and numerical solution methods;
- understand the mathematical foundations underlying CFD.

1.1 What is a Differential Equation?

Differential equations form the mathematical language used to describe physical phenomena involving change. In fluid mechanics, they arise naturally from conservation principles and constitutive relations.

1.1.1 Definitions

A differential equation is an equation that relates an unknown function to one or more of its derivatives.

A general ordinary differential equation (ODE) can be written as

$$F\left(x, y, \frac{dy}{dx}, \frac{d^2y}{dx^2}, \dots\right) = 0. \tag{1.1}$$

Similarly, a partial differential equation (PDE) takes the form

$$F\left(x, t, u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial t}, \dots\right) = 0. \tag{1.2}$$

1.1.2 ODEs vs PDEs

An ordinary differential equation contains derivatives with respect to a single independent variable.

Example:

$$\frac{dT}{dt} = -k(T - T_\infty). \tag{1.3}$$

A partial differential equation contains derivatives with respect to two or more independent variables.

Example:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}. \tag{1.4}$$

1.2 A First-Principles Example: Laminar Flow Through a Circular Pipe

One of the most natural ways to introduce ordinary differential equations (ODEs) and partial differential equations (PDEs) is through fluid mechanics. Rather than presenting differential equations as abstract mathematical objects, they can be derived directly from physical principles, showing how increasingly realistic descriptions of fluid flow naturally lead from ODEs to PDEs.

Consider a long, straight circular pipe of radius R carrying a viscous fluid. We make the following simplifying assumptions:

- the fluid is incompressible,
- the fluid is Newtonian,
- the flow is steady,
- the flow is axisymmetric,
- the flow is fully developed.

Because the flow is fully developed, the velocity has only one non-zero component in the axial direction and depends only on the radial coordinate,

$$u = u(r).$$

Since the velocity depends on only one independent variable, any governing equation will be an ordinary differential equation.

1.2.1 Applying Newton's Second Law

Consider a cylindrical element of fluid with radius r, thickness dr, and length L. Under steady conditions, the element is in equilibrium, so Newton's second law reduces to

$$\sum F = 0.$$

There are two forces acting on the element.

The first is the pressure force resulting from the pressure gradient along the pipe,

$$F_p = -\frac{dp}{dx}\,\pi r^2 L.$$

The second is the viscous shear force acting on the cylindrical surface. For a Newtonian fluid,

$$\tau = \mu \frac{du}{dr},$$

where μ is the dynamic viscosity.

Balancing pressure and viscous forces leads to

$$\frac{d}{dr}(r\tau) = -\frac{r}{2}\frac{dp}{dx}.$$

Substituting Newton's law of viscosity gives

$$\boxed{\frac{1}{r}\frac{d}{dr}\left(r\frac{du}{dr}\right) = \frac{1}{\mu}\frac{dp}{dx}}$$

This is a second-order ordinary differential equation obtained directly from Newton's second law and the constitutive relation for a Newtonian fluid.

1.2.2 Solving the ODE

Multiplying through by r,

$$\frac{d}{dr}\left(r\frac{du}{dr}\right) = \frac{r}{\mu}\frac{dp}{dx},$$

and integrating,

$$r\frac{du}{dr} = \frac{1}{2\mu}\frac{dp}{dx}r^2 + C_1.$$

Hence,

$$\frac{du}{dr} = \frac{1}{2\mu}\frac{dp}{dx}r + \frac{C_1}{r}.$$

The velocity gradient must remain finite at the centreline ($r = 0$), requiring

$$C_1 = 0.$$

Integrating again,

$$u(r) = \frac{1}{4\mu}\frac{dp}{dx}r^2 + C_2.$$

Applying the no-slip boundary condition at the pipe wall,

$$u(R) = 0,$$

gives

$$C_2 = -\frac{1}{4\mu}\frac{dp}{dx}R^2.$$

The velocity distribution is therefore

$$\boxed{u(r) = -\frac{1}{4\mu}\frac{dp}{dx}\left(R^2 - r^2\right).}$$

This parabolic velocity profile is the classical Hagen–Poiseuille solution. The maximum velocity occurs at the pipe centreline, while the velocity falls smoothly to zero at the wall due to the no-slip condition.

This example illustrates the essential ingredients of mathematical modelling in fluid mechanics:

- conservation of momentum (Newton's second law),
- a constitutive law (Newton's law of viscosity),
- boundary conditions,
- and the solution of the resulting differential equation.

Importantly, the governing equation is an ordinary differential equation because the velocity depends on only one independent variable.

1.3 From ODEs to PDEs

The preceding example also provides a natural route to partial differential equations. The governing physical laws do not change; only the assumptions about the flow become less restrictive.

1.3.1 Allowing Variation Along the Pipe

Suppose the flow is no longer fully developed. The velocity now depends on both the radial position and the axial coordinate,

$$u = u(r, x).$$

Since there are now two independent variables, spatial derivatives appear in both directions,

$$\frac{\partial u}{\partial r}, \qquad \frac{\partial u}{\partial x},$$

and the governing equation becomes a partial differential equation. The underlying conservation of momentum remains the same, but the mathematics must now describe changes in two spatial directions.

1.3.2 Introducing Time Dependence

If the flow is also allowed to vary with time,

$$u = u(r, t),$$

Newton's second law must include the fluid acceleration,

$$\rho\frac{\partial u}{\partial t} = -\frac{\partial p}{\partial x} + \mu\left(\frac{1}{r}\frac{\partial}{\partial r}\left(r\frac{\partial u}{\partial r}\right)\right).$$

The appearance of the time derivative transforms the governing equation into a PDE involving both space and time. Physically, this additional term represents the rate at which the fluid velocity changes with time.

1.3.3 Extending to Two Spatial Dimensions

Many practical flows vary in more than one spatial direction. For example, fluid flowing over a flat plate depends on both the streamwise and normal coordinates,

$$u = u(x, y).$$

The viscous diffusion of momentum is then described by

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2},$$

while fluid motion introduces nonlinear advection terms such as

$$u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y}.$$

The resulting governing equations are multidimensional partial differential equations that capture both transport and diffusion of momentum.

1.3.4 The Full Navier–Stokes Equations

Finally, removing nearly all simplifying assumptions allows the velocity to vary in three spatial dimensions and time,

$$\mathbf{u} = \mathbf{u}(x, y, z, t),$$

with pressure

$$p = p(x, y, z, t).$$

Conservation of momentum becomes

$$\rho\left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u}\right) = -\nabla p + \mu\nabla^2\mathbf{u},$$

subject to the incompressibility condition

$$\nabla\cdot\mathbf{u} = 0.$$

These are the Navier–Stokes equations, which describe the motion of incompressible Newtonian fluids in their most general form.

1.4 The Conceptual Progression

This sequence demonstrates that ordinary and partial differential equations are not fundamentally different mathematical objects. Both arise from the same conservation laws. The distinction lies in the number of independent variables required to describe the physical system.

For the pipe-flow example,

- $u = u(r)$ leads to an ordinary differential equation because the velocity depends on one independent variable.

- $u = u(r, x)$ or $u = u(r, t)$ leads to a partial differential equation because the velocity depends on two independent variables.
- $u = u(x, y, z, t)$ leads to the full Navier–Stokes equations describing realistic fluid motion.

Thus, the ODE governing Hagen–Poiseuille flow should be viewed not as an isolated example, but as a highly simplified case of the general momentum equations. By progressively relaxing simplifying assumptions, students can see how increasingly complex physical descriptions naturally lead from ordinary differential equations to partial differential equations, providing a coherent and physically motivated introduction to mathematical modelling in fluid mechanics.

1.5 Conservation Laws

The governing equations used in fluid mechanics and CFD are all derived from fundamental conservation principles. These principles are independent of the particular fluid or geometry being studied and provide the foundation upon which the Navier–Stokes equations are built.

The three most important conservation laws are:

1. Conservation of Mass
2. Conservation of Momentum
3. Conservation of Energy

1.5.1 Conservation of Mass

The law of conservation of mass states that mass can neither be created nor destroyed. For any control volume, the rate of mass accumulation must equal the difference between the mass entering and leaving the control volume.

For a general compressible flow, the differential form of the continuity equation is

$$\boxed{\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0} \tag{1.5}$$

where ρ is the fluid density and $\mathbf{u}$ is the velocity vector.

For incompressible flows, the density is constant and the continuity equation simplifies to

$$\boxed{\nabla \cdot \mathbf{u} = 0.} \tag{1.6}$$

Physically, this condition states that fluid cannot accumulate or disappear within a region of the flow.

1.5.2 Conservation of Momentum

The conservation of momentum is a direct application of Newton's second law:

$$\text{Force} = \text{Rate of change of momentum.}$$

For an incompressible Newtonian fluid, momentum conservation leads to the Navier–Stokes equations,

$$\boxed{\rho \left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} \right) = -\nabla p + \mu \nabla^2 \mathbf{u} + \rho \mathbf{g}.} \tag{1.7}$$

Each term has a physical interpretation:

- $\rho\dfrac{\partial \mathbf{u}}{\partial t}$: local acceleration,
- $\rho(\mathbf{u}\cdot\nabla)\mathbf{u}$: convective acceleration,
- $-\nabla p$: pressure forces,
- $\mu\nabla^2\mathbf{u}$: viscous diffusion,
- $\rho\mathbf{g}$: body forces such as gravity.

The Navier–Stokes equations are the primary equations solved in Computational Fluid Dynamics.

1.5.3 Conservation of Energy

The conservation of energy states that energy cannot be created or destroyed; it can only be transferred or transformed.

For heat transfer problems, the governing equation often takes the form

$$\boxed{\rho c_p\left(\frac{\partial T}{\partial t}+\mathbf{u}\cdot\nabla T\right)=k\nabla^2 T+\dot{q},} \tag{1.8}$$

where

- T is temperature,
- c_p is the specific heat capacity,
- k is the thermal conductivity,
- $\dot{q}$ is the volumetric heat-generation rate.

This equation describes the combined effects of advection, diffusion, and heat generation within the fluid.

1.6 Boundary and Initial Conditions

Differential equations alone do not provide a unique solution. Additional information must be supplied to define the physical problem completely.

1.6.1 Boundary Conditions

Boundary conditions specify the behaviour of the solution at the boundaries of the domain.

Three common types occur frequently in engineering problems.

Dirichlet Boundary Conditions

A Dirichlet condition specifies the value of a variable directly.

For example,

$$T = 300\ \text{K}$$

on a wall with fixed temperature.

Similarly, the no-slip condition for fluid flow specifies

$$u = 0$$

at a stationary solid boundary.

Neumann Boundary Conditions

A Neumann condition specifies the gradient of a variable.

For example,

$$\frac{\partial T}{\partial n} = 0$$

represents an insulated wall because no heat crosses the boundary.

Robin Boundary Conditions

Robin conditions involve a combination of the variable and its gradient.

A common example is convective heat transfer,

$$-k\frac{\partial T}{\partial n} = h(T - T_\infty),$$

where h is the heat-transfer coefficient.

1.6.2 Initial Conditions

For transient problems, the state of the system must be known at the initial time.

For example,

$$T(x, 0) = T_0(x)$$

specifies the temperature distribution at the start of a simulation.

Without appropriate initial and boundary conditions, a differential equation usually admits infinitely many possible solutions.

1.7 Classification of Partial Differential Equations

Second-order partial differential equations are commonly classified according to their mathematical properties. This classification provides insight into the physical behaviour of the system and influences the numerical methods used for solution.

1.7.1 Elliptic Equations

An important example is Laplace's equation,

$$\boxed{\nabla^2\phi = 0.} \tag{1.9}$$

Elliptic equations typically describe equilibrium or steady-state behaviour.

Examples include:

- steady heat conduction,
- electrostatics,
- incompressible pressure correction equations.

A characteristic feature of elliptic equations is that information propagates throughout the domain. Changes to a boundary condition can influence the solution everywhere.

1.7.2 Parabolic Equations

The classical diffusion equation is

$$\boxed{\frac{\partial T}{\partial t} = \alpha \nabla^2 T.} \tag{1.10}$$

Parabolic equations describe transport processes dominated by diffusion.
Examples include:

- transient heat conduction,
- species diffusion,
- viscous momentum diffusion.

These equations require both initial conditions and boundary conditions.

1.7.3 Hyperbolic Equations

The one-dimensional wave equation is

$$\boxed{\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}.} \tag{1.11}$$

Hyperbolic equations describe wave-like phenomena.
Examples include:

- acoustic waves,
- shock waves,
- compressible flow phenomena.

Unlike elliptic equations, disturbances travel at finite speeds along characteristic paths.

1.8 Analytical and Numerical Solution Methods

Once a governing differential equation has been derived, the next task is to obtain its solution.

1.8.1 Analytical Solutions

An analytical solution is an exact mathematical expression satisfying both the governing equation and the associated boundary conditions.
Examples include:

- Hagen–Poiseuille pipe flow,
- Couette flow,
- Laplace's equation in simple geometries,
- one-dimensional heat conduction problems.

Analytical solutions provide valuable physical insight and are often used to verify numerical methods.

Unfortunately, they are only available for relatively simple problems involving idealised geometries and simplifying assumptions.

1.8.2 Numerical Solutions

Most practical engineering problems involve complex geometries, turbulence, moving boundaries, chemical reactions, or multiphase flows. Such problems rarely admit analytical solutions.

Numerical methods overcome this limitation by replacing differential equations with systems of algebraic equations that can be solved using computers.

Common approaches include:

- Finite Difference Method (FDM),
- Finite Volume Method (FVM),
- Finite Element Method (FEM).

The central idea is to discretise the physical domain into a finite number of points, cells, or elements and approximate the derivatives numerically.

1.8.3 Why CFD Exists

The Navier–Stokes equations provide a complete mathematical description of fluid motion, but their nonlinear nature makes exact solutions possible only for a limited set of simple flows.

Computational Fluid Dynamics (CFD) addresses this challenge by applying numerical methods to the governing conservation equations. Modern CFD software solves discretised versions of the continuity, momentum, and energy equations to predict fluid-flow behaviour in realistic engineering systems.

1.9 Chapter Summary

In this chapter, differential equations were introduced as the mathematical framework used to describe physical systems. Beginning with the classical Hagen–Poiseuille pipe-flow problem, we demonstrated how conservation laws lead naturally to an ordinary differential equation and how progressively relaxing simplifying assumptions produces partial differential equations.

The fundamental conservation laws of mass, momentum, and energy were then introduced, culminating in the Navier–Stokes equations that form the basis of Computational Fluid Dynamics. Finally, the importance of boundary and initial conditions, the classification of partial differential equations, and the distinction between analytical and numerical solution methods were discussed.

The next chapter builds directly on these mathematical foundations by developing the governing equations of fluid flow in a systematic and general manner.

Chapter 2

Fundamental Partial Differential Equations

Partial differential equations arise whenever a physical quantity depends on both space and time. In the previous chapter, we saw how ordinary differential equations emerge from highly simplified physical models and how relaxing those assumptions naturally leads to partial differential equations. We also introduced the classification of PDEs and their role in describing physical systems.

Among the vast number of PDEs encountered in science and engineering, a small set of model equations appears repeatedly because they represent fundamental physical mechanisms. These mechanisms govern the transport of mass, momentum, energy, and other conserved quantities. Three processes are particularly important:

- Diffusion, which smooths spatial variations and reduces gradients,
- Wave propagation, which transmits disturbances through a medium,
- Convection, which transports quantities from one location to another.

Understanding these elementary PDEs provides valuable insight into more complex governing equations. In particular, many terms appearing in the Navier–Stokes equations can be interpreted as combinations of diffusion and convection processes. This chapter therefore examines several fundamental PDEs that serve as building blocks for fluid mechanics and Computational Fluid Dynamics.

2.1 The Heat Equation

The heat equation is one of the most important equations in applied mathematics. It describes diffusion, a process through which gradients are progressively reduced over time.

2.1.1 Derivation from Heat Conduction

Consider one-dimensional heat conduction in a stationary solid. Applying conservation of energy to a differential element and using Fourier's law of conduction gives

$$\rho c_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2}. \tag{2.1}$$

Defining the thermal diffusivity

$$\alpha = \frac{k}{\rho c_p}, \tag{2.2}$$

the governing equation becomes

$$\boxed{\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}.} \tag{2.3}$$

More generally, replacing temperature by a quantity $u(x,t)$ gives

$$\boxed{\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2}.} \tag{2.4}$$

where ν represents a diffusion coefficient.

2.1.2 Physical Interpretation

The second derivative measures local curvature. Regions with high positive curvature tend to gain the transported quantity, while regions with high negative curvature tend to lose it.

Consequently, sharp gradients gradually disappear and the solution becomes increasingly smooth.

An intuitive example is the diffusion of a drop of dye released into still water. Initially the concentration is highly localized, but diffusion causes the dye to spread until a uniform concentration is achieved.

2.1.3 Properties of Diffusion

The heat equation possesses several important properties:

- Diffusion smooths irregularities.
- Peaks decrease with time.
- Gradients become weaker.
- The solution tends toward equilibrium.

Unlike wave propagation, diffusion does not preserve the original shape of a disturbance. Instead, information spreads and becomes increasingly diffuse.

2.2 The Wave Equation

Whereas diffusion dissipates structure, the wave equation transmits disturbances while preserving their essential form.

2.2.1 Derivation

Consider the transverse vibration of a stretched string. Applying Newton's second law to a small element of the string yields

$$\boxed{\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2},} \tag{2.5}$$

where

- $u(x,t)$ is the displacement,
- c is the wave speed.

This equation also appears in acoustics, structural vibrations, electromagnetics, and many other fields.

2.2.2 General Solution

The one-dimensional wave equation admits the classical solution

$$\boxed{u(x,t) = f(x-ct) + g(x+ct),} \tag{2.6}$$

where

- $f(x-ct)$ represents a wave travelling in the positive x direction,
- $g(x+ct)$ represents a wave travelling in the negative x direction.

The solution demonstrates that disturbances propagate without changing shape.

2.2.3 Physical Interpretation

If a pulse is introduced into a stretched string, the pulse travels through the medium while retaining its general profile.

Unlike diffusion:

- the disturbance does not spread,
- the amplitude is not automatically reduced,
- information propagates at a finite speed.

Wave propagation therefore preserves structure rather than dissipating it.

2.3 Linear Convection

The convection equation describes the transport of a quantity by motion.

2.3.1 Governing Equation

The simplest convection equation is

$$\boxed{\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0,} \tag{2.7}$$

where c is a constant transport velocity.

2.3.2 Exact Solution

The solution is

$$\boxed{u(x,t) = f(x-ct).} \tag{2.8}$$

Thus the entire profile moves at speed c without changing its shape.

2.3.3 Physical Interpretation

Suppose a pollutant is carried downstream by a river with constant velocity.

The pollutant concentration profile remains unchanged while being transported downstream.

Consequently:

- there is no diffusion,
- there is no distortion,
- there is no smoothing,
- the profile simply translates.

Linear convection therefore represents pure transport.

2.4 Nonlinear Convection

The assumption of constant transport velocity is often unrealistic.

In many physical systems the transport speed depends upon the solution itself.

2.4.1 Governing Equation

The simplest nonlinear convection equation is

$$\boxed{\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0.} \tag{2.9}$$

Unlike the linear convection equation, the wave speed is now equal to the local value of the solution.

2.4.2 Characteristics

For linear convection, all portions of the solution move with the same speed.

For nonlinear convection,

$$c = u, \tag{2.10}$$

so different parts of the solution travel at different speeds.

Regions where u is large move faster than regions where u is small.

2.4.3 Solution Distortion

Because different parts of the solution move at different speeds, the profile gradually changes shape.

Initially smooth distributions become increasingly steep.

This behaviour is absent from linear convection and is a hallmark of nonlinear transport phenomena.

2.4.4 Shock Formation

As steepening continues, neighbouring characteristics may intersect.

At this point the solution develops a discontinuity known as a shock.

Mathematically, the gradient becomes extremely large,

$$\left|\frac{\partial u}{\partial x}\right| \to \infty.$$

Classical solutions cease to exist and special mathematical techniques are required to continue the solution.

Shock waves in compressible fluid mechanics arise from precisely this mechanism.

2.5 Burgers' Equation

Burgers' equation combines nonlinear convection and diffusion in a single model.

For this reason it is often regarded as a simplified analogue of the Navier–Stokes equations.

2.5.1 Governing Equation

The equation takes the form

$$\boxed{\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}.} \tag{2.11}$$

The left-hand side represents nonlinear transport, while the right-hand side represents diffusion.

2.5.2 Competing Physical Mechanisms

Two competing effects are present:

1. Nonlinear convection tends to steepen gradients.
2. Diffusion tends to smooth gradients.

The resulting solution depends on which effect dominates.

2.5.3 Behaviour

If diffusion is weak, the solution develops very sharp gradients.

If diffusion is sufficiently strong, these gradients are smoothed before a discontinuity can form.

Instead of an abrupt shock, the solution develops a thin but continuous transition layer.

This balance between steepening and smoothing closely resembles many phenomena encountered in fluid mechanics.

2.5.4 Relation to CFD

Many important features of the Navier–Stokes equations are already present in Burgers' equation:

- nonlinear advection,
- diffusion,

- steepening of gradients,
- smoothing by viscosity.

For this reason Burgers' equation is frequently used as a test problem when developing numerical methods for CFD.

2.6 Numerical Considerations

The behaviour of a differential equation strongly influences the numerical methods required to solve it.

2.6.1 Convection and the CFL Condition

For convection-dominated problems, numerical stability is governed by the Courant–Friedrichs–Lewy (CFL) condition,

$$\boxed{\frac{\max|u|\Delta t}{\Delta x} \leq 1.} \tag{2.12}$$

Physically, information should not travel farther than one computational cell during a single timestep.

Violating this condition typically leads to unstable numerical solutions.

2.6.2 Diffusion Stability Condition

Diffusion equations impose an additional restriction on the timestep,

$$\boxed{\frac{\nu\Delta t}{\Delta x^2} \leq \frac{1}{2}.} \tag{2.13}$$

This condition is often more restrictive than the CFL condition when diffusion dominates the solution.

2.7 A Unified Perspective

The equations studied in this chapter can be viewed as a hierarchy of increasing complexity.

Linear Convection → Nonlinear Convection → Burgers' Equation → Navier–Stokes Equations

Each equation introduces an additional physical mechanism:

- Linear convection introduces transport.
- Nonlinear convection introduces self-interaction and steepening.
- Burgers' equation introduces viscous diffusion.
- Navier–Stokes equations extend these ideas to multidimensional fluid motion.

Similarly, the heat and wave equations represent two distinct classes of physical behaviour:

- diffusion destroys structure,
- wave propagation preserves structure.

Understanding these fundamental equations provides valuable intuition for interpreting the behaviour of more complex fluid-flow models.

2.8 Chapter Summary

This chapter introduced several fundamental partial differential equations that describe the basic physical mechanisms encountered throughout science and engineering. The heat equation was shown to model diffusion and the smoothing of gradients, while the wave equation describes the propagation of disturbances at finite speed. Linear convection illustrated pure transport, whereas nonlinear convection demonstrated how self-advection can lead to solution distortion and shock formation.

These concepts were unified through Burgers' equation, which combines nonlinear convection and diffusion and serves as a simplified model for many features of fluid mechanics. Finally, numerical stability considerations were introduced through the CFL condition and diffusion stability criterion.

Together, these model equations provide the conceptual foundation for understanding the governing equations of fluid flow and the numerical methods used to solve them. In the next chapter, we build upon these ideas to develop the conservation equations that form the basis of Computational Fluid Dynamics.

Chapter 3

Two-Dimensional Partial Differential Equations

In the previous chapter, we studied several fundamental partial differential equations, including the heat equation, the wave equation, the linear convection equation, and Burgers' equation. These model equations introduced the key physical mechanisms of diffusion, propagation, and transport, while also providing insight into the effects of nonlinearity and viscosity. However, all of these examples were formulated in a single spatial dimension. While one-dimensional models are invaluable for developing intuition, most physical systems occur in two or three spatial dimensions.

Fluid flow provides a natural example of this limitation. The motion of a fluid generally depends not only on position along a single coordinate, but also on variations in multiple directions. Velocity, pressure, temperature, and concentration are therefore best described as fields defined over a spatial domain. Extending the governing equations from one to two dimensions fundamentally changes both their mathematical structure and their numerical treatment. Additional spatial derivatives must be considered, transport can occur in multiple directions simultaneously, and scalar equations often become coupled systems of equations.

This chapter develops the extension of the model equations introduced previously to two spatial dimensions. Beginning with the concept of two-dimensional fields, we formulate two-dimensional versions of the convection, diffusion, and Burgers' equations and examine their physical interpretation. We then introduce the mathematical framework used for finite-difference discretisation on Cartesian grids and discuss the role of boundary conditions and numerical stability. Finally, we demonstrate how these two-dimensional model equations relate directly to the incompressible Navier–Stokes equations, revealing the path from simple PDEs to the governing equations that underpin Computational Fluid Dynamics.

3.1 Two-Dimensional Fields

In one-dimensional problems, a dependent variable is typically represented as

$$u = u(x, t),$$

where the solution varies with one spatial coordinate and time.
In two dimensions, the dependent variable becomes

$$u = u(x, y, t),$$

and is therefore defined at every point within a two-dimensional spatial domain.
The solution is no longer a curve but a field. Examples include:

- temperature fields,
- concentration fields,
- pressure fields,
- velocity fields.

For fluid-flow problems, the velocity field contains two components,

$$\mathbf{u}(x, y, t) = \begin{bmatrix} u(x, y, t) \\ v(x, y, t) \end{bmatrix}, \tag{3.1}$$

where:

- u is the velocity in the x-direction,
- v is the velocity in the y-direction.

Numerically, a two-dimensional field is represented on a grid of points,

$$(x_i, y_j),$$

and stored as a two-dimensional array.

3.2 Linear Convection in Two Dimensions

3.2.1 Governing Equation

The one-dimensional linear convection equation was

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0.$$

In two dimensions, transport may occur in both coordinate directions.
The governing equation becomes

$$\boxed{\frac{\partial u}{\partial t} + c_x\frac{\partial u}{\partial x} + c_y\frac{\partial u}{\partial y} = 0,} \tag{3.2}$$

where c_x and c_y are constant transport velocities.

3.2.2 Physical Interpretation

This equation describes pure transport.
A disturbance introduced into the domain is carried by the velocity field along the direction

$$(c_x, c_y).$$

Unlike diffusion,

- the shape is preserved,
- gradients are unchanged,
- no smoothing occurs.

Instead, the solution translates through the domain.
For example, if

$$c_x = c_y,$$

the solution propagates diagonally across the computational domain.

3.2.3 Finite-Difference Discretisation

Using a forward difference in time and backward differences in space,

$$\frac{u_{i,j}^{n+1} - u_{i,j}^{n}}{\Delta t} + c_x \frac{u_{i,j}^{n} - u_{i-1,j}^{n}}{\Delta x} + c_y \frac{u_{i,j}^{n} - u_{i,j-1}^{n}}{\Delta y} = 0. \tag{3.3}$$

Rearranging yields

$$u_{i,j}^{n+1} = u_{i,j}^{n} - c_x \frac{\Delta t}{\Delta x}\left(u_{i,j}^{n} - u_{i-1,j}^{n}\right) - c_y \frac{\Delta t}{\Delta y}\left(u_{i,j}^{n} - u_{i,j-1}^{n}\right). \tag{3.4}$$

3.3 Nonlinear Convection in Two Dimensions

3.3.1 Governing Equations

In nonlinear convection, the velocity field transports itself.

The governing equations become

$$\boxed{\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = 0,} \tag{3.5}$$

and

$$\boxed{\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = 0.} \tag{3.6}$$

The two velocity components are coupled through the nonlinear transport terms.

3.3.2 Vector Form

The governing equations may be written compactly as

$$\boxed{\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u} = \mathbf{0}.} \tag{3.7}$$

This nonlinear convective operator will appear repeatedly throughout CFD.

3.3.3 Physical Interpretation

Unlike linear convection, the transport velocity is no longer constant.

Different regions of the flow move at different speeds.

Consequently:

- the solution distorts,
- gradients become steeper,
- sharp fronts may form.

Because transport occurs simultaneously in both spatial directions, these effects are often more complex than in the one-dimensional case.

3.4 Diffusion in Two Dimensions

3.4.1 Governing Equation

The one-dimensional diffusion equation becomes

$$\boxed{\frac{\partial u}{\partial t} = \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right).} \tag{3.8}$$

Introducing the Laplacian operator,

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2},$$

allows the equation to be written as

$$\boxed{\frac{\partial u}{\partial t} = \nu \nabla^2 u.} \tag{3.9}$$

3.4.2 Physical Interpretation

Diffusion now acts in all spatial directions.

Consider a localized peak of temperature or concentration.

Rather than spreading only along one coordinate axis, the disturbance spreads radially throughout the domain.

Consequently:

- peaks flatten,
- gradients decrease,
- the solution approaches equilibrium.

3.4.3 Finite-Difference Discretisation

Using forward Euler in time and central differences in space,

$$u_{i,j}^{n+1} = u_{i,j}^n + \nu \Delta t \left[\frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} + \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2} \right]. \tag{3.10}$$

3.5 Two-Dimensional Burgers' Equations

3.5.1 Governing Equations

The two-dimensional Burgers' equations combine nonlinear convection and diffusion.

For the u component,

$$\boxed{\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right),} \tag{3.11}$$

while for the v component,

$$\boxed{\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right).} \tag{3.12}$$

3.5.2 Physical Interpretation

The equations contain two competing mechanisms:

1. nonlinear convection, which steepens gradients,
2. diffusion, which smooths gradients.

The resulting solution depends on the relative importance of these processes.

3.5.3 Connection to Fluid Mechanics

Burgers' equations contain many of the same mathematical structures that appear in the Navier–Stokes equations:

- nonlinear advection,
- viscous diffusion,
- coupled velocity components.

However, they do not yet include pressure forces or mass conservation.
For this reason, Burgers' equations are often viewed as a simplified model of viscous fluid flow.

3.6 Mathematical Formulation of Two-Dimensional PDE Solvers

3.6.1 Computational Domain

Consider a rectangular domain

$$\Omega = [0, L_x] \times [0, L_y].$$

The domain is discretised using a Cartesian grid.
Grid points are located at

$$x_i = i\Delta x, \qquad y_j = j\Delta y,$$

with

$$i = 0, \ldots, N_x - 1, \qquad j = 0, \ldots, N_y - 1.$$

The grid spacings are

$$\Delta x = \frac{L_x}{N_x - 1}, \qquad \Delta y = \frac{L_y}{N_y - 1}. \tag{3.13}$$

3.6.2 Time Discretisation

Time is divided into discrete steps,

$$t^n = n\Delta t,$$

where Δt is the timestep.
The discrete approximation of a field is written as

$$u^n_{i,j}.$$

Each value represents the numerical approximation of the solution at grid point (i, j) and time level n.

3.6.3 Numerical Solution Strategy

Most CFD algorithms follow the same general procedure:

1. initialise the solution,
2. apply boundary conditions,
3. compute spatial derivatives,
4. advance in time,
5. repeat until convergence or the final time is reached.

This framework underlies many finite-difference, finite-volume, and finite-element methods.

3.7 Boundary Conditions

A numerical solution requires boundary conditions to be specified on the domain boundary

$$\partial\Omega.$$

3.7.1 Dirichlet Boundary Conditions

A Dirichlet condition specifies the variable directly:

$$u = g(x, y, t) \qquad \text{on} \qquad \partial\Omega.$$

Examples include prescribed temperatures and fixed wall velocities.

3.7.2 Neumann Boundary Conditions

A Neumann condition specifies the normal derivative:

$$\frac{\partial u}{\partial n} = h(x, y, t) \qquad \text{on} \qquad \partial\Omega.$$

Examples include imposed heat fluxes and symmetry conditions.

3.7.3 Boundary Conditions for Vector Fields

For velocity fields, conditions must be specified for each component,

$$u = g_1(x, y, t), \qquad v = g_2(x, y, t).$$

The selection of appropriate boundary conditions depends upon the physical problem being solved.

3.8 Stability Considerations

3.8.1 The Two-Dimensional CFL Condition

For convection-dominated problems, stability requires

$$\boxed{\frac{\max|u|\Delta t}{\Delta x} + \frac{\max|v|\Delta t}{\Delta y} \leq 1.} \tag{3.14}$$

For constant convection velocities,

$$\boxed{\frac{|c_x|\Delta t}{\Delta x} + \frac{|c_y|\Delta t}{\Delta y} \leq 1.} \tag{3.15}$$

3.8.2 Diffusion Stability Criterion

For an explicit diffusion scheme,

$$\boxed{\nu \Delta t \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} \right) \leq \frac{1}{2}.} \tag{3.16}$$

This restriction becomes increasingly severe as the grid is refined.

3.8.3 Burgers' Equations

Since Burgers' equations contain both convection and diffusion, the timestep must satisfy both stability requirements.

The most restrictive criterion determines the allowable timestep.

3.9 Connection to the Navier–Stokes Equations

The incompressible Navier–Stokes equations in two dimensions are

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial x} + \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \tag{3.17}$$

and

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial y} + \nu \left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2} \right). \tag{3.18}$$

These equations are accompanied by the incompressibility constraint

$$\boxed{\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0.} \tag{3.19}$$

Comparing these equations with Burgers' equations reveals two important additions:

1. the pressure field p,
2. the incompressibility constraint.

The pressure is not merely an additional unknown. It acts as a constraint-enforcing variable that ensures the velocity field remains divergence-free. This concept is fundamental to incompressible CFD algorithms.

3.10 Chapter Summary

This chapter extended the fundamental PDEs introduced previously from one spatial dimension to two. The concepts of fields, multidimensional transport, diffusion, and coupled systems were introduced, and two-dimensional forms of the convection, diffusion, and Burgers' equations were developed.

The chapter also introduced the computational framework used by numerical PDE solvers, including Cartesian grids, finite-difference discretisation, boundary conditions, and stability restrictions. Finally, the connection between Burgers' equations and the incompressible Navier–Stokes equations was established.

The progression from one-dimensional model equations to two-dimensional fluid-flow equations represents an important conceptual step toward practical Computational Fluid Dynamics. In the next chapter, we build upon this foundation by developing the conservation laws and governing equations that form the mathematical basis of CFD.

Chapter 4

Laplace and Poisson Equations

The previous chapter introduced the extension of convection, diffusion, and Burgers' equations from one to two spatial dimensions. These equations describe the evolution of physical fields through time and form the basis of many transport phenomena encountered in fluid mechanics. They are examples of evolution equations because the solution advances from an initial condition as time progresses.

The incompressible Navier–Stokes equations introduce an additional challenge. In addition to satisfying the momentum equations, the velocity field must remain divergence-free. This requirement introduces pressure as a constraint-enforcing variable. Unlike velocity, pressure is not transported through the domain. Instead, pressure adjusts itself so that the incompressibility condition is satisfied everywhere.

To determine the pressure field, incompressible CFD solvers typically solve a Poisson equation. The Poisson equation is closely related to the Laplace equation, one of the most important elliptic partial differential equations in applied mathematics. Unlike convection and diffusion equations, elliptic equations are not evolved in time. Instead, their solutions are determined globally by the boundary conditions and, in the case of the Poisson equation, by internal source terms.

This chapter introduces the Laplace and Poisson equations, develops their finite-difference discretisations, and demonstrates their importance in incompressible CFD. Particular emphasis is placed on the pressure Poisson equation, which forms the foundation of many modern incompressible flow solvers.

4.1 Motivation

The equations considered previously, such as convection and diffusion equations, are time-dependent evolution equations.

A typical convection–diffusion equation may be written as

$$\frac{\partial u}{\partial t} = \text{convection} + \text{diffusion}. \tag{4.1}$$

Given an initial condition, the solution is advanced forward in time.

The incompressible Navier–Stokes equations introduce an additional constraint,

$$\nabla \cdot \mathbf{u} = 0, \tag{4.2}$$

which in two dimensions becomes

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0. \tag{4.3}$$

To enforce this condition, pressure is obtained from an elliptic equation of the form

$$\nabla^2 p = b. \tag{4.4}$$

Before solving the full Navier–Stokes equations, it is therefore necessary to understand the Laplace and Poisson equations.

4.2 Evolution Equations versus Elliptic Equations

The equations studied in the previous chapters evolve in time and require an initial condition.

For example, the diffusion equation is

$$\frac{\partial u}{\partial t} = \nu \nabla^2 u. \tag{4.5}$$

This equation describes how an initial field changes as time progresses.

By contrast, elliptic equations do not describe time evolution directly.

Instead, they describe fields determined by spatial relationships and boundary conditions.

Two important examples are

$$\nabla^2 p = 0, \tag{4.6}$$

and

$$\nabla^2 p = b. \tag{4.7}$$

The first is Laplace's equation and the second is Poisson's equation.

Unlike evolution equations, elliptic equations are global in nature. A change in one part of the domain influences the solution throughout the domain.

4.3 The Laplace Equation

4.3.1 Governing Equation

The two-dimensional Laplace equation is

$$\boxed{\nabla^2 p = 0.} \tag{4.8}$$

Expanding the Laplacian gives

$$\boxed{\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} = 0.} \tag{4.9}$$

The unknown variable $p(x, y)$ is a scalar field defined throughout the domain.

4.3.2 Physical Interpretation

The Laplace equation appears in numerous steady-state problems, including:

- steady heat conduction,
- electrostatic potential,
- gravitational potential,
- potential flow theory,
- pressure-like equilibrium fields.

A key property is that the field contains no internal sources or sinks.

The solution is determined entirely by the boundary conditions.

4.3.3 The Mean-Value Property

One of the most important characteristics of Laplace's equation is the mean-value property.
Each interior point assumes a value determined by its neighbouring points.
Consequently:

- local maxima cannot occur inside the domain,
- local minima cannot occur inside the domain,
- the solution remains smooth.

These properties distinguish elliptic equations from hyperbolic and parabolic equations.

4.4 Discretisation of the Laplace Equation

Consider a Cartesian grid with spacings Δx and Δy.
The field is approximated at discrete grid points,

$$p(x_i, y_j) \approx p_{i,j}.$$

Using central differences,

$$\frac{\partial^2 p}{\partial x^2} \approx \frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2}, \tag{4.10}$$

and

$$\frac{\partial^2 p}{\partial y^2} \approx \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2}. \tag{4.11}$$

Substituting into Laplace's equation gives

$$\frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2} = 0. \tag{4.12}$$

For a uniform grid,

$$\Delta x = \Delta y,$$

the equation simplifies to

$$\boxed{p_{i,j} = \frac{1}{4}\left(p_{i+1,j} + p_{i-1,j} + p_{i,j+1} + p_{i,j-1}\right).} \tag{4.13}$$

Thus, each interior point is simply the average of its four nearest neighbours.

4.5 Iterative Solution of the Laplace Equation

The Laplace equation is typically solved iteratively.
Starting from an initial guess, the solution is repeatedly updated until convergence.
Let k denote the iteration index.
The update equation becomes

$$p_{i,j}^{k+1} = \frac{1}{4}\left(p_{i+1,j}^{k} + p_{i-1,j}^{k} + p_{i,j+1}^{k} + p_{i,j-1}^{k}\right). \tag{4.14}$$

This procedure is known as the Jacobi method.
Iterations continue until

$$\left\| p^{k+1} - p^k \right\| < \varepsilon, \tag{4.15}$$

where ε is a prescribed tolerance.

4.6 Boundary Conditions for Elliptic Problems

Because elliptic equations are boundary driven, boundary conditions play a central role.

4.6.1 Dirichlet Boundary Conditions

A Dirichlet condition prescribes the value of the solution:

$$p = g(x, y) \qquad \text{on} \qquad \partial\Omega. \tag{4.16}$$

4.6.2 Neumann Boundary Conditions

A Neumann condition prescribes the normal derivative:

$$\frac{\partial p}{\partial n} = h(x, y) \qquad \text{on} \qquad \partial\Omega. \tag{4.17}$$

The choice of boundary conditions determines the final solution.

Unlike evolution equations, the influence of the initial guess disappears once convergence is reached.

4.7 The Poisson Equation

4.7.1 Governing Equation

The two-dimensional Poisson equation is

$$\boxed{\nabla^2 p = b.} \tag{4.18}$$

Expanding the Laplacian gives

$$\boxed{\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} = b(x, y).} \tag{4.19}$$

The function $b(x, y)$ is called the source term.

4.7.2 Physical Interpretation

The Poisson equation extends Laplace's equation by including internal forcing.

While Laplace's equation describes a field with no sources or sinks, the Poisson equation accounts for source generation within the domain.

The solution is therefore determined by:

1. boundary conditions,
2. internal source terms.

4.7.3 Relationship to Laplace's Equation

If

$$b(x, y) = 0, \tag{4.20}$$

then the Poisson equation reduces to

$$\nabla^2 p = 0, \tag{4.21}$$

which is simply Laplace's equation.

Thus, Laplace's equation may be viewed as a special case of Poisson's equation.

4.8 Discretisation of the Poisson Equation

Applying central differences gives

$$\frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2} = b_{i,j}. \tag{4.22}$$

For a uniform grid,

$$\Delta x = \Delta y,$$

the update becomes

$$\boxed{p_{i,j} = \frac{1}{4}\left(p_{i+1,j} + p_{i-1,j} + p_{i,j+1} + p_{i,j-1} - b_{i,j}\Delta x^2\right).} \tag{4.23}$$

The only difference relative to the Laplace equation is the presence of the source term.

4.9 Iterative Solution of the Poisson Equation

The Poisson equation is generally solved iteratively.

A generic solution procedure consists of:

1. initialise the field,
2. impose boundary conditions,
3. update interior points,
4. reapply boundary conditions,
5. check convergence,
6. repeat until converged.

The convergence criterion may be written as

$$\left\| p^{k+1} - p^k \right\| < \varepsilon. \tag{4.24}$$

Alternatively, the residual

$$r = \nabla^2 p - b \tag{4.25}$$

can be monitored and convergence declared when

$$\|r\| < \varepsilon. \tag{4.26}$$

4.10 Why the Poisson Equation Matters in CFD

The incompressible Navier–Stokes equations are

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu \nabla^2 \mathbf{u}, \tag{4.27}$$

subject to

$$\nabla \cdot \mathbf{u} = 0. \tag{4.28}$$

The pressure field is determined by enforcing the incompressibility constraint.

This leads to a Poisson equation of the form

$$\nabla^2 p = b. \tag{4.29}$$

Thus pressure is fundamentally different from velocity.

Rather than being transported through the domain, pressure acts to enforce mass conservation.

4.11 The Pressure Poisson Equation

In projection-based methods, an intermediate velocity field is first calculated without applying a pressure correction.

This intermediate field generally does not satisfy

$$\nabla \cdot \mathbf{u} = 0.$$

A pressure correction is therefore required.

The pressure equation can be written in the form

$$\boxed{\nabla^2 p = b,} \tag{4.30}$$

where the source term depends upon the velocity field.

A commonly used expression is

$$b = \rho \left[\frac{1}{\Delta t} \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right) - \left(\frac{\partial u}{\partial x} \right)^2 - 2 \frac{\partial u}{\partial y} \frac{\partial v}{\partial x} - \left(\frac{\partial v}{\partial y} \right)^2 \right]. \tag{4.31}$$

Once the pressure field is obtained, the velocity field is corrected so that incompressibility is satisfied.

4.12 Implementation Roadmap

A practical implementation of a Laplace or Poisson solver can be organised into the following steps:

1. Define the computational domain.
2. Generate a Cartesian grid.
3. Initialise the field variable.
4. Apply boundary conditions.
5. Select Laplace or Poisson formulation.

6. Iteratively update interior points.
7. Reapply boundary conditions.
8. Check convergence.
9. Post-process and interpret the solution.

This algorithm forms the basis of many pressure solvers used in CFD.

4.13 Preparation for the Navier–Stokes Equations

The full incompressible Navier–Stokes algorithm combines convection, diffusion, pressure, and continuity.

A typical timestep consists of the following stages:

1. predict an intermediate velocity field,
2. compute the pressure source term,
3. solve the pressure Poisson equation,
4. correct the velocity field,
5. enforce incompressibility,
6. advance to the next timestep.

This sequence forms the conceptual bridge between Burgers' equations and the incompressible Navier–Stokes equations.

4.14 Chapter Summary

This chapter introduced the Laplace and Poisson equations and highlighted their importance in Computational Fluid Dynamics. Unlike convection and diffusion equations, which evolve through time, Laplace and Poisson equations are elliptic problems whose solutions are determined primarily by boundary conditions and source terms.

Finite-difference discretisations and iterative solution methods were developed, demonstrating how elliptic equations can be solved numerically. Particular attention was given to the pressure Poisson equation, which provides the mechanism through which incompressibility is enforced in CFD.

The concepts developed here complete the mathematical foundation required for pressure-based flow solvers. In the next chapter, convection, diffusion, pressure, and continuity are combined into the full incompressible Navier–Stokes equations.

Chapter 5

Incompressible Navier–Stokes Equations

The previous chapters introduced the mathematical building blocks required for Computational Fluid Dynamics. We began with differential equations and conservation laws, then developed the fundamental mechanisms of convection and diffusion, extended these concepts to two spatial dimensions, and finally introduced the Laplace and Poisson equations. Together, these topics provide the foundation required to solve realistic fluid-flow problems.

The incompressible Navier–Stokes equations combine all of these concepts into a single coupled mathematical model. The equations describe the transport of momentum by fluid motion, the diffusion of momentum by viscosity, and the action of pressure forces. Unlike the simpler model equations studied previously, the Navier–Stokes equations also impose an additional constraint: the velocity field must remain divergence-free in order to satisfy conservation of mass.

In this chapter we develop a numerical framework for solving the two-dimensional incompressible Navier–Stokes equations. The lid-driven cavity problem is used as the canonical example because it is geometrically simple while retaining all of the important physical mechanisms found in incompressible flow. Through this example, we demonstrate how convection, diffusion, pressure, and incompressibility interact within a CFD algorithm.

5.1 Governing Equations

5.1.1 Velocity Field

Consider a two-dimensional velocity field

$$\mathbf{u}(x, y, t) = \begin{bmatrix} u(x, y, t) \\ v(x, y, t) \end{bmatrix}, \tag{5.1}$$

where

- u is the velocity component in the x-direction,
- v is the velocity component in the y-direction.

5.1.2 Momentum Equations

The incompressible Navier–Stokes equations are

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial x} + \nu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right), \tag{5.2}$$

and

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial y} + \nu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right). \tag{5.3}$$

5.1.3 Continuity Equation

Conservation of mass requires that

$$\boxed{\nabla \cdot \mathbf{u} = 0.} \tag{5.4}$$

In two dimensions,

$$\boxed{\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0.} \tag{5.5}$$

This condition ensures that fluid volume is locally conserved.

5.1.4 Vector Form

The momentum equations may be written compactly as

$$\boxed{\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u} = -\frac{1}{\rho}\nabla p + \nu\nabla^2\mathbf{u}.} \tag{5.6}$$

Together with

$$\nabla \cdot \mathbf{u} = 0, \tag{5.7}$$

these equations form the incompressible Navier–Stokes system.

5.2 Physical Interpretation of Each Term

The momentum equation may be viewed as

$$\underbrace{\frac{\partial \mathbf{u}}{\partial t}}_{\text{Unsteady}} + \underbrace{(\mathbf{u}\cdot\nabla)\mathbf{u}}_{\text{Convection}} = \underbrace{-\frac{1}{\rho}\nabla p}_{\text{Pressure}} + \underbrace{\nu\nabla^2\mathbf{u}}_{\text{Diffusion}}. \tag{5.8}$$

Each term has a distinct physical interpretation.

5.2.1 Unsteady Term

The term

$$\frac{\partial \mathbf{u}}{\partial t}$$

represents the temporal variation of the velocity field.

It vanishes for steady flows.

5.2.2 Convective Term

The nonlinear term

$$(\mathbf{u}\cdot\nabla)\mathbf{u}$$

describes the transport of momentum by the fluid itself.

This is the same nonlinear convection mechanism encountered previously in Burgers' equation.

5.2.3 Pressure Gradient Term

The pressure force term

$$-\frac{1}{\rho}\nabla p$$

redistributes momentum throughout the domain.

It also plays a critical role in enforcing incompressibility.

5.2.4 Diffusion Term

The viscous term

$$\nu\nabla^2\mathbf{u}$$

acts to smooth velocity gradients.

This is the fluid-mechanics equivalent of the diffusion equation introduced earlier.

5.2.5 The Incompressibility Constraint

The continuity equation

$$\nabla \cdot \mathbf{u} = 0$$

ensures that fluid is neither created nor destroyed.

It is this condition that distinguishes incompressible flow solvers from simpler convection–diffusion models.

5.3 The Lid-Driven Cavity Problem

The lid-driven cavity is one of the most commonly studied benchmark problems in CFD.

The computational domain is a square cavity,

$$\Omega = [0, L] \times [0, L].$$

The upper wall moves horizontally with constant velocity U, while all other walls remain stationary.

Although simple, the problem contains:

- nonlinear convection,
- viscous diffusion,
- pressure coupling,
- recirculating flow.

Consequently, it serves as an ideal test case for validating incompressible flow solvers.

5.4 Boundary Conditions

5.4.1 Velocity Boundary Conditions

For the lid-driven cavity:

Top Wall

$$u = U, \qquad v = 0. \tag{5.9}$$

Bottom Wall

$$u = 0, \qquad v = 0. \tag{5.10}$$

Left Wall

$$u = 0, \qquad v = 0. \tag{5.11}$$

Right Wall

$$u = 0, \qquad v = 0. \tag{5.12}$$

These are no-slip boundary conditions.

5.4.2 Pressure Boundary Conditions

Pressure is defined only up to an arbitrary constant.

A common choice is

$$\frac{\partial p}{\partial x} = 0 \quad \text{on the vertical walls,} \tag{5.13}$$

$$\frac{\partial p}{\partial y} = 0 \quad \text{on the bottom wall,} \tag{5.14}$$

and

$$p = 0 \quad \text{on the top wall.} \tag{5.15}$$

The final condition fixes the pressure reference level.

5.5 Discretisation of the Momentum Equations

Let the computational grid consist of

$$N_x \times N_y$$

grid points.

The grid locations are

$$x_i = i\Delta x, \qquad y_j = j\Delta y.$$

The discrete variables are

$$u^n_{i,j}, \qquad v^n_{i,j}, \qquad p^n_{i,j}.$$

5.5.1 Convective Terms

Using backward differences,

$$\frac{\partial u}{\partial x} \approx \frac{u_{i,j} - u_{i-1,j}}{\Delta x}, \tag{5.16}$$

$$\frac{\partial u}{\partial y} \approx \frac{u_{i,j} - u_{i,j-1}}{\Delta y}. \tag{5.17}$$

5.5.2 Diffusion Terms

Using central differences,

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{\Delta x^2}, \tag{5.18}$$

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{\Delta y^2}. \tag{5.19}$$

Similar expressions are used for the v velocity component.

5.6 The Pressure Poisson Equation

To enforce incompressibility, pressure is obtained from

$$\boxed{\nabla^2 p = b.} \tag{5.20}$$

5.6.1 Pressure Source Term

A commonly used source term is

$$b = \rho \left[\frac{1}{\Delta t} \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right) - \left(\frac{\partial u}{\partial x} \right)^2 - 2 \frac{\partial u}{\partial y} \frac{\partial v}{\partial x} - \left(\frac{\partial v}{\partial y} \right)^2 \right]. \tag{5.21}$$

5.6.2 Finite-Difference Form

The Poisson equation becomes

$$\frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2} = b_{i,j}. \tag{5.22}$$

Rearranging gives

$$p_{i,j} = \frac{\Delta y^2 (p_{i+1,j} + p_{i-1,j}) + \Delta x^2 (p_{i,j+1} + p_{i,j-1}) - b_{i,j} \Delta x^2 \Delta y^2}{2(\Delta x^2 + \Delta y^2)}. \tag{5.23}$$

This equation is solved iteratively at each timestep.

5.7 Velocity Update Equations

Once pressure has been calculated, the velocity field can be updated.

The update equation for the u velocity is

$$\begin{aligned} u_{i,j}^{n+1} = u_{i,j}^n - u_{i,j}^n \frac{\Delta t}{\Delta x} \left(u_{i,j}^n - u_{i-1,j}^n \right) - v_{i,j}^n \frac{\Delta t}{\Delta y} \left(u_{i,j}^n - u_{i,j-1}^n \right) \\ - \frac{\Delta t}{2\rho \Delta x} \left(p_{i+1,j}^n - p_{i-1,j}^n \right) + \nu \Delta t \nabla_h^2 u. \end{aligned} \tag{5.24}$$

Similarly, the v velocity becomes

$$\begin{aligned} v_{i,j}^{n+1} = v_{i,j}^n - u_{i,j}^n \frac{\Delta t}{\Delta x} \left(v_{i,j}^n - v_{i-1,j}^n \right) - v_{i,j}^n \frac{\Delta t}{\Delta y} \left(v_{i,j}^n - v_{i,j-1}^n \right) \\ - \frac{\Delta t}{2\rho \Delta y} \left(p_{i,j+1}^n - p_{i,j-1}^n \right) + \nu \Delta t \nabla_h^2 v. \end{aligned} \tag{5.25}$$

These equations combine convection, pressure, and diffusion.

5.8 Numerical Algorithm

The complete algorithm may be summarised as follows.

1. Define the computational domain.
2. Select N_x, N_y, Δx, Δy, and Δt.
3. Initialise u, v, and p.
4. Apply boundary conditions.
5. Build the pressure source term b.
6. Solve the pressure Poisson equation.
7. Update u and v.
8. Reapply boundary conditions.
9. Advance to the next timestep.

Symbolically,

$$\text{Build } b \rightarrow \text{Solve } \nabla^2 p = b \rightarrow \text{Update } u, v \rightarrow \text{Apply BCs.}$$

5.9 Stability Considerations

5.9.1 CFL Condition

For convection,

$$\boxed{\frac{\max|u|\Delta t}{\Delta x} + \frac{\max|v|\Delta t}{\Delta y} \leq 1.} \tag{5.26}$$

5.9.2 Diffusion Stability Criterion

For explicit diffusion,

$$\boxed{\nu \Delta t \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} \right) \leq \frac{1}{2}.} \tag{5.27}$$

The timestep must satisfy both conditions.

5.10 Expected Flow Behaviour

Initially the moving lid imparts momentum to the fluid near the upper boundary.

As time progresses:

- momentum diffuses into the cavity,
- fluid begins to circulate,
- a primary vortex develops near the centre,
- secondary vortices may appear near the corners.

The Reynolds number is

$$\boxed{Re = \frac{UL}{\nu}.} \tag{5.28}$$

For small Reynolds numbers, viscous forces dominate and the flow remains smooth.

As the Reynolds number increases, inertial effects become stronger and the vortex structure becomes more pronounced.

5.11 Connection to Previous Chapters

The Navier–Stokes equations unify all of the concepts developed previously.

From Convection

$$(\mathbf{u} \cdot \nabla)\mathbf{u}$$

From Diffusion

$$\nu \nabla^2 \mathbf{u}$$

From Burgers' Equation

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} = \nu \nabla^2 \mathbf{u}$$

From the Poisson Equation

$$\nabla^2 p = b$$

New Features

$$-\frac{1}{\rho}\nabla p, \qquad \nabla \cdot \mathbf{u} = 0.$$

Thus the incompressible Navier–Stokes equations may be viewed as Burgers' equations augmented by pressure and the incompressibility constraint.

5.12 Validation and Diagnostics

A CFD solution should be assessed using several checks.

5.12.1 Divergence Check

Verify that

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \approx 0.$$

5.12.2 Boundary Condition Check

The solution should satisfy all prescribed wall velocities.

5.12.3 Steady-State Check

For a steady solution,

$$\|u^{n+1} - u^n\| < \varepsilon, \qquad \|v^{n+1} - v^n\| < \varepsilon.$$

5.12.4 Physical Behaviour Check

The cavity should exhibit recirculating flow with the strongest horizontal velocity occurring near the moving lid.

5.13 Chapter Summary

This chapter introduced the numerical solution of the two-dimensional incompressible Navier–Stokes equations using the lid-driven cavity problem as a benchmark example. The governing momentum and continuity equations were presented, together with their physical interpretation and finite-difference discretisation.

The pressure Poisson equation was shown to provide the mechanism through which incompressibility is enforced, while the velocity update equations demonstrated how convection, diffusion, and pressure interact within a CFD algorithm. Stability constraints, expected flow behaviour, and validation procedures were also discussed.

With this chapter, the mathematical progression from model partial differential equations to a complete incompressible flow solver is complete. The next step is to implement these equations computationally and explore the resulting velocity fields, pressure distributions, streamlines, vorticity, and convergence behavior.

Chapter 6

Compressible Navier-Stokes Equations

The incompressible Navier–Stokes equations provide an excellent model for many engineering flows, particularly liquids and low-speed gas flows. In these situations, density variations are typically negligible, allowing the simplifying assumption that density remains constant throughout the flow field.

However, many important applications involve significant density variations. Examples include high-speed aerodynamics, gas turbines, jet engines, rocket propulsion, explosions, shock waves, and acoustic phenomena. In such problems, variations in density, pressure, and temperature become important and cannot be ignored. The incompressibility assumption is no longer valid, and a more general mathematical framework is required.

This chapter extends the incompressible formulation developed previously to compressible flow. We examine how the conservation equations change when density becomes a variable, introduce the energy equation, discuss the role of thermodynamics through the equation of state, and develop the conservative form of the compressible Euler and Navier–Stokes equations. These developments form the foundation of modern compressible Computational Fluid Dynamics.

6.1 Motivation

The incompressible Navier–Stokes equations assume that density is constant:

$$\boxed{\rho = \text{constant.}} \tag{6.1}$$

Under this assumption, conservation of mass reduces to

$$\boxed{\nabla \cdot \mathbf{u} = 0.} \tag{6.2}$$

This approximation is appropriate for:

- liquid flows,
- low-speed gas flows,
- many environmental and hydraulic flows.

In compressible flow, density varies with space and time,

$$\boxed{\rho = \rho(x, y, t).} \tag{6.3}$$

Such behaviour occurs in:

- high-speed aerodynamic flows,
- shock waves,
- acoustic waves,
- compressible pipe flows,
- flows with large temperature variations.

The incompressibility constraint must therefore be replaced by a full mass conservation equation.

6.2 From Incompressibility to Compressibility

For incompressible flow,

$$\nabla \cdot \mathbf{u} = 0. \tag{6.4}$$

For compressible flow, conservation of mass becomes

$$\boxed{\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0.} \tag{6.5}$$

In two dimensions,

$$\boxed{\frac{\partial \rho}{\partial t} + \frac{\partial (\rho u)}{\partial x} + \frac{\partial (\rho v)}{\partial y} = 0.} \tag{6.6}$$

This equation states that changes in density arise from the net flow of mass into or out of a control volume.

Unlike incompressible flows, compressible flows permit local compression and expansion of the fluid.

6.3 Compressible Momentum Equations

The momentum equations are most naturally written in conservation form.

6.3.1 Conservation of x-Momentum

$$\boxed{\frac{\partial (\rho u)}{\partial t} + \frac{\partial (\rho u^2)}{\partial x} + \frac{\partial (\rho u v)}{\partial y} = -\frac{\partial p}{\partial x} + \text{viscous terms}.} \tag{6.7}$$

6.3.2 Conservation of y-Momentum

$$\boxed{\frac{\partial (\rho v)}{\partial t} + \frac{\partial (\rho u v)}{\partial x} + \frac{\partial (\rho v^2)}{\partial y} = -\frac{\partial p}{\partial y} + \text{viscous terms}.} \tag{6.8}$$

A key difference from incompressible flow is that the conserved variables are now

$$\rho u, \qquad \rho v,$$

rather than simply

$$u, \qquad v.$$

Density and velocity are therefore dynamically coupled.

6.4 The Need for an Energy Equation

In incompressible CFD, pressure is obtained from a Poisson equation used to enforce

$$\nabla \cdot \mathbf{u} = 0.$$

In compressible flow, pressure is no longer a constraint variable.
Instead, pressure is linked to density and temperature through thermodynamic relationships.
As a result, a third conservation equation is required.

6.4.1 Conservation of Energy

The total energy equation may be written as

$$\boxed{\frac{\partial E}{\partial t} + \nabla \cdot [(E + p)\mathbf{u}] = \text{viscous and heat-conduction terms.}} \tag{6.9}$$

The total energy is

$$\boxed{E = \rho e + \frac{1}{2}\rho\left(u^2 + v^2\right),} \tag{6.10}$$

where

- e is the internal energy per unit mass,
- $\frac{1}{2}\rho(u^2 + v^2)$ is the kinetic energy per unit volume.

The energy equation introduces temperature and thermodynamic effects into the governing system.

6.5 Equation of State

The conservation equations alone are insufficient to determine all unknowns.
An additional relationship is required to connect pressure, density, and temperature.

6.5.1 Ideal Gas Law

For an ideal gas,

$$\boxed{p = \rho R T,} \tag{6.11}$$

where

- p is pressure,
- ρ is density,
- R is the gas constant,
- T is temperature.

6.5.2 Energy Form

For a calorically perfect gas,

$$\boxed{p = (\gamma - 1)\rho e,} \tag{6.12}$$

where

- e is the internal energy per unit mass,
- γ is the ratio of specific heats.

This relationship provides closure for the compressible-flow equations.

6.6 The Compressible Euler Equations

Before introducing viscosity and heat conduction, it is useful to consider inviscid compressible flow.

The resulting equations are known as the Euler equations.

6.6.1 Conservative Variables

Define

$$\boxed{\mathbf{q} = \begin{bmatrix} \rho \\ \rho u \\ \rho v \\ E \end{bmatrix}.} \tag{6.13}$$

6.6.2 Conservation Form

The two-dimensional Euler equations can be written as

$$\boxed{\frac{\partial \mathbf{q}}{\partial t} + \frac{\partial \mathbf{F}}{\partial x} + \frac{\partial \mathbf{G}}{\partial y} = 0.} \tag{6.14}$$

6.6.3 Flux Vectors

The flux vector in the x-direction is

$$\mathbf{F} = \begin{bmatrix} \rho u \\ \rho u^2 + p \\ \rho u v \\ u(E + p) \end{bmatrix}, \tag{6.15}$$

while the flux vector in the y-direction is

$$\mathbf{G} = \begin{bmatrix} \rho v \\ \rho u v \\ \rho v^2 + p \\ v(E + p) \end{bmatrix}. \tag{6.16}$$

This conservative form is especially important because compressible flows may contain discontinuities such as shock waves.

6.7 The Compressible Navier–Stokes Equations

The compressible Navier–Stokes equations extend the Euler equations by including viscosity and heat conduction.

They may be written schematically as

$$\boxed{\frac{\partial \mathbf{q}}{\partial t} + \nabla \cdot \mathbf{F}_{\text{inv}} = \nabla \cdot \mathbf{F}_{\text{vis}}.} \tag{6.17}$$

The conserved variables remain

$$\mathbf{q} = [\rho, \rho u, \rho v, E]^T.$$

Compared with incompressible flow, several important changes occur:

- density becomes an unknown,
- pressure is obtained from thermodynamics,
- the energy equation becomes essential,
- shock waves may appear,
- conservative numerical methods are required.

6.8 The Mach Number

Compressibility effects are commonly characterised using the Mach number.

6.8.1 Definition

$$\boxed{M = \frac{|\mathbf{u}|}{a},} \tag{6.18}$$

where

- $|\mathbf{u}|$ is the flow speed,
- a is the local speed of sound.

For an ideal gas,

$$\boxed{a = \sqrt{\gamma R T}.} \tag{6.19}$$

6.8.2 Flow Regimes

$M \ll 1$	Nearly incompressible flow
$M < 1$	Subsonic flow
$M = 1$	Sonic flow
$M > 1$	Supersonic flow
$M \gg 1$	Hypersonic flow

As the Mach number increases, compressibility effects become increasingly important.

6.9 Comparison Between Incompressible and Compressible Flow

Feature	Incompressible Flow	Compressible Flow
Density	Constant	Variable
Continuity Equation	$\nabla \cdot \mathbf{u} = 0$	$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{u}) = 0$
Pressure	Constraint variable	Thermodynamic variable
Pressure Equation	Poisson equation	Equation of state
Energy Equation	Often neglected	Required
Unknowns	u, v, p	$\rho, \rho u, \rho v, E$
Shock Waves	Absent	Possible
Numerical Form	Primitive variables	Conservative variables

6.10 Implementation Roadmap for Compressible Flow

The implementation strategy differs substantially from the incompressible solver.

6.10.1 Step 1: Define the Conserved Variables

The primary unknown vector is

$$\mathbf{q} = [\rho, \rho u, \rho v, E]^T.$$

These variables are advanced in time.

6.10.2 Step 2: Recover Primitive Variables

The primitive variables are obtained from the conserved variables.

$$u = \frac{\rho u}{\rho}, \qquad v = \frac{\rho v}{\rho}. \tag{6.20}$$

The kinetic energy is

$$K = \frac{1}{2}\rho(u^2 + v^2). \tag{6.21}$$

The pressure becomes

$$p = (\gamma - 1)(E - K). \tag{6.22}$$

6.10.3 Step 3: Construct Fluxes

Evaluate the flux vectors

$$\mathbf{F}(\mathbf{q}), \qquad \mathbf{G}(\mathbf{q}).$$

6.10.4 Step 4: Advance the Solution

A conservative update takes the form

$$\mathbf{q}_{i,j}^{n+1} = \mathbf{q}_{i,j}^{n} - \frac{\Delta t}{\Delta x}\left(\mathbf{F}_{i+1/2,j} - \mathbf{F}_{i-1/2,j}\right) - \frac{\Delta t}{\Delta y}\left(\mathbf{G}_{i,j+1/2} - \mathbf{G}_{i,j-1/2}\right). \tag{6.23}$$

6.10.5 Step 5: Apply Boundary Conditions

Typical compressible-flow boundary conditions include:

- inflow boundaries,
- outflow boundaries,
- slip walls,
- no-slip walls,
- reflective boundaries,
- far-field boundaries.

6.10.6 Step 6: Choose a Stable Time Step

The timestep is governed by wave propagation.

A common CFL restriction is

$$\boxed{\Delta t \leq C_{\mathrm{CFL}} \min\left(\frac{\Delta x}{|u|+a}, \frac{\Delta y}{|v|+a}\right).} \tag{6.24}$$

Unlike incompressible flow, the speed of sound also influences stability.

6.10.7 Step 7: Monitor Physical Admissibility

The solution must satisfy

$$\rho > 0, \qquad p > 0. \tag{6.25}$$

Negative pressure or density usually indicate numerical instability.

6.11 The Shock-Tube Problem

Before attempting full compressible Navier–Stokes simulations, it is useful to study the one-dimensional Euler equations.

A standard benchmark is the shock-tube problem, in which two regions of gas with different initial pressures are separated by a diaphragm.

When the diaphragm is removed, three characteristic features develop:

- a shock wave,
- a contact discontinuity,
- a rarefaction wave.

The shock-tube problem provides an excellent first test for compressible-flow solvers.

6.12 Chapter Summary

This chapter extended the incompressible Navier–Stokes equations to compressible flow. The incompressibility constraint was replaced by full conservation of mass, while the momentum equations were reformulated in conservative form. The need for an energy equation and an equation of state was introduced, leading to a fully coupled system involving density, momentum, pressure, temperature, and energy.

The compressible Euler equations were presented as the inviscid limit of the governing equations, followed by the more general compressible Navier–Stokes equations. The importance of the Mach number, conservative formulations, and thermodynamic closure was highlighted throughout.

The transition from incompressible to compressible flow represents one of the most significant conceptual developments in fluid dynamics. The next chapter will build upon these ideas by introducing numerical techniques specifically designed for compressible-flow simulations, including shock-capturing methods, conservative discretisations, and finite-volume formulations.

Part II

CFD Implementation in MATLAB

Chapter 7

One-Dimensional Linear Convection

7.1 Introduction

The linear convection equation is one of the simplest transport equations encountered in Computational Fluid Dynamics. Although mathematically straightforward, it introduces many of the concepts that appear throughout numerical PDE solvers, including spatial discretisation, time stepping, boundary conditions, numerical stability, and numerical diffusion.

The governing equation is

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0, \tag{7.1}$$

where $u(x,t)$ is the transported quantity and c is a constant wave speed.

For this example, the initial condition is a square pulse,

$$u(x,0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0, \\ 1, & \text{otherwise.} \end{cases} \tag{7.2}$$

The objective is to observe how the pulse propagates through the domain and to compare the numerical solution with the expected analytical behaviour.

7.2 Numerical Discretisation

Using a forward difference in time,

$$\frac{\partial u}{\partial t} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}, \tag{7.3}$$

and a backward difference in space,

$$\frac{\partial u}{\partial x} \approx \frac{u_i^n - u_{i-1}^n}{\Delta x}, \tag{7.4}$$

gives

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + c\frac{u_i^n - u_{i-1}^n}{\Delta x} = 0. \tag{7.5}$$

Rearranging,

$$u_i^{n+1} = u_i^n - c\frac{\Delta t}{\Delta x}\left(u_i^n - u_{i-1}^n\right). \tag{7.6}$$

Defining the Courant–Friedrichs–Lewy (CFL) number as

$$\sigma = c\frac{\Delta t}{\Delta x}, \tag{7.7}$$

the update equation becomes

$$u_i^{n+1} = u_i^n - \sigma\left(u_i^n - u_{i-1}^n\right). \tag{7.8}$$

The MATLAB program implements this finite-difference approximation directly.

7.3 Code Walkthrough

7.3.1 Defining the Computational Grid

The computational mesh is generated using

```
nx = 101;
L = 2.0;

dx = L/(nx-1);
x = linspace(0,L,nx);
```

The variable `nx` specifies the number of grid points, while `L` defines the physical length of the domain.

The grid spacing is given by

$$\Delta x = \frac{L}{nx - 1}.$$

The MATLAB function `linspace` generates a uniformly spaced set of coordinates between $x = 0$ and $x = L$.

7.3.2 Defining the Physical Parameters

The convection speed is defined using

```
c = 1.0;
```

The governing equation therefore becomes

$$\frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} = 0.$$

Since $c > 0$, information propagates from left to right.

7.3.3 Defining the Time-Stepping Parameters

The timestep information is specified by

```
nt = 50;
dt = 0.01;
```

The variable `nt` specifies the number of timesteps, while `dt` defines the size of each timestep. The simulation therefore advances to

$$t = nt\,\Delta t.$$

The CFL number is

```
sigma = c*dt/dx;
```

which corresponds to

$$\sigma = c\frac{\Delta t}{\Delta x}.$$

The CFL number plays a central role in stability.

7.3.4 Constructing the Initial Condition

The solution field is initially assigned the value one at every grid point:

```
u = ones(1,nx);
```

This produces a uniform field

$$u(x) = 1.$$

A square pulse is then created using

```
for i = 1:nx

    if x(i) >= 0.5 && x(i) <= 1.0

        u(i) = 2.0;

    end

end
```

Each grid point is examined individually.
If the coordinate lies inside the interval

$$0.5 \leq x \leq 1.0,$$

the value of the solution is replaced by two.
The resulting profile becomes

$$u(x,0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0, \\ 1, & \text{otherwise.} \end{cases}$$

7.3.5 Saving the Initial Condition

Before the simulation begins, a copy of the initial solution is stored:

```
u_initial = u;
```

This allows the initial profile to be plotted later alongside the final numerical solution.

7.3.6 Beginning the Time Integration

The simulation is advanced in time using

```
for n = 1:nt
```

Each pass through the loop corresponds to one timestep.
At the beginning of every timestep, the previous solution is stored:

```
un = u;
```

This is necessary because all finite-difference updates must use values from the previous timestep.

7.3.7 Updating the Interior Points

The finite-difference update is performed using

```
for i = 2:nx

    u(i) = un(i) ...
         - sigma*(un(i)-un(i-1));

end
```

The loop begins at the second grid point because the first grid point is reserved for the boundary condition.

This MATLAB statement implements

$$u_i^{n+1} = u_i^n - \sigma \left(u_i^n - u_{i-1}^n \right). \tag{7.9}$$

The term

$$u_i^n - u_{i-1}^n$$

is the backward-difference approximation of the spatial derivative.

The coefficient σ is the CFL number, which controls the propagation of information between neighbouring cells.

7.3.8 Applying the Boundary Condition

After the interior points are updated, the left boundary condition is imposed:

```
u(1) = 1.0;
```

This corresponds to

$$u(0,t) = 1.$$

Boundary conditions are enforced after every timestep to maintain a well-posed numerical problem.

7.3.9 Visualising the Results

The numerical solution is visualised using

```
plot(x,u_initial,'k--','LineWidth',2);
hold on;

plot(x,u,'b','LineWidth',2);
```

The dashed black curve represents the initial condition, while the solid blue curve shows the final numerical solution.

The figure formatting is completed using

```
xlabel('x');
ylabel('u');

title('1D Linear Convection');

legend('Initial Condition','Numerical Solution');

grid on;
```

These commands add axis labels, a title, a legend, and grid lines.

The final figure provides a direct comparison between the initial square pulse and the convected numerical solution.

7.4 Algorithm Summary

The MATLAB solver follows the sequence

1. Define the computational grid.
2. Specify the square-pulse initial condition.
3. Store the initial solution.
4. Compute the CFL-based finite-difference update.
5. Apply the left boundary condition.
6. Repeat for the prescribed number of timesteps.
7. Plot the initial and final solutions.

7.5 Expected Behaviour

The exact solution of the linear convection equation translates the pulse without changing its shape.

Since the convection speed is positive, the pulse moves toward increasing values of x.

The numerical solution behaves similarly, but the first-order upwind discretisation introduces numerical diffusion. As a result, the sharp edges of the pulse become smeared as the simulation progresses.

The amount of smearing depends on the grid spacing, timestep size, and CFL number. Smaller values of Δx generally improve accuracy, while excessively large timesteps can eventually lead to instability.

7.6 Key Lessons from Problem 1

This first MATLAB implementation introduces several concepts that recur throughout Computational Fluid Dynamics:

- discretisation of derivatives using finite differences,
- grid generation,
- explicit time integration,
- boundary condition implementation,
- CFL-based stability considerations,
- numerical diffusion,
- visualisation of numerical results.

Although the linear convection equation is simple, the computational structure developed here forms the foundation for all subsequent CFD solvers in this book.

Chapter 8

CFL Condition for Linear Convection

8.1 Introduction

In the previous chapter, we developed a MATLAB solver for the one-dimensional linear convection equation and observed the propagation of a square pulse through a computational domain.

An important question naturally arises:

How large can the timestep be before the numerical method becomes unstable?

The answer is provided by the Courant–Friedrichs–Lewy (CFL) condition. The CFL condition is one of the most important concepts in Computational Fluid Dynamics because it connects the physical speed of information propagation with the numerical grid and timestep.

This chapter investigates the effect of different CFL numbers on the behaviour of the first-order upwind scheme.

8.2 The CFL Number

For the linear convection equation

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0, \tag{8.1}$$

the CFL number is defined as

$$\boxed{\sigma = c\frac{\Delta t}{\Delta x}.} \tag{8.2}$$

The quantity σ measures how far information travels during a single timestep relative to the grid spacing.

For the first-order upwind method, stability generally requires

$$\boxed{\sigma \leq 1.} \tag{8.3}$$

The purpose of this chapter is to investigate this requirement numerically.

8.3 Numerical Method

The finite-difference update used throughout this chapter is

$$u_i^{n+1} = u_i^n - \sigma \left(u_i^n - u_{i-1}^n\right). \tag{8.4}$$

The only difference from the previous chapter is that several values of σ will be tested. The values considered are

$$\sigma = 0.2, \quad 0.5, \quad 1.0, \quad 1.2.$$

These cases allow us to compare stable and unstable behaviour.

8.4 Code Walkthrough

8.4.1 Defining the Computational Grid

The computational domain is generated using

```
nx = 101;
L = 2.0;

dx = L/(nx-1);

x = linspace(0,L,nx);
```

The domain extends from

$$x = 0$$

to

$$x = 2.$$

The variable `nx` specifies the number of grid points, while `dx` represents the resulting spatial grid spacing.

8.4.2 Defining the Physical Parameters

The convection speed is defined as

```
c = 1.0;
```

Therefore, the pulse propagates toward increasing values of x.
The simulation length is specified by

```
nt = 50;
```

meaning that each CFL case will be advanced through the same number of timesteps.

8.4.3 Defining the CFL Values

A collection of CFL numbers is stored in the vector

```
sigma_values = [0.2, 0.5, 1.0, 1.2];
```

Each element corresponds to a separate numerical experiment.
The first three values satisfy

$$\sigma \leq 1,$$

while the final value violates the stability condition.

8.4.4 Creating the Figure Window

The command

```
figure
```

creates a new figure that will contain all four numerical solutions.
The results will later be displayed using MATLAB subplots.

8.4.5 Looping Over CFL Cases

Each simulation is performed inside the loop

```
for case_id = 1:length(sigma_values)
```

The variable `case_id` identifies the current CFL case.
The corresponding CFL number is extracted using

```
sigma = sigma_values(case_id);
```

8.4.6 Computing the Timestep

Rather than prescribing the timestep directly, the CFL relation is rearranged:

$$\Delta t = \frac{\sigma \Delta x}{c}. \tag{8.5}$$

This is implemented in MATLAB as

```
dt = sigma*dx/c;
```

Consequently, each simulation automatically uses a timestep consistent with the chosen CFL number.

8.4.7 Constructing the Initial Condition

The solution field is initialised as

```
u = ones(1,nx);
```

A square pulse is then created using

```
for i = 1:nx

    if x(i) >= 0.5 && x(i) <= 1.0

        u(i) = 2.0;

    end

end
```

This produces

$$u(x,0) = \begin{cases} 2, & 0.5 \le x \le 1.0, \\ 1, & \text{otherwise.} \end{cases}$$

The same initial condition is used for every CFL value to enable fair comparison.

8.4.8 Time Integration

The numerical solution is advanced using

```
for n = 1:nt

    un = u;

    for i = 2:nx

        u(i) = un(i) ...
             - sigma*(un(i)-un(i-1));

    end

    u(1) = 1.0;

end
```

This is the first-order upwind approximation of the convection equation.
Notice that the update formula contains the CFL number directly.
Therefore, changing σ immediately changes the behaviour of the numerical scheme.

8.4.9 Creating the Subplots

Each solution is displayed using

```
subplot(2,2,case_id)
```

The figure is divided into four panels.
Each panel corresponds to one CFL value.
The numerical result is plotted using

```
plot(x,u,'LineWidth',2)
```

8.4.10 Labelling the Results

The title of each subplot is generated using

```
title(['CFL = ', num2str(sigma)])
```

The function `num2str` converts the numerical CFL value into text so that it can be displayed within the figure title.

The axes are labelled using

```
xlabel('x')
ylabel('u')
```

while

```
grid on
```

adds grid lines to improve readability.

8.5 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the computational grid.

2. Define a list of CFL numbers.
3. For each CFL value:
   (a) Compute the timestep.
   (b) Create the initial condition.
   (c) Advance the solution through time.
   (d) Plot the final result.
4. Compare the solutions.

8.6 Expected Behaviour

8.6.1 CFL = 0.2

For a small CFL number, the numerical method remains stable.

However, the pulse experiences noticeable numerical diffusion and the sharp edges become smeared.

8.6.2 CFL = 0.5

The method remains stable while reducing some of the excessive smoothing observed at lower CFL values.

The solution more closely resembles the analytical result.

8.6.3 CFL = 1.0

This case lies on the stability limit.

The wave propagates across the grid with minimal numerical diffusion and generally produces the most accurate result among the four cases.

8.6.4 CFL = 1.2

This case violates the CFL condition.

Numerical instability begins to develop because information attempts to travel farther than one cell during a single timestep.

The solution may exhibit non-physical oscillations and eventually diverge.

8.7 Physical Interpretation of the CFL Condition

The CFL number may be interpreted as

$$\sigma = \frac{\text{distance travelled by the wave}}{\text{cell width}}. \tag{8.6}$$

If

$$\sigma < 1,$$

information travels less than one grid cell per timestep.

The numerical method can therefore track information propagation correctly.

If

$$\sigma > 1,$$

information attempts to jump over neighbouring cells during a single timestep.

The numerical scheme can no longer communicate information correctly between grid points, leading to instability.

8.8 Key Lessons from Problem 2

This problem introduces one of the most fundamental concepts in CFD: numerical stability.

The main conclusions are:

- the timestep cannot be chosen independently of the grid spacing;
- the CFL number controls stability for convection-dominated problems;
- stable solutions generally require
$$\sigma \leq 1;$$
- smaller CFL numbers increase stability but often increase numerical diffusion;
- CFL analysis provides the foundation for selecting timesteps in all subsequent CFD simulations.

Understanding the CFL condition is essential before progressing to nonlinear convection, diffusion, Burgers' equation, and the Navier–Stokes equations, where stability constraints become even more important.

Chapter 9

One-Dimensional Non-linear Convection

9.1 Introduction

In the previous chapters, we examined the numerical solution of the linear convection equation and investigated the influence of the CFL condition on numerical stability.

The next step is to consider nonlinear convection, where the wave speed is no longer constant. Instead, the solution itself determines the propagation speed.

The governing equation is

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0. \tag{9.1}$$

Unlike linear convection, different parts of the solution travel at different speeds. Regions with larger values of u move faster than regions with smaller values of u. As a result, the wave begins to distort and steepen as time progresses.

This chapter develops a MATLAB implementation of the nonlinear convection equation and demonstrates how nonlinear wave propagation differs from the linear case.

9.2 Physical Interpretation

The nonlinear convection equation may be written as

$$\frac{\partial u}{\partial t} = -u\frac{\partial u}{\partial x}. \tag{9.2}$$

The transport speed is now

$$c = u.$$

Consequently, the local propagation speed varies throughout the domain.

For the initial square pulse,

$$u = 2$$

inside the pulse and

$$u = 1$$

outside the pulse.

The larger values therefore travel faster than the smaller values.

This causes the leading edge of the wave to steepen and eventually can lead to the formation of discontinuities in more advanced problems.

9.3 Numerical Discretisation

Using a forward difference in time,

$$\frac{\partial u}{\partial t} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}, \tag{9.3}$$

and a backward difference in space,

$$\frac{\partial u}{\partial x} \approx \frac{u_i^n - u_{i-1}^n}{\Delta x}, \tag{9.4}$$

gives

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} + u_i^n \frac{u_i^n - u_{i-1}^n}{\Delta x} = 0. \tag{9.5}$$

Rearranging yields

$$\boxed{u_i^{n+1} = u_i^n - u_i^n \frac{\Delta t}{\Delta x} \left(u_i^n - u_{i-1}^n\right).} \tag{9.6}$$

Notice that the convection speed has become the local solution value itself.
This single modification changes the mathematical behaviour of the problem dramatically.

9.4 Code Walkthrough

9.4.1 Defining the Computational Grid

The computational domain is defined by

```
nx = 101;
L = 2.0;

dx = L/(nx-1);

x = linspace(0,L,nx);
```

The domain extends from

$$0 \leq x \leq 2.$$

The variable `nx` specifies the number of grid points while `dx` defines the spatial step size.
The coordinates of all grid points are stored in the vector `x`.

9.4.2 Defining the Time-Stepping Parameters

The simulation parameters are

```
nt = 50;
dt = 0.005;
```

The variable `nt` defines the total number of timesteps.
The smaller timestep compared with the linear convection example helps maintain numerical stability because the wave speed now varies throughout the domain.

9.4.3 Constructing the Initial Condition

The solution field is initialised as

```
u = ones(1,nx);
```

This assigns the value one everywhere.
A square pulse is then introduced:

```
for i = 1:nx

    if x(i) >= 0.5 && x(i) <= 1.0

        u(i) = 2.0;

    end

end
```

The resulting initial condition is

$$u(x,0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0, \\ 1, & \text{otherwise.} \end{cases}$$

This profile is identical to the one used in the linear convection problem.
The difference lies entirely in the governing equation.

9.4.4 Storing the Initial Solution

Before time integration begins, a copy of the initial profile is saved:

```
u_initial = u;
```

This allows comparison between the starting and final solutions.

9.4.5 Beginning the Time Loop

The simulation advances through time using

```
for n = 1:nt
```

Each pass through this loop corresponds to one timestep.
At the beginning of each timestep, the previous solution is stored:

```
un = u;
```

This guarantees that all updates are based on data from the same timestep.

9.4.6 Updating the Interior Points

The finite-difference update is performed using

```
for i = 2:nx

    u(i) = un(i) ...
         - un(i)*dt/dx*(un(i)-un(i-1));

end
```

This statement implements

$$u_i^{n+1} = u_i^n - u_i^n \frac{\Delta t}{\Delta x} \left(u_i^n - u_{i-1}^n\right). \tag{9.7}$$

The key difference from linear convection is the appearance of the factor

$$u_i^n.$$

In the linear equation, every point propagated with the same speed:

$$c = \text{constant}.$$

In the nonlinear equation, every grid point has its own propagation speed. Consequently, some regions move faster than others.

9.4.7 Applying the Boundary Condition

The left boundary condition is imposed via

```
u(1) = 1.0;
```

This corresponds to

$$u(0,t) = 1.$$

The value is enforced after every timestep.

9.4.8 Visualising the Results

The initial and final solutions are displayed using

```
plot(x,u_initial,'--','LineWidth',2)

hold on

plot(x,u,'LineWidth',2)
```

The figure annotations are added using

```
xlabel('x')
ylabel('u')

legend('Initial','Final')

title('1D Nonlinear Convection')

grid on
```

This creates a direct comparison between the starting pulse and the evolved solution.

9.5 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the computational grid.
2. Define the square-pulse initial condition.
3. Store the initial profile.

4. Advance the solution using nonlinear convection.
5. Apply the boundary condition.
6. Repeat for each timestep.
7. Plot the initial and final solutions.

9.6 Comparison with Linear Convection

For the linear convection equation,

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0,$$

all parts of the wave travel with the same speed c.
The pulse therefore translates without changing shape.
For the nonlinear convection equation,

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = 0,$$

the propagation speed depends on the solution itself.
Regions where u is larger move faster than regions where u is smaller.
This causes the wave profile to deform as it evolves.

9.7 Expected Behaviour

Initially, the square pulse resembles the profile from the linear convection problem.

As time progresses, however, the larger values within the pulse move faster than the surrounding fluid.

The front portion of the pulse therefore accelerates relative to the rear portion.
Consequently:

- the pulse no longer retains its original shape,
- the wave begins to distort,
- gradients become steeper,
- numerical difficulties become more pronounced.

These effects are characteristic of nonlinear transport phenomena.

9.8 Connection to Fluid Dynamics

Although simple, the nonlinear convection equation contains one of the most important features of the Navier–Stokes equations:

$$u\frac{\partial u}{\partial x}.$$

This term represents momentum being transported by the fluid itself.

The same nonlinear mechanism appears in fluid flow, gas dynamics, turbulence modelling, and many other areas of computational physics.

For this reason, nonlinear convection serves as an important stepping stone toward more realistic CFD simulations.

9.9 Key Lessons from Problem 3

This problem introduces the first genuinely nonlinear PDE encountered in the workbook.

The main conclusions are:

- the convection speed is no longer constant;
- different regions of the solution travel at different speeds;
- nonlinear effects distort the wave shape;
- steep gradients naturally develop;
- the nonlinear convection term is a fundamental component of the Navier–Stokes equations.

The next chapter introduces diffusion, which acts in the opposite direction by smoothing gradients rather than steepening them.

Chapter 10

One-Dimensional Diffusion

10.1 Introduction

In the previous chapter, we examined non-linear convection, where differences in propagation speed caused the wave profile to steepen and distort.

In many physical systems, however, an opposing mechanism is also present. Diffusion acts to smooth sharp gradients and distribute quantities more uniformly throughout the domain.

Examples include:

- heat conduction,
- molecular transport,
- pollutant dispersion,
- viscous momentum diffusion in fluids.

The diffusion equation is one of the most important partial differential equations in applied mathematics and engineering.

This chapter develops a MATLAB implementation of the one-dimensional diffusion equation and demonstrates how diffusion smooths an initially discontinuous profile.

10.2 Governing Equation

The one-dimensional diffusion equation is

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial x^2}, \tag{10.1}$$

where

- $u(x,t)$ is the transported quantity,
- ν is the diffusion coefficient.

Unlike convection, which transports information across the domain, diffusion acts locally to reduce gradients.

Large gradients produce stronger diffusion effects.

10.3 Physical Interpretation

The second derivative

$$\frac{\partial^2 u}{\partial x^2}$$

measures the curvature of the solution.
When the solution contains sharp corners or steep gradients, the curvature becomes large.
The diffusion equation therefore acts to:

- reduce sharp gradients,
- spread concentrated regions,
- smooth discontinuities,
- move the solution toward a uniform state.

This behaviour is fundamentally different from convection, which primarily transports information without necessarily smoothing it.

10.4 Numerical Discretisation

The time derivative is approximated using a forward difference:

$$\frac{\partial u}{\partial t} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}. \tag{10.2}$$

The second spatial derivative is approximated using a central difference:

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}. \tag{10.3}$$

Substituting into the governing equation gives

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}. \tag{10.4}$$

Rearranging yields the explicit diffusion update

$$\boxed{u_i^{n+1} = u_i^n + \nu \frac{\Delta t}{\Delta x^2} \left(u_{i+1}^n - 2u_i^n + u_{i-1}^n \right).} \tag{10.5}$$

This update is the basis of the MATLAB implementation.

10.5 Code Walkthrough

10.5.1 Defining the Computational Grid

The computational domain is generated using

```
nx = 101;
L = 2.0;

dx = L/(nx-1);

x = linspace(0,L,nx);
```

The domain extends from

$$0 \leq x \leq 2.$$

The variable `dx` represents the spacing between neighbouring grid points.

10.5.2 Defining the Simulation Parameters

The simulation parameters are

```
nt = 50;
nu = 0.05;
```

Here,

- `nt` is the number of timesteps,
- `nu` is the diffusion coefficient.

A larger diffusion coefficient produces more rapid smoothing.

10.5.3 Selecting a Stable Timestep

The timestep is chosen using

```
dt = 0.25*dx^2/nu;
```

For the explicit diffusion scheme, stability requires approximately

$$\boxed{\nu \frac{\Delta t}{\Delta x^2} \leq \frac{1}{2}.} \tag{10.6}$$

This condition is more restrictive than the CFL condition encountered for convection problems.

The chosen timestep provides a stable simulation.

10.5.4 Constructing the Initial Condition

The solution field is initialised using

```
u_field = ones(1,nx);
```

This assigns the value one throughout the domain.

A square pulse is then introduced using

```
for i = 1:nx

    if x(i) >= 0.5 && x(i) <= 1.0

        u_field(i) = 2.0;

    end

end
```

The resulting profile is

$$u(x,0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0, \\ 1, & \text{otherwise.} \end{cases}$$

This sharp profile allows the effects of diffusion to be observed clearly.

10.5.5 Storing the Initial Condition

Before the simulation begins, a copy of the initial profile is saved:

```
u_initial = u_field;
```

This will later be compared with the final numerical solution.

10.5.6 Beginning the Time Integration

The solution is advanced in time using

```
for n = 1:nt
```

At the beginning of each timestep, the current solution is copied:

```
un = u_field;
```

This ensures that every update uses values from the previous timestep.

10.5.7 Computing the Diffusion Update

The explicit diffusion scheme is implemented using

```
for i = 2:nx-1

    u_field(i) = un(i) ...
              + nu*dt/dx^2 ...
              *(un(i+1)-2*un(i)+un(i-1));

end
```

This directly implements

$$u_i^{n+1} = u_i^n + \nu \frac{\Delta t}{\Delta x^2} \left(u_{i+1}^n - 2u_i^n + u_{i-1}^n \right). \tag{10.7}$$

The quantity

$$u_{i+1} - 2u_i + u_{i-1}$$

approximates the curvature of the solution.
Large curvature produces stronger diffusion effects.

10.5.8 Applying Boundary Conditions

The boundary values are fixed using

```
u_field(1) = 1.0;
u_field(nx) = 1.0;
```

These conditions correspond to

$$u(0,t) = 1, \qquad u(L,t) = 1.$$

The boundary values remain unchanged throughout the simulation.

10.5.9 Visualising the Results

The initial and final solutions are displayed using

```
plot(x,u_initial,'--','LineWidth',2)

hold on

plot(x,u_field,'LineWidth',2)
```

The figure is labelled using

```
xlabel('x')
ylabel('u')

legend('Initial','Final')

title('1D Diffusion')

grid on
```

The resulting plot allows direct comparison between the initial square pulse and the diffused solution.

10.6 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the computational grid.
2. Define the diffusion coefficient.
3. Compute a stable timestep.
4. Construct the square-pulse initial condition.
5. Store the initial profile.
6. Advance the solution using the explicit diffusion scheme.
7. Apply boundary conditions.
8. Repeat for each timestep.
9. Plot the initial and final solutions.

10.7 Expected Behaviour

Initially, the solution contains sharp jumps at

$$x = 0.5$$

and

$$x = 1.0.$$

As time progresses:

- the sharp edges begin to smooth,

- gradients decrease,
- the pulse spreads outward,
- the peak value becomes smaller,
- the solution becomes increasingly uniform.

This behaviour is characteristic of diffusion processes.

10.8 Effect of the Diffusion Coefficient

The diffusion coefficient controls the rate of smoothing.

10.8.1 Small Diffusion Coefficient

When ν is small,

$$\nu \ll 1,$$

the solution changes slowly and the pulse remains relatively sharp.

10.8.2 Large Diffusion Coefficient

When ν is increased,

$$\nu \gg 1,$$

smoothing becomes much stronger.

The pulse rapidly spreads and approaches a nearly uniform state.

10.9 Comparison with Convection

Convection and diffusion have opposite effects on the solution.

Convection	Diffusion
Transports information	Spreads information
Moves solution	Smooths solution
Can steepen gradients	Reduces gradients
Produces wave motion	Produces spreading

Understanding the interaction between these two mechanisms is essential because both appear simultaneously in the Navier–Stokes equations.

10.10 Connection to Fluid Dynamics

In fluid mechanics, diffusion represents the action of viscosity.

Viscosity transfers momentum from regions of high velocity to regions of low velocity.

This mechanism prevents unlimited steepening of velocity gradients and plays a crucial role in stabilising fluid motion.

The diffusion equation therefore provides the foundation for understanding viscous effects in CFD.

10.11 Key Lessons from Problem 4

This problem introduces the first purely diffusive PDE in the workbook.

The main conclusions are:

- diffusion smooths sharp gradients;
- the second derivative measures curvature and drives diffusion;
- explicit diffusion schemes have important stability restrictions;
- larger diffusion coefficients increase the smoothing rate;
- diffusion acts in opposition to convection.

The next chapter combines nonlinear convection and diffusion into Burgers' equation, providing the first model problem that resembles the structure of the Navier–Stokes equations.

Chapter 11

One-Dimensional Burgers' Equation

11.1 Introduction

The previous chapters introduced two fundamental physical mechanisms:

- nonlinear convection, which tends to steepen gradients,
- diffusion, which tends to smooth gradients.

Many important fluid-flow problems involve both effects simultaneously.

A classic model that combines these competing mechanisms is Burgers' equation. Despite its apparent simplicity, Burgers' equation captures many of the mathematical features found in the Navier–Stokes equations.

For this reason, Burgers' equation is often regarded as a bridge between simple transport equations and full fluid-dynamics models.

This chapter develops a MATLAB implementation of Burgers' equation and investigates the interaction between nonlinear steepening and diffusive smoothing.

11.2 Governing Equation

The one-dimensional Burgers' equation is

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \tag{11.1}$$

where

- $u(x,t)$ is the transported quantity,
- ν is the diffusion coefficient.

The equation contains two competing physical processes:

- the nonlinear convection term
$$u\frac{\partial u}{\partial x},$$
which causes wave steepening,
- the diffusion term
$$\nu\frac{\partial^2 u}{\partial x^2},$$
which smooths gradients.

The resulting behaviour depends on the relative strength of these two mechanisms.

11.3 Physical Interpretation

The nonlinear convection term causes larger values of u to move faster than smaller values.

Consequently, wave fronts become progressively steeper.

The diffusion term has the opposite effect.

Whenever sharp gradients develop, diffusion spreads the solution and reduces the gradient magnitude.

Burgers' equation therefore represents a balance between:

- gradient steepening,
- gradient smoothing.

This competition lies at the heart of many fluid-dynamics problems.

11.4 Numerical Discretisation

Using a forward difference for the time derivative,

$$\frac{\partial u}{\partial t} \approx \frac{u_i^{n+1} - u_i^n}{\Delta t}, \tag{11.2}$$

a backward difference for the nonlinear convection term,

$$\frac{\partial u}{\partial x} \approx \frac{u_i^n - u_{i-1}^n}{\Delta x}, \tag{11.3}$$

and a central difference for the diffusion term,

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta x^2}, \tag{11.4}$$

gives

$$u_i^{n+1} = u_i^n - u_i^n \frac{\Delta t}{\Delta x}\left(u_i^n - u_{i-1}^n\right) + \nu \frac{\Delta t}{\Delta x^2}\left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right). \tag{11.5}$$

This update equation forms the basis of the MATLAB implementation.

11.5 Code Walkthrough

11.5.1 Defining the Computational Grid

The computational domain is created using

```
nx = 101;
L = 2.0;

dx = L/(nx-1);

x = linspace(0,L,nx);
```

The domain extends over

$$0 \le x \le 2.$$

The variable `dx` stores the uniform grid spacing.

11.5.2 Defining the Simulation Parameters

The simulation settings are

```
nt = 100;
nu = 0.02;
dt = 0.001;
```

The variables have the following meanings:

- `nt` is the number of timesteps,
- `nu` is the diffusion coefficient,
- `dt` is the timestep size.

The small timestep helps maintain numerical stability in the presence of both convection and diffusion.

11.5.3 Constructing the Initial Condition

The solution is initially set equal to one everywhere:

```
u_field = ones(1,nx);
```

A square pulse is then defined:

```
for i = 1:nx

    if x(i) >= 0.5 && x(i) <= 1.0

        u_field(i) = 2.0;

    end

end
```

This creates the initial profile

$$u(x,0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0, \\ 1, & \text{otherwise.} \end{cases}$$

The same initial condition has been used throughout the one-dimensional examples so that the effects of different PDEs can be compared directly.

11.5.4 Saving the Initial Condition

Before time integration begins, the initial profile is stored:

```
u_initial = u_field;
```

This allows comparison between the starting and final states.

11.5.5 Beginning the Time Loop

The numerical simulation advances through time using

```
for n = 1:nt
```

At each timestep, a copy of the current solution is created:

```
un = u_field;
```

This ensures that every update uses values from the previous timestep.

11.5.6 Computing the Convection Term

The nonlinear convection contribution is computed as

```
convection = un(i)*dt/dx ...
          *(un(i)-un(i-1));
```

This expression represents

$$u_i^n \frac{\Delta t}{\Delta x} \left(u_i^n - u_{i-1}^n\right).$$

The convection term tends to steepen the solution.

Regions with larger values move faster than regions with smaller values.

11.5.7 Computing the Diffusion Term

The diffusion contribution is

```
diffusion = nu*dt/dx^2 ...
         *(un(i+1)-2*un(i)+un(i-1));
```

This corresponds to

$$\nu \frac{\Delta t}{\Delta x^2} \left(u_{i+1}^n - 2u_i^n + u_{i-1}^n\right).$$

The diffusion term reduces gradients and smooths sharp features.

11.5.8 Updating the Solution

The complete Burgers' equation update is

```
u_field(i) = un(i) ...
          - convection ...
          + diffusion;
```

This combines both physical mechanisms into a single update.

The negative convection contribution promotes steepening, while the positive diffusion contribution promotes smoothing.

The final solution results from the competition between these two effects.

11.5.9 Applying Boundary Conditions

The boundary values are fixed using

```
u_field(1) = 1.0;
u_field(nx) = 1.0;
```

This corresponds to

$$u(0,t) = 1, \qquad u(L,t) = 1.$$

11.5.10 Visualising the Results

The initial and final profiles are displayed using

```
plot(x,u_initial,'--','LineWidth',2)

hold on

plot(x,u_field,'LineWidth',2)
```

The figure is labelled using

```
xlabel('x')
ylabel('u')

legend('Initial','Final')

title('1D Burgers'' Equation')

grid on
```

The resulting graph clearly illustrates the evolution of the solution.

11.6 Algorithm Summary

The MATLAB solver follows the sequence

1. Generate the computational grid.
2. Define the simulation parameters.
3. Construct the square-pulse initial condition.
4. Store the initial profile.
5. Compute the nonlinear convection term.
6. Compute the diffusion term.
7. Update the solution.
8. Apply boundary conditions.
9. Repeat for each timestep.
10. Plot the results.

11.7 Expected Behaviour

The solution evolves differently from both pure convection and pure diffusion.
The nonlinear convection term tends to steepen the wave front.
The diffusion term simultaneously smooths the developing gradients.
The resulting profile typically exhibits

- gradual spreading,
- distorted wave shapes,
- reduced peak sharpness,
- smoother gradients than the pure nonlinear convection solution.

The final solution reflects the balance between convection and diffusion.

11.8 Effect of the Diffusion Coefficient

The parameter ν determines the relative importance of diffusion.

11.8.1 Small Viscosity

When

$$\nu \ll 1,$$

the convection term dominates.

The solution develops steep gradients and resembles the nonlinear convection case.

11.8.2 Large Viscosity

When

$$\nu \gg 1,$$

diffusion dominates.

The solution becomes increasingly smooth and resembles the diffusion equation.

11.9 Connection to Fluid Dynamics

Burgers' equation contains two essential ingredients of the Navier–Stokes equations:

- nonlinear convection,
- diffusion (viscosity).

The major component still missing is pressure.

In the Navier–Stokes equations, pressure influences the velocity field and couples different regions of the flow together.

Nevertheless, Burgers' equation captures much of the mathematical complexity associated with fluid motion and is therefore an extremely valuable model problem.

11.10 Comparison with Previous Problems

Equation	Main Behaviour
Linear Convection	Wave translation without shape change
Nonlinear Convection	Wave steepening and distortion
Diffusion	Gradient smoothing and spreading
Burgers' Equation	Competition between steepening and smoothing

Burgers' equation can therefore be viewed as the natural combination of the previous two chapters.

11.11 Key Lessons from Problem 5

This chapter introduced the first PDE that combines multiple physical mechanisms.

The main conclusions are:

- nonlinear convection steepens gradients;
- diffusion smooths gradients;
- Burgers' equation combines both effects;

- the diffusion coefficient controls the balance between steepening and smoothing;
- Burgers' equation serves as an important stepping stone toward the Navier–Stokes equations.

The next chapter extends these ideas from one dimension to two dimensions, introducing multidirectional transport and the additional numerical challenges associated with higher-dimensional PDEs.

Chapter 12

Two-Dimensional Linear Convection

12.1 Introduction

The previous chapters examined transport phenomena in one spatial dimension. While these examples introduced the fundamental ideas of discretisation, stability, convection, and diffusion, most practical fluid-flow problems occur in two or three dimensions.

The next step is therefore to extend the linear convection equation to two dimensions.

In two dimensions, information can propagate simultaneously in both the x- and y-directions. Consequently, the numerical scheme must account for spatial variations in both coordinates.

This chapter develops a MATLAB solver for the two-dimensional linear convection equation and demonstrates how a two-dimensional pulse is transported across the computational domain.

12.2 Governing Equation

The two-dimensional linear convection equation is

$$\frac{\partial u}{\partial t} + c_x \frac{\partial u}{\partial x} + c_y \frac{\partial u}{\partial y} = 0, \tag{12.1}$$

where

- $u(x, y, t)$ is the transported quantity,
- c_x is the convection speed in the x-direction,
- c_y is the convection speed in the y-direction.

Unlike the one-dimensional problem, information now propagates simultaneously in two spatial directions.

12.3 Initial Condition

The initial condition consists of a square pulse:

$$u(x, y, 0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0 \text{ and } 0.5 \leq y \leq 1.0, \\ 1, & \text{otherwise.} \end{cases} \tag{12.2}$$

The pulse occupies a square region in the centre of the domain.

All boundaries are maintained at

$$u = 1.$$

12.4 Numerical Discretisation

Using a forward difference in time,

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j}^{n+1} - u_{i,j}^{n}}{\Delta t}, \tag{12.3}$$

and backward differences in space,

$$\frac{\partial u}{\partial x} \approx \frac{u_{i,j}^{n} - u_{i-1,j}^{n}}{\Delta x}, \tag{12.4}$$

$$\frac{\partial u}{\partial y} \approx \frac{u_{i,j}^{n} - u_{i,j-1}^{n}}{\Delta y}, \tag{12.5}$$

gives

$$u_{i,j}^{n+1} = u_{i,j}^{n} - c_x \frac{\Delta t}{\Delta x}\left(u_{i,j}^{n} - u_{i-1,j}^{n}\right) - c_y \frac{\Delta t}{\Delta y}\left(u_{i,j}^{n} - u_{i,j-1}^{n}\right). \tag{12.6}$$

This update formula forms the basis of the MATLAB implementation.

12.5 Code Walkthrough

12.5.1 Creating the Computational Grid

The two-dimensional mesh is generated using

```
nx = 81;
ny = 81;

Lx = 2.0;
Ly = 2.0;

dx = Lx/(nx-1);
dy = Ly/(ny-1);
```

The computational domain extends over

$$0 \le x \le 2, \qquad 0 \le y \le 2.$$

The variables `nx` and `ny` define the number of grid points in the x- and y-directions. The corresponding grid spacings are stored in `dx` and `dy`.

12.5.2 Generating the Coordinates

The coordinate vectors are constructed using

```
x = linspace(0,Lx,nx);
y = linspace(0,Ly,ny);
```

A two-dimensional mesh is then created:

```
[X,Y] = meshgrid(x,y);
```

The matrices `X` and `Y` contain the coordinates of every point in the computational domain. These arrays are later used for visualisation.

12.5.3 Defining the Simulation Parameters

The simulation parameters are

```
nt = 50;
dt = 0.005;

cx = 1.0;
cy = 1.0;
```

The variables `cx` and `cy` represent the convection speeds in the x- and y-directions.

Since both speeds are positive, the pulse will move diagonally toward the upper-right corner of the domain.

12.5.4 Initialising the Solution Field

The solution field is created using

```
u = ones(ny,nx);
```

Unlike the one-dimensional examples, the solution is now stored in a two-dimensional array.

Each entry corresponds to one grid point in the computational domain.

12.5.5 Constructing the Square Pulse

The initial condition is produced using two nested loops:

```
for j = 1:ny

    for i = 1:nx

        if x(i) >= 0.5 && x(i) <= 1.0 && ...
           y(j) >= 0.5 && y(j) <= 1.0

            u(j,i) = 2.0;

        end

    end

end
```

The inner loop moves along the x-direction while the outer loop moves through the y-direction.

Any point lying inside the central square is assigned the value two.

All other locations retain the value one.

12.5.6 Beginning the Time Loop

The solution is advanced using

```
for n = 1:nt
```

At the beginning of each timestep, a temporary copy of the solution is created:

```
un = u;
```

This ensures that all updates are based on values from the previous timestep.

12.5.7 Updating the Interior Points

The two-dimensional convection update is implemented using

```
for j = 2:ny

    for i = 2:nx

        u(j,i) = un(j,i) ...
               - cx*dt/dx ...
               *(un(j,i)-un(j,i-1)) ...
               - cy*dt/dy ...
               *(un(j,i)-un(j-1,i));

    end

end
```

This statement directly implements

$$u_{i,j}^{n+1} = u_{i,j}^{n} - c_x \frac{\Delta t}{\Delta x}\left(u_{i,j}^{n} - u_{i-1,j}^{n}\right) - c_y \frac{\Delta t}{\Delta y}\left(u_{i,j}^{n} - u_{i,j-1}^{n}\right). \tag{12.7}$$

The first convection term transports information along the x-direction.
The second convection term transports information along the y-direction.
Both effects occur simultaneously.

12.5.8 Applying Boundary Conditions

After updating the interior points, the boundaries are fixed:

```
u(1,:) = 1.0;
u(end,:) = 1.0;

u(:,1) = 1.0;
u(:,end) = 1.0;
```

These statements impose

$$u = 1$$

along all four boundaries.
The notation `:` instructs MATLAB to select entire rows or columns.

12.5.9 Visualising the Solution

The numerical solution is displayed using

```
surf(X,Y,u)
```

The function `surf` generates a three-dimensional surface plot.
The coordinate matrices `X` and `Y` define the horizontal axes, while `u` defines the surface height.
The plot is labelled using

```
xlabel('x')
ylabel('y')
zlabel('u')

title('2D Linear Convection')
```

To improve the appearance of the figure, MATLAB is instructed to smooth the surface shading:

```
shading interp
```

12.6 Algorithm Summary

The numerical procedure follows the sequence

1. Generate a two-dimensional computational grid.
2. Define the initial square pulse.
3. Store the solution in a two-dimensional array.
4. Advance the solution using the two-dimensional convection equation.
5. Apply boundary conditions.
6. Repeat for the required number of timesteps.
7. Visualise the final solution.

12.7 Expected Behaviour

Because both convection speeds are positive,

$$c_x > 0, \qquad c_y > 0,$$

the pulse moves diagonally toward increasing values of both x and y.
The exact solution would preserve the shape of the square pulse.
However, the first-order upwind discretisation introduces numerical diffusion.
As a result:

- the pulse becomes smoother,
- sharp corners become rounded,
- the height of the pulse decreases,
- the edges become increasingly smeared.

These effects are analogous to those observed in the one-dimensional convection problem.

12.8 Effect of the Convection Speeds

The direction of pulse motion depends on the signs and magnitudes of c_x and c_y.

12.8.1 Equal Speeds

If

$$c_x = c_y,$$

the pulse moves diagonally across the domain.

12.8.2 Unequal Speeds

If

$$c_x > c_y,$$

the pulse moves predominantly in the x-direction.
Similarly, if

$$c_y > c_x,$$

the pulse moves predominantly in the y-direction.

12.8.3 Negative Speeds

Changing the sign of either convection speed reverses the direction of motion along the corresponding axis.

12.9 Comparison with the One-Dimensional Problem

The one-dimensional convection equation transports information along a single coordinate direction.

The two-dimensional equation extends this concept by allowing transport in multiple directions simultaneously.

Consequently:

- the solution becomes a surface rather than a curve,
- two spatial derivatives must be discretised,
- two convection speeds must be considered,
- visualisation becomes more challenging.

Nevertheless, the underlying numerical ideas remain the same.

12.10 Connection to CFD

Most practical CFD simulations involve two- or three-dimensional domains.

Therefore, the transition from one dimension to two dimensions represents a major step in numerical modelling.

The techniques introduced here form the basis for:

- two-dimensional diffusion,
- multidimensional Burgers' equations,
- incompressible Navier–Stokes solvers,
- compressible flow solvers.

Understanding the storage and manipulation of two-dimensional solution fields is therefore essential for modern CFD.

12.11 Key Lessons from Problem 6

This chapter introduced the first two-dimensional PDE solver in the workbook.

The main conclusions are:

- convection can occur simultaneously in multiple directions;
- solution variables are stored as two-dimensional arrays;
- finite differences must be applied along both coordinate directions;
- boundary conditions must be enforced on all edges of the domain;
- numerical diffusion remains present when first-order upwind schemes are used.

The next chapter extends these ideas further by introducing two-dimensional nonlinear convection, where the propagation speeds are no longer constant but become part of the solution itself.

Chapter 13

Two-Dimensional Non-linear Convection

13.1 Introduction

In the previous chapter, we extended the linear convection equation from one dimension to two dimensions. The convection speeds were prescribed constants and therefore every part of the solution travelled at the same rate.

The next step is to introduce nonlinearity. In nonlinear convection, the velocity field itself determines how information propagates through the domain.

Unlike the linear case, different regions of the flow may move at different speeds. Consequently, the solution can deform, stretch, and develop steep gradients as it evolves.

This chapter develops a MATLAB implementation of the two-dimensional nonlinear convection equations and investigates the interaction between two coupled velocity fields.

13.2 Governing Equations

The two-dimensional nonlinear convection equations are

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = 0, \tag{13.1}$$

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = 0. \tag{13.2}$$

The variables

$$u(x, y, t)$$

and

$$v(x, y, t)$$

represent velocity components in the x- and y-directions.

Unlike the two-dimensional linear convection equation, the propagation speeds are no longer constants. The velocity field itself transports the solution.

13.3 Physical Interpretation

The nonlinear convection terms

$$u\frac{\partial u}{\partial x}$$

and

$$v\frac{\partial u}{\partial y}$$

describe the transport of the u-velocity component.
Similarly,

$$u\frac{\partial v}{\partial x}$$

and

$$v\frac{\partial v}{\partial y}$$

describe the transport of the v-velocity component.
The key feature is that the solution transports itself.
Regions of high velocity move faster than regions of low velocity.
As a result:

- wave shapes distort,
- gradients become steeper,
- the solution evolves nonlinearly,
- the velocity components influence one another.

13.4 Initial Condition

Both velocity components are initialized using a square pulse:

$$u(x,y,0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0 \text{ and } 0.5 \leq y \leq 1.0, \\ 1, & \text{otherwise}, \end{cases} \tag{13.3}$$

and

$$v(x,y,0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0 \text{ and } 0.5 \leq y \leq 1.0, \\ 1, & \text{otherwise}. \end{cases} \tag{13.4}$$

Boundary values are maintained at

$$u = 1, \qquad v = 1.$$

13.5 Numerical Discretisation

Using forward Euler in time and backward differences in space gives

$$u_{i,j}^{n+1} = u_{i,j}^n - u_{i,j}^n \frac{\Delta t}{\Delta x}\left(u_{i,j}^n - u_{i-1,j}^n\right) - v_{i,j}^n \frac{\Delta t}{\Delta y}\left(u_{i,j}^n - u_{i,j-1}^n\right), \tag{13.5}$$

and

$$v_{i,j}^{n+1} = v_{i,j}^n - u_{i,j}^n \frac{\Delta t}{\Delta x}\left(v_{i,j}^n - v_{i-1,j}^n\right) - v_{i,j}^n \frac{\Delta t}{\Delta y}\left(v_{i,j}^n - v_{i,j-1}^n\right). \tag{13.6}$$

These update equations form the basis of the MATLAB implementation.

13.6 Code Walkthrough

13.6.1 Creating the Computational Grid

The computational mesh is generated using

```
nx = 81;
ny = 81;

Lx = 2.0;
Ly = 2.0;

dx = Lx/(nx-1);
dy = Ly/(ny-1);
```

The domain extends over

$$0 \leq x \leq 2, \qquad 0 \leq y \leq 2.$$

The variables `dx` and `dy` define the grid spacing in each coordinate direction.

13.6.2 Generating the Coordinate Arrays

The coordinate vectors are defined using

```
x = linspace(0,Lx,nx);
y = linspace(0,Ly,ny);
```

A two-dimensional mesh is then generated:

```
[X,Y] = meshgrid(x,y);
```

The arrays `X` and `Y` contain the coordinates of every grid point.

13.6.3 Defining the Simulation Parameters

The timestep information is specified by

```
nt = 50;
dt = 0.001;
```

The smaller timestep is necessary because nonlinear convection produces stronger gradients than linear convection.

13.6.4 Initialising the Velocity Fields

The two velocity components are initialised as

```
u = ones(ny,nx);
v = ones(ny,nx);
```

Initially,

$$u = 1, \qquad v = 1$$

everywhere in the domain.

13.6.5 Constructing the Square Pulse

The square pulse is created using

```
for j = 1:ny

    for i = 1:nx

        if x(i) >= 0.5 && x(i) <= 1.0 && ...
           y(j) >= 0.5 && y(j) <= 1.0

            u(j,i) = 2.0;
            v(j,i) = 2.0;

        end

    end

end
```

Any point lying inside the central square region is assigned the value two.
Outside this region the velocity remains equal to one.

13.6.6 Beginning the Time Loop

The simulation advances through time using

```
for n = 1:nt
```

A copy of the current solution is stored:

```
un = u;
vn = v;
```

The copies are required because all updates must use information from the previous timestep.

13.6.7 Updating the u-Velocity Component

The update equation for the u-velocity is

```
u(j,i) = un(j,i) ...
      - un(j,i)*dt/dx ...
      *(un(j,i)-un(j,i-1)) ...
      - vn(j,i)*dt/dy ...
      *(un(j,i)-un(j-1,i));
```

This expression implements

$$u_{i,j}^{n+1} = u_{i,j}^n - u_{i,j}^n \frac{\Delta t}{\Delta x}\left(u_{i,j}^n - u_{i-1,j}^n\right) - v_{i,j}^n \frac{\Delta t}{\Delta y}\left(u_{i,j}^n - u_{i,j-1}^n\right). \tag{13.7}$$

The first term represents convection in the x-direction.
The second term represents convection in the y-direction.

13.6.8 Updating the v-Velocity Component

The update equation for the v-velocity is

```
v(j,i) = vn(j,i) ...
      - un(j,i)*dt/dx ...
      *(vn(j,i)-vn(j,i-1)) ...
      - vn(j,i)*dt/dy ...
      *(vn(j,i)-vn(j-1,i));
```

This equation is similar to the u-equation but operates on the second velocity component. The equations are coupled because both u and v appear in each update.

13.6.9 Applying Boundary Conditions

After updating the interior points, boundary conditions are enforced:

```
u(1,:) = 1.0;
u(end,:) = 1.0;
u(:,1) = 1.0;
u(:,end) = 1.0;

v(1,:) = 1.0;
v(end,:) = 1.0;
v(:,1) = 1.0;
v(:,end) = 1.0;
```

This imposes

$$u = 1, \qquad v = 1$$

on all domain boundaries.

13.6.10 Visualising the u-Velocity

The first velocity component is visualised using

```
figure

surf(X,Y,u)

title('u velocity')

xlabel('x')
ylabel('y')
zlabel('u')

shading interp
```

This produces a surface plot of the u-velocity field.

13.6.11 Visualising the v-Velocity

The second velocity component is displayed similarly:

```
figure

surf(X,Y,v)

title('v velocity')

xlabel('x')
ylabel('y')
```

```
9  zlabel('v')
10
11 shading interp
```

This allows both velocity fields to be analysed independently.

13.7 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the two-dimensional grid.
2. Initialise the velocity fields.
3. Create the square-pulse initial condition.
4. Store copies of the previous timestep.
5. Update the u-velocity field.
6. Update the v-velocity field.
7. Apply boundary conditions.
8. Repeat for all timesteps.
9. Plot both velocity fields.

13.8 Expected Behaviour

The solution evolves differently from the two-dimensional linear convection problem.
Because the propagation speed varies throughout the domain:

- the wave shape distorts,
- gradients become steeper,
- the square pulse no longer maintains its original form,
- the velocity fields evolve nonlinearly,
- the u and v components influence each other.

The central pulse gradually loses its square shape as the simulation progresses.

13.9 Coupling Between the Velocity Components

Unlike scalar convection equations, two velocity fields must now be solved simultaneously.
The update of u depends on both

$$u$$

and

$$v,$$

and the update of v also depends on both variables.
Consequently, neither equation can be solved independently.
This coupling is one of the defining characteristics of multidimensional fluid-flow problems.

13.10 Connection to the Navier–Stokes Equations

The nonlinear convection terms appearing in this chapter are identical in form to those found in the incompressible Navier–Stokes equations.

The only major components still missing are:

- pressure forces,
- viscous diffusion,
- incompressibility constraints.

Nevertheless, the nonlinear transport mechanisms responsible for much of the mathematical complexity of fluid dynamics are already present.

13.11 Key Lessons from Problem 7

This chapter introduced the first coupled system of transport equations in the workbook.

The main conclusions are:

- two velocity components must be solved simultaneously;
- the velocity field transports itself;
- nonlinear effects distort the solution;
- gradients tend to steepen as the simulation progresses;
- the governing equations provide an important step toward the Navier–Stokes equations.

The next chapter introduces two-dimensional diffusion, which acts to smooth the gradients generated by nonlinear convection.

Chapter 14

Two-Dimensional Diffusion

14.1 Introduction

In the previous chapter, we examined two-dimensional nonlinear convection, where velocity fields transported themselves through the computational domain and generated increasingly distorted solution profiles.

Diffusion acts in the opposite manner. Rather than transporting information, diffusion spreads information and reduces gradients.

The two-dimensional diffusion equation is one of the fundamental equations of mathematical physics and appears in many engineering applications, including:

- heat conduction,
- mass transport,
- pollutant dispersion,
- viscous momentum diffusion,
- groundwater flow.

This chapter develops a MATLAB implementation of the two-dimensional diffusion equation and investigates how a square pulse spreads throughout a domain.

14.2 Governing Equation

The two-dimensional diffusion equation is

$$\frac{\partial u}{\partial t} = \nu \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right), \tag{14.1}$$

where

- $u(x, y, t)$ is the transported quantity,
- ν is the diffusion coefficient.

The Laplacian operator

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}$$

measures the curvature of the solution in both spatial directions.
Large curvature leads to stronger diffusion.

14.3 Physical Interpretation

Diffusion acts to reduce spatial gradients.

Whenever a region contains values significantly larger than its surroundings, diffusion transfers information outward into neighbouring cells.

Consequently:

- sharp gradients are smoothed,
- discontinuities become less pronounced,
- peaks decrease in magnitude,
- the solution becomes increasingly uniform.

Unlike convection, diffusion has no preferred direction.
Instead, spreading occurs equally in all directions.

14.4 Initial Condition

The solution begins as a square pulse:

$$u(x,y,0) = \begin{cases} 2, & 0.5 \le x \le 1.0 \text{ and } 0.5 \le y \le 1.0, \\ 1, & \text{otherwise.} \end{cases} \tag{14.2}$$

The pulse is positioned near the centre of the domain.
All boundaries are maintained at

$$u = 1.$$

14.5 Numerical Discretisation

Using a forward difference in time,

$$\frac{\partial u}{\partial t} \approx \frac{u_{i,j}^{n+1} - u_{i,j}^{n}}{\Delta t}, \tag{14.3}$$

and central differences for the second derivatives,

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j}^{n} - 2u_{i,j}^{n} + u_{i-1,j}^{n}}{\Delta x^2}, \tag{14.4}$$

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1}^{n} - 2u_{i,j}^{n} + u_{i,j-1}^{n}}{\Delta y^2}, \tag{14.5}$$

gives

$$u_{i,j}^{n+1} = u_{i,j}^{n} + \nu \Delta t \left(\frac{u_{i+1,j}^{n} - 2u_{i,j}^{n} + u_{i-1,j}^{n}}{\Delta x^2} + \frac{u_{i,j+1}^{n} - 2u_{i,j}^{n} + u_{i,j-1}^{n}}{\Delta y^2} \right). \tag{14.6}$$

This update equation forms the basis of the MATLAB solver.

14.6 Code Walkthrough

14.6.1 Generating the Computational Grid

The computational domain is defined using

```
nx = 81;
ny = 81;

Lx = 2.0;
Ly = 2.0;

dx = Lx/(nx-1);
dy = Ly/(ny-1);
```

The domain extends over

$$0 \leq x \leq 2, \qquad 0 \leq y \leq 2.$$

The variables `dx` and `dy` contain the grid spacing in each coordinate direction.

14.6.2 Generating Coordinate Arrays

The coordinates are generated using

```
x = linspace(0,Lx,nx);
y = linspace(0,Ly,ny);

[X,Y] = meshgrid(x,y);
```

The matrices `X` and `Y` contain the coordinates of every point in the domain and are later used for visualisation.

14.6.3 Defining the Simulation Parameters

The timestep information is defined as

```
nt = 100;

nu = 0.05;

dt = 0.0005;
```

The variables have the following meanings:

- `nt` is the number of timesteps,
- `nu` is the diffusion coefficient,
- `dt` is the timestep size.

The small timestep helps satisfy the stability restriction associated with explicit diffusion schemes.

14.6.4 Initialising the Solution Field

The solution array is created using

```
u_field = ones(ny,nx);
```

The solution is therefore initially equal to one everywhere.

14.6.5 Constructing the Square Pulse

The square pulse is introduced using

```
for j = 1:ny

    for i = 1:nx

        if x(i) >= 0.5 && x(i) <= 1.0 && ...
           y(j) >= 0.5 && y(j) <= 1.0

            u_field(j,i) = 2.0;

        end

    end

end
```

The central square is assigned the value two while the remainder of the domain remains equal to one.

14.6.6 Beginning the Time Integration

The numerical solution advances using

```
for n = 1:nt
```

At the start of each timestep a copy of the solution is stored:

```
un = u_field;
```

This prevents newly updated values from influencing subsequent calculations.

14.6.7 Computing the Diffusion Terms

The second derivative in the x-direction is evaluated as

```
diffusion_x = ...
(un(j,i+1)-2*un(j,i)+un(j,i-1))/dx^2;
```

This approximates

$$\frac{\partial^2 u}{\partial x^2}.$$

Similarly, the second derivative in the y-direction is

```
diffusion_y = ...
(un(j+1,i)-2*un(j,i)+un(j-1,i))/dy^2;
```

which approximates

$$\frac{\partial^2 u}{\partial y^2}.$$

14.6.8 Updating the Interior Points

The numerical solution is updated using

```
u_field(j,i) = un(j,i) ...
            + nu*dt ...
            *(diffusion_x + diffusion_y);
```

This directly implements

$$u_{i,j}^{n+1} = u_{i,j}^{n} + \nu \Delta t \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right). \tag{14.7}$$

The Laplacian operator distributes information in all directions.

14.6.9 Applying Boundary Conditions

After the interior points are updated, the boundaries are fixed:

```
u_field(1,:) = 1.0;
u_field(end,:) = 1.0;

u_field(:,1) = 1.0;
u_field(:,end) = 1.0;
```

This imposes

$$u = 1$$

on all four boundaries throughout the simulation.

14.6.10 Visualising the Solution

The final solution is displayed using

```
surf(X,Y,u_field)
```

The plot is labelled using

```
xlabel('x')
ylabel('y')
zlabel('u')

title('2D Diffusion')
```

To improve visual quality, the surface shading is smoothed:

```
shading interp
```

14.7 Algorithm Summary

The numerical procedure follows the sequence

1. Generate a two-dimensional computational grid.
2. Define the diffusion coefficient and timestep.
3. Construct the square-pulse initial condition.
4. Compute second derivatives in the x- and y-directions.
5. Update the solution using the diffusion equation.

6. Apply boundary conditions.
7. Repeat for all timesteps.
8. Visualise the final solution.

14.8 Expected Behaviour

Initially, the solution contains sharp corners and large gradients.

As the simulation progresses:

- the square pulse spreads outward,
- the peak height decreases,
- sharp corners become rounded,
- gradients become smoother,
- the domain approaches a more uniform state.

Unlike convection, there is no net movement of the pulse.
Instead, the pulse simply spreads in all directions.

14.9 Effect of the Diffusion Coefficient

The diffusion coefficient determines the rate of spreading.

14.9.1 Small Diffusion Coefficient

When

$$\nu \ll 1,$$

diffusion is weak.
The pulse remains relatively sharp for a long period of time.

14.9.2 Large Diffusion Coefficient

When

$$\nu \gg 1,$$

diffusion becomes much stronger.
The pulse rapidly spreads across the computational domain and the solution quickly approaches a uniform distribution.

14.10 Two-Dimensional Stability Condition

The explicit diffusion equation imposes a stricter stability requirement than its one-dimensional counterpart.

For a uniform grid with

$$\Delta x = \Delta y,$$

a typical stability condition is

$$\nu \frac{\Delta t}{\Delta x^2} \leq \frac{1}{4}. \tag{14.8}$$

As the grid becomes finer, the timestep must be reduced significantly.

This is one reason why diffusion-dominated simulations can become computationally expensive.

14.11 Comparison with Two-Dimensional Convection

The two-dimensional linear convection equation transports information through the domain.

The two-dimensional diffusion equation spreads information through the domain.

Consequently:

Convection	Diffusion
Moves the pulse	Spreads the pulse
Directional transport	Isotropic spreading
Preserves structures	Smooths structures
Can transport sharp fronts	Eliminates sharp fronts

Understanding both processes is essential because they appear simultaneously in fluid mechanics.

14.12 Connection to CFD

In fluid dynamics, diffusion represents viscous effects.

Viscosity transfers momentum from high-speed regions toward low-speed regions, reducing velocity gradients.

The diffusion operator therefore plays a central role in the Navier–Stokes equations.

The numerical techniques introduced in this chapter are directly applicable to viscous flow simulations.

14.13 Key Lessons from Problem 8

This chapter introduced the two-dimensional diffusion equation.

The main conclusions are:

- diffusion spreads information in all spatial directions;
- second derivatives measure curvature and drive the diffusion process;
- sharp gradients and corners are rapidly smoothed;
- explicit diffusion schemes require careful timestep selection for stability;
- diffusion represents the mathematical basis of viscous effects in fluid mechanics.

The next chapter combines two-dimensional nonlinear convection with diffusion to produce the two-dimensional Burgers' equations, which provide an even closer approximation to the structure of the Navier–Stokes equations.

Chapter 15

Two-Dimensional Burgers' Equations

15.1 Introduction

The previous two chapters introduced two-dimensional nonlinear convection and two-dimensional diffusion separately.

In many fluid-flow problems, these two mechanisms occur simultaneously. Nonlinear convection transports momentum through the domain, while diffusion acts to smooth velocity gradients.

The two-dimensional Burgers' equations combine these effects into a coupled system of partial differential equations and provide an important stepping stone toward the Navier–Stokes equations.

This chapter develops a MATLAB implementation of the two-dimensional Burgers' equations and investigates the interaction between nonlinear transport and diffusion within a two-dimensional velocity field.

15.2 Governing Equations

The two-dimensional Burgers' equations are

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = \nu\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right), \tag{15.1}$$

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = \nu\left(\frac{\partial^2 v}{\partial x^2} + \frac{\partial^2 v}{\partial y^2}\right). \tag{15.2}$$

The variables

$$u(x, y, t)$$

and

$$v(x, y, t)$$

represent the velocity components in the x- and y-directions respectively.

The parameter ν is the diffusion coefficient.

15.3 Physical Interpretation

The nonlinear convection terms

$$u\frac{\partial}{\partial x}$$

and

$$v\frac{\partial}{\partial y}$$

transport momentum through the domain.

These terms cause large velocity regions to move more rapidly than neighbouring low-velocity regions and therefore tend to steepen gradients.

The diffusion terms

$$\nu\nabla^2 u$$

and

$$\nu\nabla^2 v$$

act in the opposite direction by smoothing those gradients.

Consequently, Burgers' equations represent a balance between:

- nonlinear steepening,
- viscous smoothing.

This competition is one of the defining characteristics of fluid mechanics.

15.4 Initial Condition

Both velocity fields are initialized as

$$u(x,y,0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0 \text{ and } 0.5 \leq y \leq 1.0, \\ 1, & \text{otherwise}, \end{cases} \tag{15.3}$$

and

$$v(x,y,0) = \begin{cases} 2, & 0.5 \leq x \leq 1.0 \text{ and } 0.5 \leq y \leq 1.0, \\ 1, & \text{otherwise}. \end{cases} \tag{15.4}$$

The velocity field therefore contains a square pulse embedded within a uniform background.

Boundary values are maintained at

$$u = 1, \qquad v = 1.$$

15.5 Numerical Discretisation

The update equation for the u-velocity component is

$$u_{i,j}^{n+1} = u_{i,j}^n - u_{i,j}^n \frac{\Delta t}{\Delta x}\left(u_{i,j}^n - u_{i-1,j}^n\right) - v_{i,j}^n \frac{\Delta t}{\Delta y}\left(u_{i,j}^n - u_{i,j-1}^n\right) \tag{15.5}$$

$$+ \nu\Delta t\left[\frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} + \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2}\right]. \tag{15.6}$$

Similarly,

$$v_{i,j}^{n+1} = v_{i,j}^n - u_{i,j}^n \frac{\Delta t}{\Delta x}\left(v_{i,j}^n - v_{i-1,j}^n\right) - v_{i,j}^n \frac{\Delta t}{\Delta y}\left(v_{i,j}^n - v_{i,j-1}^n\right) \tag{15.7}$$

$$+ \nu \Delta t \left[\frac{v_{i+1,j}^n - 2v_{i,j}^n + v_{i-1,j}^n}{\Delta x^2} + \frac{v_{i,j+1}^n - 2v_{i,j}^n + v_{i,j-1}^n}{\Delta y^2}\right]. \tag{15.8}$$

These equations are direct extensions of the one-dimensional Burgers' equation examined previously.

15.6 Code Walkthrough

15.6.1 Generating the Computational Grid

The computational mesh is generated using

```
nx = 81;
ny = 81;

Lx = 2.0;
Ly = 2.0;

dx = Lx/(nx-1);
dy = Ly/(ny-1);
```

The computational domain extends over

$$0 \le x \le 2, \qquad 0 \le y \le 2.$$

15.6.2 Creating the Coordinate Arrays

The coordinate vectors are generated using

```
x = linspace(0,Lx,nx);
y = linspace(0,Ly,ny);

[X,Y] = meshgrid(x,y);
```

The matrices `X` and `Y` store the coordinates of every computational point.

15.6.3 Defining the Simulation Parameters

The numerical parameters are

```
nt = 120;

nu = 0.01;

dt = 0.001;
```

Here

- `nt` is the number of timesteps,
- `nu` is the diffusion coefficient,
- `dt` is the timestep size.

The timestep must be sufficiently small to maintain stability in the presence of both convection and diffusion.

15.6.4 Initialising the Velocity Fields

The velocity arrays are created using

```
u = ones(ny,nx);
v = ones(ny,nx);
```

Initially,

$$u = 1, \qquad v = 1$$

throughout the domain.

15.6.5 Creating the Square Pulse

The square pulse is constructed using

```
for j = 1:ny

    for i = 1:nx

        if x(i) >= 0.5 && x(i) <= 1.0 && ...
           y(j) >= 0.5 && y(j) <= 1.0

            u(j,i) = 2.0;
            v(j,i) = 2.0;

        end

    end

end
```

Both velocity components are assigned the same initial condition.

15.6.6 Beginning the Time Integration

The simulation advances using

```
for n = 1:nt
```

The current solution is first copied:

```
un = u;
vn = v;
```

The copies are used throughout the update process.

15.6.7 Computing the Convection Terms

The convection contribution to the u-equation is

```
conv_u = ...
un(j,i)*dt/dx*(un(j,i)-un(j,i-1)) ...
+ vn(j,i)*dt/dy*(un(j,i)-un(j-1,i));
```

Similarly, the convection contribution for the v-equation is

```
conv_v = ...
un(j,i)*dt/dx*(vn(j,i)-vn(j,i-1)) ...
+ vn(j,i)*dt/dy*(vn(j,i)-vn(j-1,i));
```

These terms transport momentum through the computational domain.

15.6.8 Computing the Diffusion Terms

The diffusion term for the u-velocity component is

```
diff_u = nu*dt*( ...
(un(j,i+1)-2*un(j,i)+un(j,i-1))/dx^2 ...
+(un(j+1,i)-2*un(j,i)+un(j-1,i))/dy^2 );
```

The corresponding v-equation term is

```
diff_v = nu*dt*( ...
(vn(j,i+1)-2*vn(j,i)+vn(j,i-1))/dx^2 ...
+(vn(j+1,i)-2*vn(j,i)+vn(j-1,i))/dy^2 );
```

These terms smooth velocity gradients.

15.6.9 Updating the Velocity Fields

The final updates are

```
u(j,i) = un(j,i) - conv_u + diff_u;

v(j,i) = vn(j,i) - conv_v + diff_v;
```

The convection terms promote steepening while the diffusion terms counteract this behaviour.

15.6.10 Applying Boundary Conditions

After updating the interior points, boundary values are enforced:

```
u(1,:) = 1;
u(end,:) = 1;
u(:,1) = 1;
u(:,end) = 1;

v(1,:) = 1;
v(end,:) = 1;
v(:,1) = 1;
v(:,end) = 1;
```

This imposes constant boundary values around the entire domain.

15.6.11 Visualising the Solution

The u-velocity field is displayed using

```
figure

surf(X,Y,u)

title('u Velocity')
xlabel('x')
ylabel('y')
zlabel('u')

shading interp
```

Similarly, the v-velocity field is visualised using

```
figure

surf(X,Y,v)

title('v Velocity')
xlabel('x')
ylabel('y')
zlabel('v')

shading interp
```

15.7 Algorithm Summary

The MATLAB solver follows the sequence

1. Generate the two-dimensional computational grid.
2. Initialise the velocity fields.
3. Create the square-pulse initial condition.
4. Compute convection contributions.
5. Compute diffusion contributions.
6. Update both velocity components.
7. Apply boundary conditions.
8. Repeat for all timesteps.
9. Visualise the final solution.

15.8 Expected Behaviour

The solution evolves through the competition between nonlinear convection and diffusion.

The convection terms tend to:

- transport the pulse,
- distort the solution,
- steepen velocity gradients.

The diffusion terms tend to:

- smooth the gradients,
- reduce sharp features,
- spread the solution.

The final shape depends on the balance between these competing mechanisms.

15.9 Effect of the Diffusion Coefficient

The parameter ν controls the relative importance of diffusion.

15.9.1 Small Diffusion Coefficient

When

$$\nu \ll 1,$$

convection dominates.
Sharp gradients persist and strong nonlinear behaviour develops.

15.9.2 Large Diffusion Coefficient

When

$$\nu \gg 1,$$

diffusion dominates.
Velocity gradients are quickly smoothed and the solution becomes increasingly uniform.

15.10 Connection to the Navier–Stokes Equations

The structure of Burgers' equations closely resembles that of the incompressible Navier–Stokes equations.
The system already contains:

- nonlinear convection,
- viscous diffusion,
- multidimensional velocity fields.

The primary ingredients still missing are:

- pressure gradients,
- incompressibility constraints.

For this reason, Burgers' equations are often regarded as simplified Navier–Stokes equations.

15.11 Key Lessons from Problem 9

This chapter introduced the two-dimensional Burgers' equations.
The main conclusions are:

- convection and diffusion operate simultaneously;
- velocity gradients may either steepen or smooth depending on local conditions;
- both velocity components are coupled;
- diffusion moderates the nonlinear effects produced by convection;
- Burgers' equations provide an important intermediate step toward solving the Navier–Stokes equations.

The next chapter introduces the Laplace equation, which forms the foundation of many steady-state potential-flow and pressure-correction methods used in Computational Fluid Dynamics.

Chapter 16

Laplace's Equation

16.1 Introduction

The previous chapters focused on transient transport equations involving convection and diffusion. In this chapter, we consider a different class of partial differential equation: an elliptic equation.

Laplace's equation arises in many areas of engineering and physics, including:

- electrostatics,
- heat conduction,
- groundwater flow,
- potential flow theory,
- pressure-correction methods in CFD.

Unlike the transport equations studied previously, Laplace's equation describes a steady-state solution.

There is no time dependence, and the solution is determined entirely by the boundary conditions.

16.2 Governing Equation

The two-dimensional Laplace equation is

$$\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} = 0, \tag{16.1}$$

where

$$p(x, y)$$

is the unknown variable.

The quantity

$$\nabla^2 p$$

is known as the Laplacian of p.

Laplace's equation states that the curvature in the x-direction exactly balances the curvature in the y-direction.

16.3 Physical Interpretation

Laplace's equation governs equilibrium states.

For example, in heat conduction:

- regions of high temperature transfer heat to neighbouring regions,
- regions of low temperature receive heat from neighbouring regions,
- eventually a steady-state temperature distribution is established.

The final steady-state solution satisfies Laplace's equation.

An important property of Laplace solutions is the mean-value principle:

> The value at a point equals the average of its surrounding values.

This principle forms the basis of the numerical discretisation.

16.4 Boundary Conditions

Consider a rectangular domain

$$0 \leq x \leq 2, \qquad 0 \leq y \leq 1.$$

The boundary conditions are

$$p(0, y) = 0, \tag{16.2}$$

$$p(2, y) = y, \tag{16.3}$$

$$\frac{\partial p}{\partial y}(x, 0) = 0, \tag{16.4}$$

$$\frac{\partial p}{\partial y}(x, 1) = 0. \tag{16.5}$$

The left boundary is fixed at zero.

The right boundary varies linearly with y.

The upper and lower boundaries satisfy zero-gradient conditions.

16.5 Numerical Discretisation

Using central differences for the second derivatives,

$$\frac{\partial^2 p}{\partial x^2} \approx \frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2}, \tag{16.6}$$

and

$$\frac{\partial^2 p}{\partial y^2} \approx \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2}, \tag{16.7}$$

gives

$$\frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2} = 0. \tag{16.8}$$

Rearranging for $p_{i,j}$ yields

$$p_{i,j} = \frac{\Delta y^2 \left(p_{i+1,j} + p_{i-1,j}\right) + \Delta x^2 \left(p_{i,j+1} + p_{i,j-1}\right)}{2\left(\Delta x^2 + \Delta y^2\right)}. \tag{16.9}$$

This expression is used iteratively until convergence.

16.6 Code Walkthrough

16.6.1 Generating the Computational Grid

The computational mesh is created using

```
nx = 31;
ny = 31;

Lx = 2.0;
Ly = 1.0;

dx = Lx/(nx-1);
dy = Ly/(ny-1);
```

The domain extends from

$$0 \le x \le 2, \qquad 0 \le y \le 1.$$

The mesh contains 31×31 grid points.

16.6.2 Generating Coordinate Arrays

The coordinates are generated using

```
x = linspace(0,Lx,nx);
y = linspace(0,Ly,ny);

[X,Y] = meshgrid(x,y);
```

The arrays `X` and `Y` contain the coordinates of every grid point.

16.6.3 Initializing the Solution Field

The pressure field is initialized as

```
p = zeros(ny,nx);
```

Initially, every grid point contains zero.
The correct solution emerges through the iterative process.

16.6.4 Applying Boundary Conditions

The prescribed boundary conditions are imposed using

```
p(:,1) = 0;
p(:,end) = y';
```

The left boundary is fixed at zero.
The right boundary varies linearly from

$$p = 0$$

to

$$p = 1.$$

The upper and lower Neumann boundary conditions are implemented using

```
p(1,:) = p(2,:);
p(end,:) = p(end-1,:);
```

These statements force the normal derivative to be zero.

16.6.5 Defining the Convergence Criterion

The iterative procedure continues until the solution changes by only a small amount:

```
tolerance = 1.0e-4;
error = 1.0;
```

The variable `error` measures the difference between successive iterations.

16.6.6 Beginning the Iterative Loop

The numerical procedure is written as

```
while error > tolerance
```

The loop continues until convergence is achieved.
At the beginning of each iteration, a copy of the current solution is stored:

```
pn = p;
```

16.6.7 Updating the Interior Points

The Laplace equation update is implemented using

```
for j = 2:ny-1

    for i = 2:nx-1

        p(j,i) = ...
        (dy^2*(pn(j,i+1)+pn(j,i-1)) ...
        + dx^2*(pn(j+1,i)+pn(j-1,i))) ...
        /(2*(dx^2+dy^2));

    end

end
```

This statement directly implements the finite-difference approximation of Laplace's equation.
The value at each point becomes a weighted average of its neighbouring points.

16.6.8 Reapplying Boundary Conditions

After updating the interior points, the boundary conditions are enforced again:

```
p(:,1) = 0;
p(:,end) = y';

p(1,:) = p(2,:);
p(end,:) = p(end-1,:);
```

This guarantees that the solution always satisfies the prescribed constraints.

16.6.9 Computing the Error

Convergence is monitored using

```
error = max(max(abs(p-pn)));
```

The iteration terminates when the maximum change becomes smaller than the specified tolerance.

16.6.10 Visualising the Solution

The final solution is displayed using

```
surf(X,Y,p)
```

The figure is labelled using

```
xlabel('x')
ylabel('y')
zlabel('p')

title('Laplace Equation Solution')

shading interp
```

The resulting surface represents the steady-state pressure field.

16.7 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the computational grid.
2. Initialise the pressure field.
3. Apply the boundary conditions.
4. Store the previous solution.
5. Update the interior points.
6. Reapply the boundary conditions.
7. Calculate the convergence error.
8. Repeat until the error falls below the tolerance.
9. Visualise the solution.

16.8 Expected Behaviour

The pressure field gradually adjusts to satisfy all boundary conditions.
The solution evolves smoothly throughout the domain.
Eventually:

- the pressure field becomes steady,
- large gradients disappear,
- the solution satisfies Laplace's equation,
- further iterations produce negligible changes.

The final surface smoothly connects all prescribed boundary values.

16.9 Importance of Boundary Conditions

Unlike convection and diffusion equations, Laplace's equation contains no time derivative.

Consequently, the entire solution is determined solely by the boundary conditions.

Changing the boundary conditions completely alters the resulting solution.

This characteristic makes elliptic equations extremely sensitive to how boundaries are specified.

16.10 Connection to CFD

Laplace's equation plays an important role in Computational Fluid Dynamics.

Applications include:

- potential flow simulations,
- streamfunction formulations,
- pressure correction methods,
- incompressible flow algorithms.

Many CFD solvers repeatedly solve Laplace-like equations as part of larger numerical procedures.

Understanding Laplace's equation is therefore essential before progressing to the pressure-Poisson equation.

16.11 Key Lessons from Problem 10

This chapter introduced the first elliptic PDE in the workbook.

The main conclusions are:

- Laplace's equation describes steady-state equilibrium solutions;
- the solution is determined entirely by the boundary conditions;
- finite-difference methods replace derivatives with neighbouring-point averages;
- iterative methods are used to obtain the numerical solution;
- Laplace's equation forms the foundation for many pressure-based CFD algorithms.

The next chapter extends these ideas to Poisson's equation, where a source term is introduced into the governing equation and the pressure field is influenced by internal forcing within the domain.

Chapter 17

Poisson's Equation

17.1 Introduction

In the previous chapter, we solved Laplace's equation, which describes steady-state equilibrium problems with no internal sources.

Many practical engineering applications, however, involve internal generation or forcing terms. Examples include:

- heat generation within a solid,
- electrical charge distributions,
- fluid pressure fields,
- gravitational potentials.

Such problems are governed by Poisson's equation.

Poisson's equation is one of the most important equations in Computational Fluid Dynamics because it forms the basis of the pressure-correction step used in incompressible Navier–Stokes solvers.

This chapter develops a MATLAB implementation of Poisson's equation and demonstrates how source terms influence the solution.

17.2 Governing Equation

The two-dimensional Poisson equation is

$$\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} = b, \tag{17.1}$$

where

- $p(x, y)$ is the unknown variable,
- $b(x, y)$ is a source term.

The Laplacian operator may be written as

$$\nabla^2 p = b.$$

Unlike Laplace's equation,

$$\nabla^2 p = 0,$$

Poisson's equation contains forcing within the domain.

17.3 Physical Interpretation

The source term alters the equilibrium solution.

Positive sources act like generators, increasing the local value of the solution.

Negative sources act like sinks, decreasing the local value.

A useful interpretation is to imagine a stretched membrane:

- Laplace's equation describes a membrane influenced only by its boundaries,
- Poisson's equation describes a membrane pushed or pulled internally by applied forces.

The resulting shape depends on both the boundary conditions and the internal sources.

17.4 Source Distribution

For this example, two point sources are introduced.

The source array is defined such that

$$b = 100$$

at one location and

$$b = -100$$

at another.

These act as a source-sink pair.

The remainder of the domain has

$$b = 0.$$

The resulting pressure field is therefore influenced by both positive and negative internal forcing.

17.5 Numerical Discretisation

Using central differences,

$$\frac{\partial^2 p}{\partial x^2} \approx \frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2}, \tag{17.2}$$

and

$$\frac{\partial^2 p}{\partial y^2} \approx \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2}, \tag{17.3}$$

gives

$$\frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2} = b_{i,j}. \tag{17.4}$$

Rearranging yields

$$p_{i,j} = \frac{(p_{i+1,j} + p_{i-1,j})\Delta y^2 + (p_{i,j+1} + p_{i,j-1})\Delta x^2 - b_{i,j}\Delta x^2 \Delta y^2}{2(\Delta x^2 + \Delta y^2)}. \tag{17.5}$$

This expression forms the basis of the iterative solver.

17.6 Code Walkthrough

17.6.1 Generating the Computational Grid

The computational mesh is generated using

```
nx = 50;
ny = 50;

Lx = 2.0;
Ly = 1.0;

dx = Lx/(nx-1);
dy = Ly/(ny-1);
```

The computational domain extends over

$$0 \leq x \leq 2, \qquad 0 \leq y \leq 1.$$

17.6.2 Generating Coordinate Arrays

The coordinates are defined using

```
x = linspace(0,Lx,nx);
y = linspace(0,Ly,ny);

[X,Y] = meshgrid(x,y);
```

The arrays `X` and `Y` store the coordinates of every grid point.

17.6.3 Initialising the Solution

The pressure field is initialised using

```
p = zeros(ny,nx);
```

Initially, the solution is zero everywhere.

17.6.4 Creating the Source Term

The source distribution is stored in an array:

```
b = zeros(ny,nx);
```

Two sources are then introduced:

```
b(round(ny/4),round(nx/4)) = 100;

b(round(3*ny/4),round(3*nx/4)) = -100;
```

The first location acts as a source while the second acts as a sink.
This source-sink pair generates a non-trivial pressure field.

17.6.5 Applying Boundary Conditions

The boundary values are fixed at zero:

```
p(:,1) = 0;
p(:,end) = 0;

p(1,:) = 0;
p(end,:) = 0;
```

Thus,

$$p = 0$$

along all boundaries of the domain.

17.6.6 Defining the Number of Iterations

The iterative procedure is performed for a prescribed number of iterations:

```
nit = 100;
```

The larger the value of `nit`, the closer the numerical solution approaches the exact steady-state solution.

17.6.7 Beginning the Iterative Procedure

The iteration loop is

```
for n = 1:nit
```

At the beginning of each iteration, the previous solution is stored:

```
pn = p;
```

17.6.8 Updating the Interior Points

The finite-difference approximation of Poisson's equation is implemented using

```
for j = 2:ny-1

    for i = 2:nx-1

        p(j,i) = ...
        ((pn(j,i+1)+pn(j,i-1))*dy^2 ...
        +(pn(j+1,i)+pn(j-1,i))*dx^2 ...
        - b(j,i)*dx^2*dy^2) ...
        /(2*(dx^2+dy^2));

    end

end
```

This equation differs from Laplace's equation through the appearance of the source term

$$b_{i,j}.$$

The source modifies the local equilibrium state.

17.6.9 Reapplying Boundary Conditions

After updating the interior points, the boundaries are enforced again:

```
p(:,1) = 0;
p(:,end) = 0;

p(1,:) = 0;
p(end,:) = 0;
```

This ensures that the solution always satisfies the prescribed conditions.

17.6.10 Visualising the Pressure Field

The final solution is displayed using

```
surf(X,Y,p)
```

The figure is labelled using

```
xlabel('x')
ylabel('y')
zlabel('p')

title('Poisson Equation Solution')

shading interp
```

The resulting surface illustrates how the pressure field responds to the source-sink pair.

17.7 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the computational grid.
2. Initialise the pressure field.
3. Construct the source distribution.
4. Apply boundary conditions.
5. Store the previous solution.
6. Update the interior points.
7. Reapply boundary conditions.
8. Repeat for the prescribed number of iterations.
9. Visualise the final solution.

17.8 Expected Behaviour

The presence of positive and negative sources causes the pressure field to deform.
Near the positive source:

- pressure increases,
- contours bulge outward,
- local maxima develop.

Near the negative source:

- pressure decreases,
- contours curve inward,
- local minima develop.

The resulting surface reflects the combined influence of both forcing locations.

17.9 Comparison with Laplace's Equation

Laplace's equation may be viewed as a special case of Poisson's equation:

$$\nabla^2 p = 0.$$

The addition of the source term introduces internal forcing.
Consequently,

Laplace Equation	Poisson Equation
No source term	Contains source term
Boundary-driven solution	Boundary and source driven solution
Pure equilibrium	Forced equilibrium
Smoother solutions	More complex solution structures

17.10 Connection to CFD

Poisson's equation plays a central role in incompressible flow solvers.

In many CFD algorithms, the pressure field is obtained by solving a pressure-Poisson equation of the form

$$\nabla^2 p = b, \tag{17.6}$$

where the source term depends on the velocity field.

This equation enforces mass conservation and ensures that the velocity field remains divergence-free.

Consequently, efficient Poisson solvers are among the most important components of practical CFD codes.

17.11 Key Lessons from Problem 11

This chapter introduced Poisson's equation, one of the most important elliptic PDEs in numerical modelling.

The main conclusions are:

- Poisson's equation extends Laplace's equation through the addition of a source term;
- internal forcing influences the solution throughout the domain;
- source terms generate local maxima and minima;
- iterative methods provide an effective numerical solution strategy;
- Poisson's equation forms the mathematical foundation of pressure-correction methods in CFD.

The next chapter builds directly on these concepts by introducing the cavity-flow problem, where the pressure-Poisson equation is coupled with the momentum equations to solve the incompressible Navier–Stokes equations.

Chapter 18

Pressure Poisson Equation

18.1 Introduction

In the previous chapter, we studied Poisson's equation with prescribed source terms. In Computational Fluid Dynamics, a particularly important application of Poisson's equation is the pressure Poisson equation.

The pressure Poisson equation plays a central role in incompressible flow solvers. Rather than prescribing pressure directly, the pressure field is computed from the velocity field in such a way that mass conservation is satisfied throughout the domain.

This chapter develops a MATLAB implementation of the pressure Poisson equation and demonstrates how pressure is related to spatial variations in velocity.

18.2 Governing Equation

The pressure Poisson equation is

$$\nabla^2 p = b, \tag{18.1}$$

where

$$p(x, y)$$

is the pressure field and

$$b(x, y)$$

is a source term constructed from velocity gradients.

Unlike the Poisson equation of the previous chapter, the source term is not prescribed arbitrarily. Instead, it is computed from the velocity field.

18.3 Source Term

A commonly used source term is

$$b = \rho \left[\frac{1}{\Delta t} \left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \right) - \left(\frac{\partial u}{\partial x} \right)^2 - 2 \frac{\partial u}{\partial y} \frac{\partial v}{\partial x} - \left(\frac{\partial v}{\partial y} \right)^2 \right], \tag{18.2}$$

where

- $u(x, y)$ is the horizontal velocity,
- $v(x, y)$ is the vertical velocity,

- $p(x, y)$ is the pressure,
- ρ is the density.

The source term measures how strongly the velocity field violates incompressibility.
The pressure field subsequently acts to correct these violations.

18.4 Physical Interpretation

The pressure Poisson equation provides a mechanism for enforcing mass conservation.

When the velocity field develops regions of local compression or expansion, the source term becomes nonzero.

The pressure field responds by generating pressure gradients that redistribute momentum and restore incompressibility.

Consequently:

- pressure is not prescribed directly,
- pressure depends on velocity gradients,
- pressure acts as a constraint force,
- pressure enforces continuity.

This is fundamentally different from compressible flow, where pressure evolves through thermodynamic relations.

18.5 Numerical Discretisation

The Laplacian operator is approximated using central differences:

$$\frac{\partial^2 p}{\partial x^2} \approx \frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2}, \tag{18.3}$$

$$\frac{\partial^2 p}{\partial y^2} \approx \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2}. \tag{18.4}$$

Substituting into the governing equation yields

$$p_{i,j} = \frac{(p_{i+1,j} + p_{i-1,j})\Delta y^2 + (p_{i,j+1} + p_{i,j-1})\Delta x^2 - b_{i,j}\Delta x^2 \Delta y^2}{2(\Delta x^2 + \Delta y^2)}. \tag{18.5}$$

This equation is solved iteratively until the pressure distribution converges.

18.6 Code Walkthrough

18.6.1 Generating the Computational Grid

The computational domain is created using

```
nx = 81;
ny = 81;

Lx = 2.0;
Ly = 2.0;

dx = Lx/(nx-1);
dy = Ly/(ny-1);
```

The domain extends over

$$0 \leq x \leq 2, \qquad 0 \leq y \leq 2.$$

18.6.2 Generating Coordinate Arrays

The coordinate vectors are generated using

```
x = linspace(0,Lx,nx);
y = linspace(0,Ly,ny);

[X,Y] = meshgrid(x,y);
```

The arrays `X` and `Y` contain the coordinates of every point in the domain.

18.6.3 Defining Physical Parameters

The physical parameters are

```
rho = 1.0;
dt = 0.001;
```

The density and timestep appear explicitly within the source-term calculation.

18.6.4 Defining the Velocity Fields

For this example, analytical velocity fields are prescribed:

```
u = sin(pi*X).*sin(pi*Y);

v = cos(pi*X).*cos(pi*Y);
```

These velocity fields contain spatial gradients that generate a nontrivial pressure source term.

They are not obtained from a flow solver but are specified directly for demonstration purposes.

18.6.5 Initialising the Pressure Field

The pressure field is initialised as

```
p = zeros(ny,nx);
```

Initially, the pressure is zero throughout the domain.

The numerical procedure subsequently determines the pressure distribution implied by the velocity gradients.

18.6.6 Initialising the Source Array

The source term array is created using

```
b = zeros(ny,nx);
```

Each entry of this array will be computed from local velocity derivatives.

18.6.7 Computing Velocity Derivatives

The velocity gradients are approximated using central differences:

```
dudx = (u(j,i+1)-u(j,i-1))/(2*dx);

dudy = (u(j+1,i)-u(j-1,i))/(2*dy);

dvdx = (v(j,i+1)-v(j,i-1))/(2*dx);

dvdy = (v(j+1,i)-v(j-1,i))/(2*dy);
```

These gradients measure how the velocity field changes throughout the domain.

18.6.8 Constructing the Source Term

The source term is evaluated using

```
b(j,i) = rho*((1/dt)*(dudx+dvdy) ...
       - dudx^2 ...
       - 2*dudy*dvdx ...
       - dvdy^2);
```

This implements the finite-difference approximation of the pressure source equation.
The resulting source field contains information about the local structure of the velocity field.

18.6.9 Defining the Number of Iterations

The pressure equation is solved iteratively:

```
nit = 500;
```

A larger value typically produces a more accurate solution.

18.6.10 Beginning the Iterative Procedure

The iterative loop is

```
for k = 1:nit
```

A copy of the previous pressure field is stored using

```
pn = p;
```

18.6.11 Updating the Interior Points

The pressure field is updated using

```
p(j,i) = ...
((pn(j,i+1)+pn(j,i-1))*dy^2 ...
+(pn(j+1,i)+pn(j-1,i))*dx^2 ...
- b(j,i)*dx^2*dy^2) ...
/(2*(dx^2+dy^2));
```

This is identical in form to the Poisson equation solver from the previous chapter.
The difference lies in the construction of the source term.

18.6.12 Applying Boundary Conditions

The pressure boundary conditions are

```
p(:,1) = p(:,2);

p(:,end) = p(:,end-1);

p(1,:) = p(2,:);

p(end,:) = 0.0;
```

These conditions correspond to zero pressure gradient on three boundaries and a fixed pressure value along the upper boundary.

18.6.13 Visualising the Pressure Field

The final pressure distribution is displayed using

```
contourf(X,Y,p,30)

colorbar
```

The plot is labelled using

```
xlabel('x')

ylabel('y')

title('Pressure Poisson Equation')
```

The resulting contour map reveals how pressure varies throughout the domain.

18.7 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the computational grid.
2. Define velocity fields.
3. Compute velocity gradients.
4. Construct the source term.
5. Initialise the pressure field.
6. Solve the pressure Poisson equation iteratively.
7. Apply boundary conditions.
8. Visualise the resulting pressure field.

18.8 Expected Behaviour

The pressure distribution reflects the structure of the velocity gradients.

Regions with strong velocity variation generate stronger source terms and therefore larger pressure variations.

The resulting pressure field is smooth because the Poisson equation naturally distributes information throughout the domain.

Typical observations include:

- pressure maxima and minima,
- smooth pressure contours,
- pressure variations associated with velocity gradients,
- a global pressure field rather than purely local effects.

18.9 Why Pressure Depends on Velocity

For incompressible flow, pressure is not an independent variable.

Instead, pressure adjusts itself so that the continuity equation

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0 \tag{18.6}$$

remains satisfied.

The pressure Poisson equation therefore provides the mathematical link between velocity and pressure.

Without this step, the velocity field would generally violate mass conservation.

18.10 Connection to Navier–Stokes Solvers

Most incompressible Navier–Stokes algorithms follow the sequence

1. predict an intermediate velocity field,
2. compute the pressure source term,
3. solve the pressure Poisson equation,
4. correct the velocity field,
5. enforce incompressibility.

The pressure Poisson equation is therefore one of the most important numerical components of modern CFD software.

18.11 Key Lessons from Problem 12

This chapter introduced the pressure Poisson equation.

The main conclusions are:

- pressure is computed from velocity gradients rather than prescribed directly;
- the source term measures local deviations from incompressibility;
- the pressure field acts to enforce mass conservation;
- pressure information propagates globally through the Poisson equation;
- pressure Poisson solvers form the foundation of incompressible Navier–Stokes methods.

The next chapter combines the pressure Poisson equation with the momentum equations to solve the lid-driven cavity flow problem, providing the first complete incompressible Navier–Stokes solver in the workbook.

Chapter 19

Lid-Driven Cavity Flow

19.1 Introduction

The previous chapter introduced the pressure Poisson equation and demonstrated how pressure can be computed from velocity gradients.

In this chapter, the pressure Poisson equation is coupled with the momentum equations to solve the incompressible Navier–Stokes equations.

The resulting problem is the lid-driven cavity flow, one of the most important benchmark problems in Computational Fluid Dynamics.

The geometry consists of a square cavity filled with fluid. The upper wall moves at a constant speed while the remaining walls remain stationary.

Although the geometry is simple, the resulting flow exhibits many important fluid-dynamics phenomena:

- nonlinear convection,
- viscous diffusion,
- pressure gradients,
- boundary-layer formation,
- recirculation zones,
- incompressibility constraints.

For these reasons, lid-driven cavity flow is widely used for validating CFD solvers.

19.2 Governing Equations

The two-dimensional incompressible Navier–Stokes equations are

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} + v\frac{\partial u}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial x} + \nu\nabla^2 u, \tag{19.1}$$

$$\frac{\partial v}{\partial t} + u\frac{\partial v}{\partial x} + v\frac{\partial v}{\partial y} = -\frac{1}{\rho}\frac{\partial p}{\partial y} + \nu\nabla^2 v, \tag{19.2}$$

where

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}.$$

The incompressibility condition is

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0. \tag{19.3}$$

The governing variables are

- $u(x, y, t)$: horizontal velocity,
- $v(x, y, t)$: vertical velocity,
- $p(x, y, t)$: pressure,
- ρ: density,
- ν: kinematic viscosity.

19.3 Physical Interpretation

The momentum equations contain three distinct physical mechanisms:

- convection transports momentum through the domain,
- pressure gradients redistribute momentum,
- diffusion smooths velocity gradients.

The continuity equation ensures conservation of mass.
Together, these equations describe the motion of incompressible fluids.

19.4 Boundary Conditions

The computational domain is

$$0 \le x \le 2, \qquad 0 \le y \le 2.$$

The moving lid is located at the upper boundary.
The velocity boundary conditions are

$$u = 1, \qquad v = 0, \qquad \text{top wall}, \tag{19.4}$$

and

$$u = 0, \qquad v = 0, \qquad \text{all remaining walls}. \tag{19.5}$$

The motion of the top wall drives the flow throughout the cavity.

19.5 Code Walkthrough

19.5.1 Generating the Computational Grid

The computational mesh is defined using

```
nx = 41;
ny = 41;

Lx = 2.0;
Ly = 2.0;

dx = Lx/(nx-1);
dy = Ly/(ny-1);
```

The domain therefore contains

$$41 \times 41$$

computational nodes.

19.5.2 Generating Coordinate Arrays

The coordinate vectors are generated using

```
x = linspace(0,Lx,nx);
y = linspace(0,Ly,ny);

[X,Y] = meshgrid(x,y);
```

These arrays are later used for visualisation.

19.5.3 Defining Physical Parameters

The fluid properties are

```
rho = 1.0;
nu = 0.1;

dt = 0.001;

nt = 500;
nit = 50;
```

The parameters represent

- fluid density,
- kinematic viscosity,
- timestep,
- number of timesteps,
- number of pressure iterations.

19.5.4 Initialising the Solution Fields

The velocity and pressure fields are initialised using

```
u_vel = zeros(ny,nx);

v_vel = zeros(ny,nx);

p = zeros(ny,nx);

b = zeros(ny,nx);
```

Initially, the fluid is stationary and the pressure field is uniform.

19.5.5 Beginning the Time Integration

The simulation advances through time using

```
for n = 1:nt
```

Copies of the previous solution are stored:

```
un = u_vel;

vn = v_vel;
```

These arrays are required because all updates must use data from the previous timestep.

19.5.6 Constructing the Pressure Source Term

The pressure source term is computed from velocity gradients.

The gradient calculations are

```
dudx = (un(j,i+1)-un(j,i-1))/(2*dx);

dudy = (un(j+1,i)-un(j-1,i))/(2*dy);

dvdx = (vn(j,i+1)-vn(j,i-1))/(2*dx);

dvdy = (vn(j+1,i)-vn(j-1,i))/(2*dy);
```

The source term becomes

```
b(j,i) = rho*((1/dt)*(dudx+dvdy) ...
       - dudx^2 ...
       - 2*dudy*dvdx ...
       - dvdy^2);
```

This source term is used in the pressure Poisson equation.

19.5.7 Solving the Pressure Poisson Equation

The pressure field is computed iteratively:

```
for k = 1:nit

    pn = p;

    p(j,i) = ...
    ((pn(j,i+1)+pn(j,i-1))*dy^2 ...
    +(pn(j+1,i)+pn(j-1,i))*dx^2 ...
    - b(j,i)*dx^2*dy^2) ...
    /(2*(dx^2+dy^2));

end
```

The pressure solution enforces approximate incompressibility.

19.5.8 Pressure Boundary Conditions

The pressure boundary conditions are

```
p(:,1) = p(:,2);

p(:,end) = p(:,end-1);

p(1,:) = p(2,:);

p(end,:) = 0.0;
```

These conditions are applied after every pressure iteration.

19.5.9 Updating the Horizontal Velocity

The u-velocity equation is

```
u_vel(j,i) = un(j,i) ...
          - un(j,i)*dt/dx ...
          *(un(j,i)-un(j,i-1)) ...
          - vn(j,i)*dt/dy ...
          *(un(j,i)-un(j-1,i)) ...
          - dt/(2*rho*dx) ...
          *(p(j,i+1)-p(j,i-1)) ...
          + nu*dt/dx^2 ...
          *(un(j,i+1)-2*un(j,i)+un(j,i-1)) ...
          + nu*dt/dy^2 ...
          *(un(j+1,i)-2*un(j,i)+un(j-1,i));
```

The various terms represent

- convection,
- pressure forces,
- viscous diffusion.

19.5.10 Updating the Vertical Velocity

The v-velocity field is updated using

```
v_vel(j,i) = vn(j,i) ...
          - un(j,i)*dt/dx ...
          *(vn(j,i)-vn(j,i-1)) ...
          - vn(j,i)*dt/dy ...
          *(vn(j,i)-vn(j-1,i)) ...
          - dt/(2*rho*dy) ...
          *(p(j+1,i)-p(j-1,i)) ...
          + nu*dt/dx^2 ...
          *(vn(j,i+1)-2*vn(j,i)+vn(j,i-1)) ...
          + nu*dt/dy^2 ...
          *(vn(j+1,i)-2*vn(j,i)+vn(j-1,i));
```

This equation has the same structure as the u-equation but governs the vertical velocity component.

19.5.11 Applying Velocity Boundary Conditions

The no-slip conditions are enforced using

```
u_vel(1,:) = 0.0;
u_vel(:,1) = 0.0;
u_vel(:,end) = 0.0;

u_vel(end,:) = 1.0;

v_vel(1,:) = 0.0;
v_vel(end,:) = 0.0;
v_vel(:,1) = 0.0;
v_vel(:,end) = 0.0;
```

The top boundary moves with unit velocity while all other boundaries remain stationary.

19.5.12 Visualising Pressure and Velocity

The pressure field is displayed using

```
contourf(X,Y,p,30)

colorbar
```

Velocity vectors are plotted using

```
quiver(X,Y,u_vel,v_vel,'k')
```

The figure is labelled using

```
xlabel('x')

ylabel('y')

title('Lid-Driven Cavity: Pressure and Velocity')
```

19.5.13 Visualising Streamlines

The streamline pattern is generated using

```
streamslice(X,Y,u_vel,v_vel)
```

The streamlines clearly reveal the recirculating motion within the cavity.

19.6 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the computational grid.
2. Initialise pressure and velocity fields.
3. Compute the pressure source term.
4. Solve the pressure Poisson equation.
5. Update velocity components.
6. Apply boundary conditions.

7. Repeat for all timesteps.
8. Plot pressure contours.
9. Plot velocity vectors and streamlines.

19.7 Expected Behaviour

The moving lid transfers momentum into the fluid.

Initially the fluid is stationary, but the motion of the lid causes the velocity field to develop throughout the cavity.

As the simulation progresses:

- fluid near the lid moves horizontally,
- momentum diffuses downward,
- pressure gradients develop,
- a primary recirculation zone forms,
- the solution approaches a steady state.

The dominant feature is a large vortex occupying much of the cavity interior.

19.8 Role of Pressure

Pressure is responsible for maintaining incompressibility.

Without the pressure correction step:

- the continuity equation would not be satisfied,
- the velocity field would develop nonphysical divergence,
- mass conservation would be violated.

The pressure Poisson equation therefore acts as the mechanism that couples velocity and pressure throughout the domain.

19.9 Importance as a CFD Benchmark

The lid-driven cavity problem remains one of the most widely used verification tests in CFD because it contains:

- nonlinear convection,
- diffusion,
- pressure coupling,
- incompressibility constraints,
- nontrivial flow structures.

A successful cavity-flow simulation demonstrates that all major parts of an incompressible Navier–Stokes solver are functioning correctly.

19.10 Key Lessons from Problem 13

This chapter introduced the first complete incompressible Navier–Stokes solver in the workbook.

The main conclusions are:

- pressure and velocity are strongly coupled;
- the pressure Poisson equation enforces incompressibility;
- moving boundaries can generate complex flow patterns;
- convection, diffusion, and pressure all contribute to fluid motion;
- lid-driven cavity flow provides a standard benchmark for CFD verification.

The next chapter investigates the influence of Reynolds number on cavity flow and examines how viscosity affects the resulting flow structures.

includechapter20

Chapter 20

Assessment of Divergence Error in Incompressible Flow

20.1 Introduction

In the previous chapters, we developed a numerical solver for the incompressible Navier–Stokes equations and investigated the effects of Reynolds number on cavity flow.

A key requirement of incompressible flow is that the velocity field must satisfy the continuity equation at every point in the computational domain.

In practice, numerical errors prevent the continuity equation from being satisfied exactly. Therefore, it is essential to evaluate the divergence of the computed velocity field and determine how closely the solution satisfies the incompressibility constraint.

This chapter introduces the concept of divergence error and demonstrates how it can be used as a diagnostic tool for validating incompressible flow simulations.

20.2 Incompressibility Condition

The continuity equation for incompressible flow is

$$\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} = 0. \tag{20.1}$$

The quantity

$$\nabla \cdot \mathbf{u} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y} \tag{20.2}$$

is known as the divergence of the velocity field.

For a perfectly incompressible flow,

$$\boxed{\nabla \cdot \mathbf{u} = 0.} \tag{20.3}$$

Any deviation from zero indicates a violation of mass conservation.

20.3 Physical Interpretation

The divergence measures whether fluid is artificially created or destroyed within a computational cell.

20.3.1 Positive Divergence

When

$$\nabla \cdot \mathbf{u} > 0,$$

more fluid leaves a region than enters it.
This corresponds to an artificial source of mass.

20.3.2 Negative Divergence

When

$$\nabla \cdot \mathbf{u} < 0,$$

more fluid enters a region than leaves it.
This corresponds to an artificial sink of mass.

20.3.3 Zero Divergence

When

$$\nabla \cdot \mathbf{u} = 0,$$

mass is conserved locally.
This is the desired result for incompressible flow.

20.4 Why Divergence Error Matters

The pressure Poisson equation was introduced specifically to enforce incompressibility.
If the pressure field is not solved accurately enough:

- mass conservation is violated,
- velocity errors accumulate,
- flow structures become nonphysical,
- numerical instability may develop.

Evaluating divergence therefore provides a direct measure of solver quality.

20.5 Numerical Discretisation

The velocity derivatives are approximated using central differences.
For the horizontal velocity derivative,

$$\frac{\partial u}{\partial x} \approx \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x}, \tag{20.4}$$

and for the vertical velocity derivative,

$$\frac{\partial v}{\partial y} \approx \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y}. \tag{20.5}$$

The divergence at each grid point becomes

$$\boxed{\nabla \cdot \mathbf{u} = \frac{u_{i+1,j} - u_{i-1,j}}{2\Delta x} + \frac{v_{i,j+1} - v_{i,j-1}}{2\Delta y}.} \tag{20.6}$$

This quantity is evaluated across the entire computational domain.

20.6 Code Walkthrough

20.6.1 Creating the Divergence Array

A new array is first allocated:

```
div = zeros(ny,nx);
```

Each element will contain the divergence computed at one grid point.

20.6.2 Computing Velocity Derivatives

The derivative of the horizontal velocity is calculated using

```
dudx = (u_vel(j,i+1) ...
     - u_vel(j,i-1)) ...
     /(2*dx);
```

This approximates

$$\frac{\partial u}{\partial x}.$$

Similarly,

```
dvdy = (v_vel(j+1,i) ...
     - v_vel(j-1,i)) ...
     /(2*dy);
```

approximates

$$\frac{\partial v}{\partial y}.$$

20.6.3 Computing the Divergence

The divergence is computed as

```
div(j,i) = dudx + dvdy;
```

This directly implements

$$\nabla \cdot \mathbf{u} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}.$$

The procedure is repeated for every interior grid point.

20.6.4 Loop Structure

The calculation is performed using

```
for j = 2:ny-1

    for i = 2:nx-1

        dudx = ...
        dvdy = ...

        div(j,i) = dudx + dvdy;

    end

end
```

Boundary points are excluded because neighbouring values are required by the central-difference stencil.

20.6.5 Computing a Global Error Measure

The overall divergence error is quantified using

```
div_norm = ...
sqrt(sum(sum(div.^2)));
```

This quantity is the Euclidean norm of the divergence field.
Mathematically,

$$\text{div\_norm} = \sqrt{\sum_i \sum_j (\nabla \cdot \mathbf{u})^2}. \tag{20.7}$$

A smaller value indicates better satisfaction of the continuity equation.

20.6.6 Visualising the Divergence Field

The divergence is displayed using

```
contourf(X,Y,div,30)

colorbar
```

The figure is labelled using

```
xlabel('x')

ylabel('y')

title('Divergence Error')
```

This contour map reveals where incompressibility errors are concentrated.

20.6.7 Displaying the Error Norm

The global error is displayed using

```
disp(['Divergence norm = ', ...
num2str(div_norm)])
```

This provides a simple quantitative measure of solution quality.

20.7 Algorithm Summary

The divergence assessment procedure follows the sequence

1. Obtain the computed velocity field.
2. Calculate velocity derivatives.
3. Compute divergence at each grid point.
4. Construct the divergence field.
5. Compute a global divergence norm.
6. Visualise the divergence distribution.
7. Assess the quality of the flow solution.

20.8 Expected Behaviour

If the pressure Poisson equation has been solved accurately,

$$\nabla \cdot \mathbf{u} \approx 0$$

throughout most of the computational domain.
Consequently:

- divergence values remain small,
- the divergence contour plot appears nearly uniform,
- the divergence norm is small,
- the velocity field satisfies incompressibility.

Small errors may still occur because of finite numerical precision and discretisation effects.

20.9 Where Errors Usually Occur

The largest divergence errors often appear near boundaries.
Several factors contribute to this behaviour:

- boundary conditions introduce asymmetry,
- derivative approximations are less accurate near walls,
- velocity gradients are typically largest near solid boundaries,
- pressure corrections may require additional iterations.

For this reason, divergence plots frequently show localized errors near cavity corners and walls.

20.10 Influence of Pressure Iterations

The number of pressure-Poisson iterations plays a major role in divergence reduction.

20.10.1 Too Few Iterations

When the pressure equation is insufficiently converged:

- divergence remains relatively large,
- pressure corrections are incomplete,
- mass conservation is poorly enforced.

20.10.2 More Iterations

Increasing the number of pressure iterations typically:

- reduces divergence,
- improves pressure accuracy,
- strengthens incompressibility enforcement.

However, additional iterations increase computational cost.

20.11 Using Divergence as a Diagnostic Tool

The divergence field is one of the most useful diagnostics in CFD.

It provides information about:

- pressure-solver performance,
- mesh quality,
- timestep suitability,
- overall numerical accuracy.

A CFD simulation should never be accepted without checking that the divergence remains sufficiently small.

20.12 Connection to CFD Practice

Professional incompressible flow solvers routinely monitor divergence as a convergence criterion.

Typical workflows involve:

1. solving the momentum equations,
2. solving the pressure Poisson equation,
3. correcting the velocity field,
4. computing divergence,
5. repeating until divergence falls below a prescribed tolerance.

Thus, divergence monitoring forms a central component of modern CFD verification and validation procedures.

20.13 Key Lessons from Problem 15

This chapter introduced divergence error as a measure of incompressibility.

The main conclusions are:

- incompressible flow requires $\nabla \cdot \mathbf{u} = 0$;
- divergence measures local violations of mass conservation;
- the pressure Poisson equation reduces divergence errors;
- divergence contours reveal where numerical errors are concentrated;
- divergence norms provide a quantitative measure of solution quality.

The next chapter introduces the compressible continuity equation, where density is allowed to vary and the incompressibility constraint is no longer applicable.

Chapter 21

Compressible Continuity Equation

21.1 Introduction

All previous fluid-flow problems in this workbook assumed incompressible flow, meaning that the density remained constant throughout the simulation.

Many real engineering flows, however, involve significant density variations. Examples include:

- gas dynamics,
- aerospace applications,
- high-speed flows,
- acoustic waves,
- shock-wave propagation.

In such situations, density can no longer be treated as constant and must be solved as part of the governing equations.

The starting point for compressible fluid dynamics is the continuity equation, which expresses conservation of mass.

This chapter develops a MATLAB implementation of the one-dimensional compressible continuity equation and demonstrates how density evolves under a prescribed velocity field.

21.2 Governing Equation

The one-dimensional compressible continuity equation is

$$\frac{\partial \rho}{\partial t} + \frac{\partial (\rho u)}{\partial x} = 0, \tag{21.1}$$

where

- $\rho(x,t)$ is the density,
- $u(x,t)$ is the velocity,
- ρu is the mass flux.

This equation states that mass cannot be created or destroyed.

Any change in density must result from mass entering or leaving a region.

21.3 Physical Interpretation

The quantity

$$\rho u$$

represents the rate at which mass flows through a unit area.
The continuity equation therefore states that:

- if more mass enters a region than leaves it, density increases,
- if more mass leaves a region than enters it, density decreases,
- if incoming and outgoing mass fluxes are equal, density remains unchanged.

Unlike incompressible flow, the density field evolves with time.

21.4 Comparison with Incompressible Flow

For incompressible flow,

$$\rho = \text{constant}.$$

The continuity equation reduces to

$$\frac{\partial u}{\partial x} = 0 \tag{21.2}$$

in one dimension.
For compressible flow, density is no longer constant and must be updated at each timestep using the continuity equation.
This represents a major conceptual change from the incompressible flow problems studied previously.

21.5 Initial Condition

The density field is initialised as

$$\rho(x,0) = \begin{cases} 2, & 0.4 \le x \le 0.6, \\ 1, & \text{otherwise.} \end{cases} \tag{21.3}$$

This creates a region of high density embedded within a uniform background.
The velocity field is prescribed as

$$u = 1.$$

The density pulse is therefore transported across the domain by the velocity field.

21.6 Numerical Discretisation

Using a forward difference in time,

$$\frac{\partial \rho}{\partial t} \approx \frac{\rho_i^{n+1} - \rho_i^n}{\Delta t}, \tag{21.4}$$

and a backward difference for the flux derivative,

$$\frac{\partial (\rho u)}{\partial x} \approx \frac{(\rho u)_i^n - (\rho u)_{i-1}^n}{\Delta x}, \tag{21.5}$$

gives

$$\rho_i^{n+1} = \rho_i^n - \frac{\Delta t}{\Delta x} \left[(\rho u)_i^n - (\rho u)_{i-1}^n \right]. \tag{21.6}$$

This update equation forms the basis of the MATLAB implementation.

21.7 Code Walkthrough

21.7.1 Generating the Computational Grid

The computational domain is defined using

```
nx = 101;

L = 1.0;

dx = L/(nx-1);

x = linspace(0,L,nx);
```

The domain extends over

$$0 \leq x \leq 1.$$

The variable `dx` stores the grid spacing.

21.7.2 Defining Simulation Parameters

The simulation parameters are

```
nt = 100;

dt = 0.001;
```

where

- `nt` is the number of timesteps,
- `dt` is the timestep size.

21.7.3 Initialising Density and Velocity

The density and velocity fields are initialised as

```
rho = ones(1,nx);

u = ones(1,nx);
```

Initially,

$$\rho = 1, \qquad u = 1$$

throughout the domain.

21.7.4 Creating the Density Pulse

The initial density disturbance is introduced using

```
for i = 1:nx

    if x(i) >= 0.4 && x(i) <= 0.6

        rho(i) = 2.0;

    end

end
```

The density is therefore doubled within the central region.

21.7.5 Saving the Initial Condition

The initial density field is stored using

```
rho_initial = rho;
```

This allows comparison between the initial and final solutions.

21.7.6 Beginning the Time Integration

The numerical solution advances using

```
for n = 1:nt
```

At the beginning of each timestep, a copy of the density field is created:

```
rhon = rho;
```

21.7.7 Computing the Mass Flux

The mass flux is calculated using

```
flux = rhon.*u;
```

This quantity represents

$$\rho u.$$

The continuity equation evolves density according to changes in this flux.

21.7.8 Updating the Density Field

The density update is implemented using

```
for i = 2:nx

    rho(i) = rhon(i) ...
           - dt/dx ...
           *(flux(i)-flux(i-1));

end
```

This directly implements

$$\rho_i^{n+1} = \rho_i^n - \frac{\Delta t}{\Delta x}\left[(\rho u)_i^n - (\rho u)_{i-1}^n\right]. \tag{21.7}$$

The density changes according to the difference between neighbouring mass fluxes.

21.7.9 Applying the Boundary Condition

A fixed density is maintained at the left boundary:

```
rho(1) = 1.0;
```

This prevents undefined values at the domain boundary.

21.7.10 Visualising the Solution

The density field is plotted using

```
plot(x,rho_initial,'--','LineWidth',2)

hold on

plot(x,rho,'LineWidth',2)
```

The figure is labelled using

```
xlabel('x')

ylabel('\rho')

legend('Initial','Final')

title('Compressible Continuity Equation')

grid on
```

The resulting graph shows how the density field evolves.

21.8 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the computational grid.
2. Initialise density and velocity fields.
3. Create the density pulse.
4. Compute the mass flux.

5. Update the density field.
6. Apply boundary conditions.
7. Repeat for all timesteps.
8. Plot the initial and final density profiles.

21.9 Expected Behaviour

Because the velocity field is positive,

$$u > 0,$$

the density pulse moves toward increasing x.

The solution behaves similarly to a convection equation, but the transported quantity is now density.

Consequently:

- the density pulse propagates through the domain,
- mass is conserved,
- the density field evolves with time,
- numerical diffusion may slightly smooth the pulse.

The final density profile resembles a transported version of the initial pulse.

21.10 Mass Conservation

The continuity equation expresses conservation of mass.

A useful diagnostic is the total mass within the domain:

$$M = \int \rho \, dx. \tag{21.8}$$

In a well-behaved numerical simulation, the total mass should remain approximately constant.

Small changes may occur because of discretisation errors and boundary conditions.

21.11 Connection to Compressible Flow

The continuity equation is the first of three equations required for compressible flow simulations:

- continuity equation,
- momentum equation,
- energy equation.

Together, these equations form the Euler equations and the Navier–Stokes equations for compressible fluids.

The continuity equation therefore provides the foundation for all compressible CFD methods.

21.12 Differences from Incompressible Flow

Compared with incompressible flow:

Incompressible Flow	Compressible Flow
Density constant	Density varies
Continuity enforces divergence-free velocity	Continuity evolves density
Pressure enforces mass conservation	Density becomes an unknown variable
Fewer conserved variables	Additional variables required

Understanding this difference is essential when moving from incompressible CFD to compressible CFD.

21.13 Key Lessons from Problem 16

This chapter introduced the compressible continuity equation.

The main conclusions are:

- density becomes a time-dependent variable;
- the continuity equation expresses conservation of mass;
- mass flux drives density evolution;
- density disturbances propagate through the domain;
- the continuity equation forms the first component of the compressible flow equations.

The next chapter extends these ideas by introducing the one-dimensional compressible Euler equations, where density, momentum, and energy are solved simultaneously.

Chapter 22

One-Dimensional Compressible Euler Equations

22.1 Introduction

The previous chapter introduced the compressible continuity equation, where density was allowed to vary with time.

In practical compressible-flow simulations, however, density alone is not sufficient to describe the fluid. Momentum and energy must also be evolved simultaneously.

The governing equations for inviscid compressible flow are known as the Euler equations.

These equations form the foundation of many areas of Computational Fluid Dynamics, including:

- high-speed aerodynamics,
- gas dynamics,
- shock-wave propagation,
- supersonic flow,
- aerospace engineering.

This chapter develops a MATLAB implementation of the one-dimensional compressible Euler equations and introduces the conservative-variable formulation used in modern CFD solvers.

22.2 Governing Equations

The one-dimensional compressible Euler equations are

$$\frac{\partial}{\partial t}\begin{bmatrix}\rho\\ \rho u\\ E\end{bmatrix}+\frac{\partial}{\partial x}\begin{bmatrix}\rho u\\ \rho u^2+p\\ u(E+p)\end{bmatrix}=0, \tag{22.1}$$

where

- ρ is density,
- u is velocity,
- p is pressure,
- E is total energy density.

The conserved-variable vector is

$$\mathbf{q} = \begin{bmatrix} \rho \\ \rho u \\ E \end{bmatrix}. \tag{22.2}$$

Modern compressible-flow solvers are almost always formulated in terms of these conserved quantities.

22.3 Conservative Variables

The three conserved variables have clear physical meanings.

22.3.1 Density

$$\rho$$

represents mass per unit volume.

22.3.2 Momentum

$$\rho u$$

represents momentum per unit volume.

22.3.3 Total Energy

$$E$$

contains both internal and kinetic energy.

These quantities remain conserved when discontinuities such as shocks develop.

22.4 Equation of State

The Euler equations require an additional relationship connecting pressure to the conserved variables.

For an ideal gas,

$$p = (\gamma - 1)\left(E - \frac{1}{2}\rho u^2\right), \tag{22.3}$$

where

$$\gamma$$

is the ratio of specific heats.

For air,

$$\gamma \approx 1.4.$$

This equation of state allows pressure to be computed from the conserved variables.

22.5 Initial Condition

The simulation begins with a discontinuous initial condition:

$$x < 0.5$$

contains

$$\rho = 1, \qquad u = 0, \qquad p = 1,$$

while

$$x \geq 0.5$$

contains

$$\rho = 0.125, \qquad u = 0, \qquad p = 0.1.$$

This initial state resembles the beginning of a shock-tube problem.
The large pressure and density differences generate strong wave motion.

22.6 Numerical Method

The Euler equations are written in conservation form:

$$\frac{\partial \mathbf{q}}{\partial t} + \frac{\partial \mathbf{F}}{\partial x} = 0, \tag{22.4}$$

where

$$\mathbf{F}$$

is the flux vector.
A finite-volume style update gives

$$\mathbf{q}_i^{n+1} = \mathbf{q}_i^n - \frac{\Delta t}{\Delta x}\left(\mathbf{F}_{i+1/2} - \mathbf{F}_{i-1/2}\right). \tag{22.5}$$

The numerical challenge lies in computing the interface fluxes.

22.7 Code Walkthrough

22.7.1 Generating the Computational Grid

The computational domain is created using

```
nx = 201;

L = 1.0;

dx = L/(nx-1);

x = linspace(0,L,nx);
```

The domain extends over

$$0 \leq x \leq 1.$$

The relatively fine mesh helps capture emerging wave structures.

22.7.2 Defining Gas Properties

The ratio of specific heats is defined as

```
gamma = 1.4;
```

This value is commonly used for air.

22.7.3 Defining Time Integration Parameters

The simulation parameters are

```
nt = 200;

CFL = 0.5;
```

The timestep is computed dynamically using the CFL condition.

22.7.4 Creating the Conservative Variables

The conserved-variable array is defined as

```
q = zeros(3,nx);
```

The three rows correspond to

- density,
- momentum,
- energy.

22.7.5 Constructing the Initial Condition

The discontinuous initial state is generated using

```
if x(i) < 0.5

    rho = 1.0;
    u = 0.0;
    p = 1.0;

else

    rho = 0.125;
    u = 0.0;
    p = 0.1;

end
```

This creates a strong pressure and density jump.

22.7.6 Computing the Energy

The total energy is calculated using

```
E = p/(gamma-1) ...
  + 0.5*rho*u^2;
```

The first term is internal energy.
The second term is kinetic energy.

22.7.7 Storing the Conservative Variables

The initial state is stored using

```
q(1,i) = rho;

q(2,i) = rho*u;

q(3,i) = E;
```

These variables are the quantities that will be updated throughout the simulation.

22.7.8 Recovering Primitive Variables

At every timestep, the primitive variables are recovered from the conserved variables:

```
rho = q(1,:);

u = q(2,:)./rho;

E = q(3,:);
```

Pressure is then computed using

```
p = (gamma-1) ...
   *(E - 0.5*rho.*u.^2);
```

22.7.9 Computing the Sound Speed

The local sound speed is

```
a = sqrt(gamma*p./rho);
```

This quantity determines the maximum propagation speed of information.

22.7.10 Computing the Timestep

The CFL timestep is

```
max_speed = max(abs(u)+a);

dt = CFL*dx/max_speed;
```

This ensures numerical stability.

22.7.11 Constructing the Flux Vector

The Euler fluxes are

```
F(1,:) = rho.*u;

F(2,:) = rho.*u.^2 + p;

F(3,:) = u.*(E+p);
```

These represent

- mass flux,
- momentum flux,
- energy flux.

22.7.12 Computing Interface Fluxes

The numerical method constructs fluxes at cell interfaces:

```
F_half(:,i) = ...
0.5*(FL+FR) ...
- 0.5*smax*(qR-qL);
```

This is a local Lax–Friedrichs (Rusanov) flux.
The parameter

$$s_{\max}$$

represents the maximum wave speed across the interface.
The dissipation term improves stability when discontinuities are present.

22.7.13 Updating the Conservative Variables

The update equation is

```
q(:,i) = qn(:,i) ...
       - dt/dx ...
       *(F_half(:,i)-F_half(:,i-1));
```

This directly implements conservation of mass, momentum, and energy.

22.7.14 Applying Boundary Conditions

Simple extrapolation is used:

```
q(:,1) = q(:,2);

q(:,end) = q(:,end-1);
```

These conditions minimise artificial reflections at the boundaries.

22.7.15 Visualising the Solution

After the simulation, the primitive variables are reconstructed and plotted.
Density:

```
plot(x,rho,'LineWidth',2)
```

Velocity:

```
plot(x,u,'LineWidth',2)
```

Pressure:

```
plot(x,p,'LineWidth',2)
```

The resulting figures illustrate the evolving flow field.

22.8 Algorithm Summary

The numerical procedure follows the sequence

1. Generate the computational grid.
2. Initialise density, velocity, and pressure.

3. Construct conservative variables.
4. Recover primitive variables.
5. Compute CFL timestep.
6. Construct physical fluxes.
7. Compute interface fluxes.
8. Update conservative variables.
9. Apply boundary conditions.
10. Repeat for all timesteps.
11. Plot density, velocity, and pressure.

22.9 Expected Behaviour

The initial discontinuity immediately generates propagating waves.

As the simulation evolves:

- density changes throughout the domain,
- pressure gradients accelerate the fluid,
- velocity develops from an initially stationary state,
- wave structures emerge from the discontinuity.

These waves are the first indication of compressible-flow behaviour.

22.10 Why Conservative Variables Are Important

Compressible flows commonly contain sharp features such as

- shock waves,
- contact discontinuities,
- expansion waves.

Conservative formulations ensure that:

- mass is conserved,
- momentum is conserved,
- energy is conserved.

These conservation properties are essential for physically correct solutions.

22.11 Connection to Compressible CFD

The Euler equations form the foundation of compressible fluid mechanics.

More advanced models such as the compressible Navier–Stokes equations are obtained by adding:

- viscous stresses,
- heat conduction,
- additional transport effects.

Thus, understanding the Euler equations is a necessary step before studying realistic high-speed flows.

22.12 Key Lessons from Problem 17

This chapter introduced the one-dimensional compressible Euler equations.

The main conclusions are:

- density, momentum, and energy must be solved simultaneously;
- pressure is obtained from an equation of state;
- conservative variables ensure proper conservation laws;
- compressible flows support wave propagation;
- the Euler equations form the basis of modern compressible CFD.

The next chapter applies these equations to the classical shock-tube problem, where the formation of shocks, rarefaction waves, and contact discontinuities can be observed directly.

Chapter 23

Shock-Tube Problem

23.1 Introduction

The previous chapter introduced the one-dimensional compressible Euler equations and demonstrated how density, momentum, and energy can be evolved simultaneously using a conservative numerical scheme.

One of the most important benchmark problems in compressible fluid dynamics is the shock-tube problem.

The shock tube was originally proposed by Ludwig Prandtl and remains a standard test for validating compressible-flow solvers.

The problem begins with two regions of gas separated by a diaphragm. One side contains fluid at high pressure while the other contains fluid at low pressure.

When the diaphragm is removed, several wave structures emerge:

- a rarefaction wave,
- a contact discontinuity,
- a shock wave.

These features provide an excellent test of a numerical method's ability to capture strong gradients and discontinuities.

23.2 Governing Equations

The one-dimensional compressible Euler equations are

$$\frac{\partial}{\partial t}\begin{bmatrix}\rho\\ \rho u\\ E\end{bmatrix}+\frac{\partial}{\partial x}\begin{bmatrix}\rho u\\ \rho u^2+p\\ u(E+p)\end{bmatrix}=0. \tag{23.1}$$

The conserved variables are

$$\mathbf{q}=\begin{bmatrix}\rho\\ \rho u\\ E\end{bmatrix}, \tag{23.2}$$

where

- ρ is density,
- u is velocity,

- p is pressure,
- E is total energy density.

The pressure is obtained from the ideal-gas equation of state

$$p = (\gamma - 1)\left(E - \frac{1}{2}\rho u^2\right). \tag{23.3}$$

23.3 Shock-Tube Initial Condition

The domain is divided into two constant states.

23.3.1 Left State

For

$$x < 0.5,$$

the gas properties are

$$\rho_L = 1.0, \tag{23.4}$$

$$u_L = 0, \tag{23.5}$$

$$p_L = 1.0. \tag{23.6}$$

23.3.2 Right State

For

$$x > 0.5,$$

the gas properties are

$$\rho_R = 0.125, \tag{23.7}$$

$$u_R = 0, \tag{23.8}$$

$$p_R = 0.1. \tag{23.9}$$

Initially, both regions are stationary.
The large pressure difference drives the subsequent flow.

23.4 Physical Interpretation

At time

$$t = 0,$$

the diaphragm separating the two states is removed.
The high-pressure gas expands into the low-pressure region.
This generates three important wave structures.

23.4.1 Rarefaction Wave

The high-pressure fluid expands toward the left.
This expansion region is known as a rarefaction wave.
Within the rarefaction region:

- pressure decreases,
- density decreases,
- velocity increases smoothly.

Unlike a shock wave, the rarefaction wave is a continuous structure.

23.4.2 Contact Discontinuity

A contact surface forms between the two gases.
Across this discontinuity:

- density changes abruptly,
- pressure remains continuous,
- velocity remains continuous.

The contact discontinuity moves with the flow.

23.4.3 Shock Wave

A shock wave travels into the low-pressure region.
Across the shock:

- pressure rises sharply,
- density rises sharply,
- velocity changes abruptly.

The shock is the most difficult feature to capture numerically.

23.5 Code Walkthrough

23.5.1 Generating the Computational Grid

The computational domain is created using

```
nx = 201;

L = 1.0;

dx = L/(nx-1);

x = linspace(0,L,nx);
```

The domain extends over

$$0 \leq x \leq 1.$$

23.5.2 Defining Gas Properties

The ratio of specific heats is

```
gamma = 1.4;
```

This value corresponds approximately to air.

23.5.3 Defining Simulation Parameters

The CFL number is

```
CFL = 0.5;
```

The simulation advances until

```
t_final = 0.2;
```

The current simulation time is initialized as

```
t = 0.0;
```

23.5.4 Creating the Conservative Variables

The solution array is allocated using

```
q = zeros(3,nx);
```

The three rows store:

1. density,
2. momentum,
3. energy.

23.5.5 Implementing the Initial Discontinuity

The shock-tube initial condition is created using

```
if x(i) < 0.5

    rho = 1.0;
    u = 0.0;
    p = 1.0;

else

    rho = 0.125;
    u = 0.0;
    p = 0.1;

end
```

This establishes the pressure jump responsible for the wave motion.

23.5.6 Computing the Total Energy

The total energy is

```
E = p/(gamma-1) ...
  + 0.5*rho*u^2;
```

The first term is internal energy while the second term is kinetic energy.

23.5.7 Beginning the Time Integration

The simulation advances using

```
while t < t_final
```

The solution evolves until the desired final time is reached.

23.5.8 Recovering Primitive Variables

The conserved variables are converted back into primitive variables:

```
rho = q(1,:);

u = q(2,:)./rho;

E = q(3,:);
```

Pressure is then computed using

```
p = (gamma-1) ...
   *(E - 0.5*rho.*u.^2);
```

23.5.9 Computing the Local Sound Speed

The speed of sound is

```
a = sqrt(gamma*p./rho);
```

This quantity controls information propagation throughout the flow.

23.5.10 Adaptive CFL Timestep

The timestep is computed dynamically:

```
max_speed = max(abs(u)+a);

dt = CFL*dx/max_speed;
```

The timestep adjusts automatically to satisfy the stability condition.

23.5.11 Constructing the Fluxes

The Euler fluxes are

```
F(1,:) = rho.*u;

F(2,:) = rho.*u.^2 + p;

F(3,:) = u.*(E+p);
```

These represent transport of mass, momentum, and energy.

23.5.12 Computing Numerical Fluxes

Interface fluxes are constructed using a local Lax–Friedrichs flux:

```
F_half(:,i) = ...
0.5*(FL+FR) ...
- 0.5*smax*(qR-qL);
```

The wave speed estimate

$$s_{\max}$$

introduces numerical dissipation that stabilizes the scheme near discontinuities.

23.5.13 Updating the Conservative Variables

The finite-volume update is

```
q(:,i) = qn(:,i) ...
      - dt/dx ...
      *(F_half(:,i)-F_half(:,i-1));
```

This update conserves mass, momentum, and energy.

23.5.14 Applying Boundary Conditions

Simple extrapolation is used:

```
q(:,1) = q(:,2);

q(:,end) = q(:,end-1);
```

These boundary conditions minimise artificial reflections.

23.5.15 Updating the Time Variable

The simulation time is advanced using

```
t = t + dt;
```

23.5.16 Visualising the Solution

After the simulation is complete, the solution is visualised using four plots.

Density:

```
plot(x,rho,'LineWidth',2)
```

Velocity:

```
plot(x,u,'LineWidth',2)
```

Pressure:

```
plot(x,p,'LineWidth',2)
```

Energy:

```
plot(x,E,'LineWidth',2)
```

These figures clearly reveal the wave structures generated by the shock tube.

23.6 Algorithm Summary

The numerical procedure follows the sequence

1. Create the computational grid.
2. Define the shock-tube initial condition.

3. Convert primitive variables to conservative variables.
4. Compute pressure and sound speed.
5. Determine the CFL timestep.
6. Construct physical fluxes.
7. Compute interface fluxes.
8. Update the conservative variables.
9. Apply boundary conditions.
10. Advance the simulation time.
11. Repeat until the final time is reached.
12. Plot density, velocity, pressure, and energy.

23.7 Expected Behaviour

As the simulation evolves, three major wave structures appear.
A rarefaction wave expands into the high-pressure region.
A contact discontinuity propagates through the domain.
A shock wave moves into the low-pressure region.
The resulting profiles contain:

- smooth expansion regions,
- sharp density jumps,
- strong pressure gradients,
- moving discontinuities.

These are characteristic features of compressible gas dynamics.

23.8 Importance of Conservative Formulations

Shock tubes illustrate why conservation laws are essential.

Non-conservative methods often produce incorrect shock speeds and inaccurate wave structures.

Conservative finite-volume methods ensure that:

- mass is conserved,
- momentum is conserved,
- energy is conserved,
- shocks propagate correctly.

This is one of the fundamental principles of modern compressible CFD.

23.9 Connection to High-Speed Aerodynamics

Shock waves are common in many engineering applications:

- supersonic aircraft,
- rocket nozzles,
- gas turbines,
- explosions,
- atmospheric re-entry vehicles.

The shock-tube problem therefore serves as a simplified model for many realistic compressible-flow phenomena.

23.10 Key Lessons from Problem 18

This chapter introduced the classical shock-tube problem.

The main conclusions are:

- compressible flows generate multiple wave types;
- rarefaction waves, contact discontinuities, and shocks emerge naturally from the Euler equations;
- conservative variables are essential for capturing discontinuities correctly;
- adaptive CFL timesteps improve numerical stability;
- the shock tube provides one of the most important benchmark tests in compressible fluid dynamics.

With this problem, the workbook concludes the progression from simple one-dimensional convection equations to the numerical simulation of compressible gas-dynamic wave propagation using the Euler equations.

Bibliography